

Hello Generic Agent 教程

Datawhale

2026-05-06

目录

[:bullseye:] 学习目标	3
1.1 安装 Python	3
1.2 下载项目	4
1.3 安装最小依赖	4
1.4 配置 API 密钥 (mykey.py)	4
1.5 启动 GenericAgent	8
1.6 常见问题排查	10
[:memo:] 小结	12
第 2 章 浏览器能力解锁	13
[:bullseye:] 学习目标	13
2.1 安装 tmwd_cdp_bridge 插件	13
2.2 验证安装	17
2.3 实战示例	17
2.4 常见问题	20
[:memo:] 小结	22
第 3 章 基础使用：第一个小时	23
[:bullseye:] 学习目标	23
3.1 界面介绍	23
3.2 视觉能力	23
3.3 飞书 CLI	25
3.4 Everything CLI (Windows 文件搜索)	26
3.5 多轮对话与上下文	27
3.6 常见问题	28
[:memo:] 小结	29
第 4 章 记忆与 Skill 系统：让 GA 越用越强	30
[:bullseye:] 学习目标	30
4.1 Skill 与记忆概览	30
4.2 创建新 Skill	31
4.3 Skill Search：搜索外部 Skill	31
4.4 迁移已有 Skill	32
4.5 记忆系统详解	32
4.6 内置 Skill 一览	33
4.7 常见问题	34
[:memo:] 小结	35
第 5 章 聊天平台接入	36
[:bullseye:] 学习目标	36

5.1 为什么要接入 Bot	36
5.2 支持的平台一览	36
5.3 微信个人号接入	36
5.4 其他平台接入	37
5.5 常见问题	48
[:memo:] 小结	49
第 6 章 进阶功能	50
[:bullseye:] 学习目标	50
6.1 自主行动模式 (Autonomous)	50
6.2 定时任务 (Scheduled Task)	51
6.3 反射模式 (Reflect)	52
6.4 子代理模式 (Subagent)	53
6.5 Plan 模式 (复杂任务规划)	54
[:memo:] 小结	56
第 7 章 上下文信息密度——GenericAgent 的第一性原理	57
7.1 一个反直觉的事实：上下文越长，表现越差	57
7.2 GenericAgent 的核心设计约束：上下文信息密度最大化	58
7.3 四层信息密度优化：GA 的系统性解法	61
7.4 从” Prompt Engineering” 到” Context Engineering”	62
7.5 本章小结	62
第 8 章 GenericAgent 系统全貌	64
8.1 统一 Agent 循环	64
8.2 两种执行模式	65
8.3 四大核心机制预览	66
8.4 从全貌到细节：一个具体例子	68
8.5 本章小结	68
第 9 章 最小原子工具集——用 9 个工具覆盖完整能力环	69
9.1 工具膨胀的代价	69
9.2 GA 的 9 个原子工具	70
9.3 能力边界与执行纪律	72
9.4 工具组合的威力：实验印证	73
9.5 故障升级：工具执行失败时的应对	74
9.6 本章小结	74
第 10 章 分层记忆架构——让 Agent 拥有持久且有序的知识	76
10.1 为什么 Full-History Append 行不通	76
10.2 GA 的四层记忆架构	76
10.3 元记忆：记忆的管理规则	78
10.4 工作记忆：短期上下文的锚点	79
10.5 按需读取 vs 默认注入	80
10.6 本章小结	82
第 11 章 上下文截断与压缩——主动管理每一轮的信息预算	83
11.1 上下文预算的量化模型	83
11.2 四阶段压缩流水线	83
11.3 浏览器层的选择性提取	86
11.4 工具 Schema 省略优化	86
11.5 实验印证：压缩与记忆协同下的效率收敛	87
11.6 本章小结	87
第 12 章 自我进化——从” 用完即忘” 到” 越用越强”	89

12.1 经验蒸馏的动机	89
12.2 GA 的三阶段进化路径	89
12.3 进化的触发与守门	92
12.4 自主探索：让 Agent 自己”上课”	93
12.5 本章小结	95
第 13 章 涌现能力——当约束催生创造力	96
13.1 从”功能内置”到”能力涌现”	96
13.2 两种执行模式与基础原语	97
13.3 子 Agent：递归自调用	98
13.4 看门狗：反射模式的第一个应用	99
13.5 定时任务：反射模式的第二个应用	99
13.6 自主空闲行为：<10 行的持续自治	100
13.7 为什么”少即是多”	100
13.8 本章小结	101
全书回顾	101
Case 1：自动在社交软件上发消息 / 图片 / 文件	102
[:bullseye:] 学习目标	102
14.1 适用场景	102
14.2 整体流程概览	102
14.3 Step 1：激活应用窗口	103
14.4 Step 2：搜索联系人	103
14.5 Step 3：验证当前会话	103
14.6 Step 4：发送内容	104
14.7 Step 5：验证发送结果	104
14.8 实战：让 GA 读本章内容并实践	105
14.9 避坑指南	105
14.10 本章小结	106

第 1 章 安装与环境配置

学完本章，你将拥有一个可以正常对话的 GenericAgent（GA）运行环境。

[:bullseye:] 学习目标

1. 在本地安装好 Python 并下载 GenericAgent 项目代码
2. 完成 mykey.py API 密钥配置，让 GA 能连接大模型
3. 成功启动 GA 并完成第一次对话

1.1 安装 Python

GA 依赖 Python 运行，我们先把它装好。

[:warning:] 推荐 **Python 3.11 或 3.12**。不要使用 3.14（与 pywebview 等依赖不兼容）。

Windows

1. 打开下载链接：<https://www.python.org/ftp/python/3.12.10/python-3.12.10-amd64.exe>
2. 运行安装包，底部的“Add python.exe to PATH”一定要勾上，然后点击安装
3. 验证安装：按 Win + R 输入 cmd 打开终端，输入：

```
python --version
```

看到 Python 3.x.x 就说明安装成功了。

macOS

macOS 和大多数 Linux 发行版自带 Python 3，打开终端（Terminal）验证：

```
python3 --version
```

看到 Python 3.x.x（3.11 或 3.12）即可。如果版本低于 3.10，前往 [Python 官网](https://www.python.org/) 下载安装。

1. 打开下载链接：<https://www.python.org/ftp/python/3.12.10/python-3.12.10-macos11.pkg>
2. 运行安装包，一步一步跟着流程安装即可
3. 验证安装：打开系统自带终端，输入：

```
python3 --version
```

看到 Python 3.x.x 就说明安装成功了。

1.2 下载项目

我们需要把 GenericAgent 的代码下载到本地。两种方式任选其一：

方式一：下载 ZIP（推荐新手）

1. 打开 [GitHub 仓库页面](#)
2. 点绿色 **Code** 按钮 → **Download ZIP**
3. 解压到你喜欢的位置（例如 D:\GenericAgent）

方式二：Git Clone

如果已经安装了 Git，在终端执行：

```
git clone https://github.com/lsdefine/GenericAgent.git
```

1.3 安装最小依赖

打开终端，进入项目目录（运行以下命令），安装两个核心依赖：

```
# 1. cd 到下载的 GA 文件目录
```

```
cd d:      (如果你的安装地址在 D 盘，终端打开后默认在 c 盘，安装在 c 盘跳过此步骤，仅限 windows 用户)
```

```
cd "你的 GenericAgent 路径"      (示例：cd D:/Document/GenericAgent-main)
```

```
# 2. 安装最小环境依赖
```

```
pip install streamlit pywebview
```

```
# 如果你的 Python 3 对应 pip3，则用：pip3 install streamlit pywebview
```

[light_bulb] 其余依赖不用手动装——1.5 节会教你让 GA 自己把剩余的包全装好。

1.4 配置 API 密钥（mykey.py）

GA 需要连接大模型才能工作。我们通过 mykey.py 告诉它用哪个模型、怎么连。

[Japanese_symbol_for_beginner] mykey 配置最简流程（5 步搞定）

如果你是第一次接触，只需要跟着下面 5 步做：

第 1 步：复制模板文件 进入你下载的 GenericAgent 项目文件夹，找到 mykey_template.py 这个文件，复制一份，然后把副本重命名为 mykey.py。

第 2 步：编辑 mykey.py，填入你的 API 信息 用任意文本编辑器（记事本、VS Code 都行）打开 mykey.py。先找到 mykey 里你要用的模型 API 对应的格式模板，然后照着填。你会看到里面有很多 # 开头的行——这些是**注释**，程序运行时会自动忽略它们，相当于”说明书”。

填写 API 信息有两种方式，任选其一：- **方式 A**：找到对应的配置区域，把你的 API Key、地址等信息填好后，**删掉该行最前面的所有 # 号**，让这一行变成”真正的代码”。- **方式 B**：不动原来的注释，直接在文件里**另起一行**，把你的配置写上去（不加 #）。

简单来说：**有 # 的行 = 不生效，没有 # 的行 = 会生效。**

第 3 步：多个 API 怎么切换？ 如果你填了不止一个 API 配置，GA 默认使用文件里**第一个生效的配置**（即第一个没被 # 注释掉的配置）。想切换模型有两种办法：- **办法 A**：启动 GA 后，点击界面侧边栏的**切换模型按钮**，直接切。- **办法 B**：回到 mykey.py，把你想用的那组配置挪到所有生效配置的最前面，保存后重启 GA。

第 4 步：保存文件 改完以后一定要**保存**！快捷键：Windows 按 Ctrl + S，Mac 按 Cmd + S。

第 5 步：重启 GA 每次修改 mykey.py 后，都需要**关掉 GA 再重新启动**，新的配置才会生效。

下面是更详细的配置说明和各渠道的具体写法：

1. 进入项目文件夹，把 mykey_template.py 复制一份，重命名为 mykey.py
2. 用任意文本编辑器打开 mykey.py，填入你的 API 信息。**选一种填就行**，不用的配置可以删掉或留着不管

新手推荐配置：Claude 主力 + GPT 兜底

直接复制到 mykey.py，替换两个 apikey / apibase：

```
# —— 主力：Claude Opus 4.6 (CC switch 反代 (Reverse Proxy)，最常见) ——
native_claude_config0 = {
    'name': 'claude-main',                    # /llms 显示名 & mixin 引用名
    'apikey': 'sk-user-< 你的 relay-key>',      # 非 sk-ant- 前缀 → Bearer 鉴权
    'apibase': 'https://<your-cc-switch-host>/claude/office', # CC switch 端点
    'model': 'claude-opus-4-6',
    'fake_cc_system_prompt': True,            # CC 透传渠道必须置 True
    'thinking_type': 'adaptive',              # 某些渠道必须要求填 thinking_type 字段
    'max_retries': 3,
    'read_timeout': 180,
}

# —— 备选：GPT-5.4 做兜底 ——
native_oai_config = {
    'name': 'gpt-backup',
    'apikey': 'sk-< 你的 OpenAI key>',
    'apibase': 'https://api.openai.com/v1',
    'model': 'gpt-5.4',
    'reasoning_effort': 'high',
    'max_retries': 3,
    'read_timeout': 120,
}

# —— Mixin 自动切换 (Failover) ——
```

```

mixin_config = {
    'llm_nos': ['claude-main', 'gpt-backup'],
    'max_retries': 10,
    'base_delay': 0.5,
    'spring_back': 300,
}

```

每次启动 GA 默认读的是第一个 api 信息。可以点击设置切换。

[::clipboard:] 所有内置渠道一览（点击展开）

一线直连渠道（填 apikey / apibase 即用）

渠道	推荐变量名	apikey 形式	apibase	示例 model	备注
Anthropic 官方	native_claude_config_ant-xxx	sk-ant-xxx	https://api.anthropic.com	claude-opus-4-6[1m]	sk-ant- 前缀自动切 x-api-key 鉴权
OpenAI 官方	native_oai_config-sk-proj-xxx	sk-proj-xxx	https://api.openai.com/v1	gpt-5.4	支持 api_mode: 'responses'
OpenRouter	oai_config_openrouter-xxx	sk-or-xxx	https://openrouter.ai/api/v1	anthropic/claude-opus-4-6	model 用 provider/model 格式
智谱 GLM-5.1	native_claude_config_glm-xxx	glm-xxx	https://open.bigmodel.cn/api/anthropic	glm-5.1	推荐用 Anthropic 路径
MiniMax (Anthropic)	native_claude_config_minimax-xxx	sk-minimax-xxx	https://api.minimaxi.com/anthropic	MiniMax-M2.7	204K 上下文
MiniMax (OAI)	oai_config_minimaxcp-xxx	sk-cp-xxx	https://api.minimaxi.com/v1	MiniMax-M2.7	回复带 think 标签
Moonshot / Kimi	oai_config_kimi	sk-xxx	https://api.moonshot.cn/v1	kimi-k2-turbo-preview	温度强制 1.0
DeepSeek V4	native_oai_config_deepseek-xxx	sk-deepseek-xxx	https://api.deepseek.com	deepseek-v4-pro	[::warning:] 不带 /v1
阶跃星辰	oai_config_stepfun-xxx	sk-stepfun-xxx	https://api.stepfun.com/v1	step-2-16k	OAI 兼容
豆包 / 火山引擎	oai_config_volcengine-xxx	sk-volcengine-xxx	https://ark.cn-beijing.volces.com/api/v3	doubao-seed-1-8	OAI 兼容
硅基流动	oai_config_siliconflow-xxx	sk-siliconflow-xxx	https://api.siliconflow.cn/v1	deepseek-ai/DeepSeek-V3	新用户 16 元免费额度

反代 / 透传类渠道（需要 fake_cc_system_prompt = True）

渠道类型	推荐变量名	apibase	示例 model	备注
CC Switch（最常见）	native_claude_config0	https://host/claude/official	claude-opus-4-6	多数中文低价站走此协议
CRS 反代	native_claude_config_crs	https://host/api	claude-opus-4-6[1m]	CRS 官方协议
AnyRouter	native_claude_config_anyrouter	https://host/v1	claude-opus-4-6	与 CC switch 同协议族
Sider（订阅桥接）	sider_cookie	自动	gpt-5.4 / claude-opus-4-6	没有 API 时的兜底

本地模型

方案	推荐变量名	apibase	示例 model	备注
Ollama	native_oai_ollama	http://127.0.0.1:11434/v1	qwen2.5:14b	末尾 /v1 不能漏
llama.cpp	oai_config_llamacpp	http://127.0.0.1:8080/v1	default	建议走文本协议
vLLM	native_oai_vllm	http://127.0.0.1:8000/v1	你 load 的模型 id	需支持 function calling
LM Studio	oai_config_lmstudio	http://localhost:1234/v1	LM Studio 模型 id	GUI 本地部署最省心

[gear:] 关键可调字段速查（点击展开）

字段	默认	作用	何时要改
name	取 model	显示名 & mixin 引用名	有 mixin 时建议显式填
apikey	—	鉴权 Key	必填
apibase	—	API 端点地址	必填
model	—	模型名，后缀 [1m] 触发 1m 上下文	必填
fake_cc_system_prompt	False	伪装 Claude Code CLI 指纹	CC switch / CRS 必须 True
api_mode	chat_completion	可选 responses	[warning:] GPT-5.4 只能走 responses ，必须显式设置
thinking_type	adaptive	adaptive / enabled / disabled	关思考用 disabled
thinking_budget_tokens		仅 enabled 生效	low≈4096 / high≈32768
reasoning_effort	—	none ~ xhigh	o 系列 / Responses API 支持
temperature	1	采样温度	Kimi 强制 1.0
max_tokens	8192	单次回复最大 token	长思考可提到 32768
context_win	24000	历史裁剪阈值	1m 上下文设 800000
max_retries	1	自动退避重试次数	不稳定渠道改 3
connect_timeout	5	连接超时秒	海外端点调大
read_timeout	30	流式读取超时秒	开思考须 180+
stream	True	是否走 SSE 流式	CDN 截断时改 False
proxy	—	单 session 代理	海外端点加代理

[NEW_button:] DeepSeek V4 接入配置

DeepSeek 于 2026 年 4 月 24 日发布了 V4 系列模型，相比之前的 V3 / R1 有重大升级：

特性	说明
模型	deepseek-v4-flash（免费/低价）和 deepseek-v4-pro（旗舰）
上下文	1M tokens
思考模式	默认开启，支持 reasoning_effort: high（默认）/ max
Tool Calling	[check_mark_button:] 已完善，Agent 场景可用
兼容性	OpenAI SDK 格式，base_url 为 https://api.deepseek.com（注意： 不再带 /v1 后缀 ）

[warning:] **旧配置迁移提醒**：如果你之前用的是 deepseek-chat / deepseek-reasoner，官方已宣布这两个模型名将逐步弃用。它们目前等价于 deepseek-v4-flash 的非思考 / 思考模式。建议尽早迁移到新模型名。

```

native_oai_config_deepseek = {
    'name': 'deepseek-v4',
    'apikey': 'sk-< 你的 DeepSeek API Key>',
    'apibase': 'https://api.deepseek.com',          # [:warning:] 不带 /v1
    'model': 'deepseek-v4-pro',
    'thinking_type': 'enabled',                     # 开启思考链（默认就是开）
    'reasoning_effort': 'high',                     # high 或 max
    'read_timeout': 180,                            # 思考模式耗时长，务必调大
    'stream': True,                                 # 推荐开启，实时显示回复
}

```

最简配置（推荐 V4-Pro）

[:light_bulb:] 如何获取 API Key：前往 [DeepSeek 开放平台](#) 注册账号，在「API Keys」页面创建密钥即可。新用户通常有免费额度。

[:glowing_star:] Native Claude 接口配置（推荐） DeepSeek V4 支持 Anthropic 协议端点，GA 的 native_claude 接口对思考链和 Tool Calling 的处理更成熟，**推荐优先使用此方式**：

```

native_claude_config_deepseek = {
    'name': 'deepseek-v4-native',
    'apikey': 'sk-< 你的 DeepSeek API Key>',
    'apibase': 'https://api.deepseek.com/anthropic', # Anthropic 兼容端点
    'model': 'deepseek-v4-pro',
    'thinking_type': 'enabled',
    'reasoning_effort': 'high',
    'read_timeout': 180,
    'stream': True,
}

```

[:light_bulb:] 两种接口（OpenAI 格式 / Anthropic 格式）用同一个 sk- 开头的 API Key，不需要另外申请。

[:counterclockwise_arrows_button:] Mixin 配置模式：允许 GA 在模型断开后自动切换模型
一次配好主 + 备 + 兜底，任何一个 429/5xx/超时都自动切下一个：

```

mixin_config = {
    'llm_nos': ['claude-main', 'claude-backup', 'gpt-backup'], # 按优先级
    'max_retries': 10,    # 整个 rotation 总重试上限
    'base_delay': 0.5,    # 秒，指数退避起始
    'spring_back': 300,   # 秒，切到备用后多久尝试回到主
}

```

约束：llm_nos 中的名字必须精确匹配到其他 config 的 name 字段；所有被引用的 session **必须同属** Native 系列（NativeClaude + NativeOAI 可混）或**全不属** Native 系列。

1.5 启动 GenericAgent

首次启动

在终端中执行：

```
# 1. cd 到项目目录
cd " 你的 GenericAgent 路径"
```

```
# 2. 启动
python launch.pyw
```

如果是 windows 系统，可以双击 launch.pyw 启动。

看到浏览器弹出 Streamlit 聊天界面（或 pywebview 窗口），就说明启动成功了。如果用命令行模式 python agentmain.py，终端出现 >>> 提示符即为正常。

让 GA 自动安装剩余依赖

启动后，在对话框输入一句话，GA 会自己读代码、找出需要的包、全部装好：

请查看你的代码，安装所有用得上的 python 依赖

[:hammer_and_wrench:] 推荐：提升使用体验的两个任务

建立 Git 连接（方便以后更新代码）：

请帮我建立 git 连接，方便以后更新代码

GA 会自动配好。如果你电脑上没有 Git，它也会帮你下载 portable 版。

创建桌面快捷方式（以后双击图标就能启动）：

请帮我在桌面创建一个 launch.pyw 的快捷方式

使用 Hub 总控制台（可选）

hub.pyw 是 GA 的总控制台——一键启动/停止所有后台服务，并实时查看日志。

启动方式：在终端执行 python3 hub.pyw，或直接双击 hub.pyw 文件。勾选想启动的服务即可，不用记命令行参数。

[:clipboard:] Hub 可管理的服务列表（点击展开）

#	服务名	角色	启动命令
1	reflect/autonomous.py	自主行动反射器：30 分钟无输入自动触发	python agentmain.py --reflect reflect/autonomous.py
2	reflect/scheduler.py	定时任务调度器 + L4 会话归档	python agentmain.py --reflect reflect/scheduler.py
3	frontends/dingtalkapp.py	钉钉机器人	python frontends/dingtalkapp.py
4	frontends/fsapp.py	飞书 / Lark 机器人	python frontends/fsapp.py
5	frontends/qqapp.py	QQ 开放平台机器人	python frontends/qqapp.py
6	frontends/qtapp.py	PySide6 桌面悬浮球	python frontends/qtapp.py
7	frontends/stapp.py	默认 Streamlit Web UI	python -m streamlit run frontends/stapp.py
8	frontends/stapp2.py	Anthropic 风格 Streamlit UI	python -m streamlit run frontends/stapp2.py
9	frontends/tgapp.py	Telegram 机器人	python frontends/tgapp.py
10	frontends/wechatapp.py	个人微信（首次扫码登录）	python frontends/wechatapp.py
11	frontends/wecomapp.py	企业微信机器人	python frontends/wecomapp.py

1.6 常见问题排查

Q1: GA 调用工具时输出一大堆乱码 / 不会调用工具了？

原因：你的 mykey.py 中使用了非 Native 前缀的变量名（如 oai_config、claude_config），走的是旧版本文本协议，部分模型在该协议下工具调用不稳定。

解决方法：把变量名前缀改成 native_ 开头：

改之前	改之后
claude_config	native_claude_config
oai_config	native_oai_config

改完重启 GA 即可。Native 协议使用模型原生的 tool use 格式，工具调用更稳定。

Q2: Windows 安装后闪退 / 重启后无法启动？

常见原因（按发生概率）：

1. **Python 版本不对**：装了 3.14，pywebview/streamlit/PySide6 还没跟进
2. **Python 没加 PATH**：安装时“Add Python to PATH”没勾
3. **pip 依赖装了一半**：中途断网或关窗，导致包半装
4. **mykey.py 配置写错**：变量名冲突让整个模块 import 失败

排查步骤：

1. 看 Python 版本（建议 3.11 或 3.12）

```
python --version
```

2. 手动起 agentmain 看报错

```
cd <你的 GenericAgent 目录>
```

```
python agentmain.py
```

3. 看 launch.pyw 报什么（改后缀为 .py 才能看 stderr）

```
python launch.py
```

修复方法：

1. 卸载 3.14，装 [Python 3.12](#)，安装时勾 Add Python to PATH
2. mykey.py 里只保留一个启用的 **config**，其他全部用 # 注释
3. 让 GA 自己补依赖（见 1.5 节“让 GA 自动安装剩余依赖”）

Q3: pip 装完后不知道在哪运行 / 不知道 cd 到哪？

pip install 装的是 Python 第三方包（依赖），**不是** GenericAgent 本体。GenericAgent 是一个 GitHub 仓库，必须手动下载 + 解压。

正确步骤：

1. 打开 <https://github.com/lsdefine/GenericAgent>
2. 绿色 Code 按钮 → Download ZIP
3. 解压到你喜欢的位置（例如 D:\GenericAgent）
4. 终端里 cd D:\GenericAgent
5. python agentmain.py

永久解决：让 GA 给你建桌面快捷方式（见 1.5 节“折叠块”可选：让 GA 帮你做的事），之后双击图标即可启动。

Q4: DinTalClaw 懒人包和命令行安装有什么区别？

项目	DinTalClaw 懒人包	命令行安装
便利性	解压即用，内置 Python + 依赖	需装 Python、pip install
体积	约 500MB	约 50MB 源码 + 按需依赖
版本	锁定某个 commit，可能落后	跟进最新 git
自我升级	不能直接 git pull	可以 git pull 或让 GA 自己更新
修改源码	需手动找到解包后的 python 环境	改完即生效

建议：想快速体验选懒人包；想长期用选命令行 + git clone。

Q5: 能否支持 Win7?

不能。 Python 3.9+ 已官方放弃 Win7，pywebview 依赖的 WebView2 也装不上。

临时方案：Win7 上用命令行模式 + Python 3.8，只跑 agentmain.py，不启 launch.pyw。但很多前端也跑不起来，**强烈建议升级到 Win10/11。**

Q6: 怎样判断是否正常启动了？界面和别人的不一样

现象	判定
命令行出现 >>> 提示符，敲字能触发 [LLM Running...]	[check_mark_button:] CLI 正常
浏览器 / pywebview 窗口出现 Streamlit 聊天界面	[check_mark_button:] GUI 正常
终端一直没输出	[cross_mark:] 卡在 import 或 mykey 配置
界面有但模型选择列表为空	[warning:] mykey.py 没有可用 LLM 配置

“界面不一样”通常是两种 Streamlit 前端：stapp.py（默认界面）和 stapp2.py（Anthropic 风格浅色主题），二者均正常，选你喜欢的即可。

Q7: 如何更新到最新版本？能让 GA 自己更新吗？

能！两种方式：

方式 A • 手动：

```
cd <GenericAgent 目录>
git pull
```

若是 Download ZIP 安装的，没有 .git 目录，就到 GitHub 重新下 ZIP 覆盖。

方式 B • 让 GA 自己更新（推荐）：

请帮我建立 git 连接，方便以后更新代码

之后任何时候输入：

请 git 更新你的代码，然后看看 commit 有什么新功能

GA 会自己 git pull + 解读 commit log + 汇报给你。

Q8: API mode 需要改成 responses 吗？

默认 chat_completions 即可。只有两种情况改 responses：

1. 你用的是 OpenAI 官方 /v1/responses 端点且想传 reasoning.effort 结构化参数
2. 上游渠道只开了 Responses API 没开 chat/completions

```
native_oai_config_responses = {
    ...
    'api_mode': 'responses',
    'reasoning_effort': 'high',
}
```

Q9: API 繁忙 / 高峰期失败一次就停，能配重试吗？

能，两个层面：

- **单 session**: 'max_retries': 3 (默认 1)，加大即可
- **多 session fallback**: 用 mixin_config 的 max_retries (整个 rotation 总重试)

```
native_claude_config = {
    ...
    'max_retries': 5,          # 429/5xx/超时自动退避重试
    'connect_timeout': 10,    # 连接超时 5 → 10 秒
    'read_timeout': 180,      # 流式读取 30 → 180 秒 (思考模式必须加大)
}
```

Q10: 接 API key 时需要配上下文长度吗？

通常不需要。默认 context_win=24000 (NativeClaude 默认 28000)，这只是历史裁剪阈值，不是模型硬上限。

什么时候要改：

- 用 1M 上下文 Claude 时: 'model': 'claude-opus-4-6[1m]' + 'context_win': 800000
- MiniMax 204K: 'context_win': 100000
- 本地小模型 8K: 'context_win': 6000

[open_file_folder:] 相关文件速查

内容	路径
API 密钥配置模板	mykey_template.py
API 密钥配置 (你自己创建)	mykey.py
主启动脚本	launch.pyw
服务总控台	hub.pyw
CLI 入口	agentmain.py
默认 Web UI	frontends/stapp.py
Anthropic 风格 Web UI	frontends/stapp2.py

[memo:] 小结

- **环境三步走**: 装 Python 3.11/3.12 → 下载项目 → pip install streamlit pywebview
 - **配置只需一个文件**: 复制 mykey_template.py 为 mykey.py，填好 apikey 和 apibase 即可
 - **启动后让 GA 自己装剩余依赖**，不用手动折腾
-

下一章：第 2 章 浏览器能力解锁

第 2 章 浏览器能力解锁

学完本章，GA 将能帮你操作一切在浏览器上的任务——像一个真正的”上网助手”。

[[:bullseye:]] 学习目标

1. 为浏览器安装 `tmwd_cdp_bridge` 插件，让 GA 获得网页操控能力
 2. 验证插件安装成功，完成一次浏览器自动化操作
 3. 知道 GA 在浏览器中能做什么、不能做什么（登录态、验证码等）
-

2.1 安装 `tmwd_cdp_bridge` 插件

GA 默认只能读写文件和执行代码。要让它操作浏览器，我们需要安装 **`tmwd_cdp_bridge`**——一个基于 CDP（Chrome DevTools Protocol，Chrome 官方开放的浏览器调试通道）的浏览器扩展。

前置条件

- 已安装 **Chrome / Edge / 夸克** 中的任意一款浏览器
- 已下载 GA 项目源码，`GenericAgent\assets\tmwd_cdp_bridge` 目录完整存在
- **已至少启动过一次 GA**（首次启动会生成插件所需的配置文件，否则安装会报错）

[[:warning:]] **请勿移动或删除 `tmwd_cdp_bridge` 目录**。插件采用”从文件夹加载”的方式安装，目录丢失或路径变更会导致插件失效。建议将 GA 项目放在稳定路径下（例如 `D:\GenericAgent`）。

三款浏览器的安装步骤大同小异，选择你正在使用的浏览器，按对应步骤操作即可。

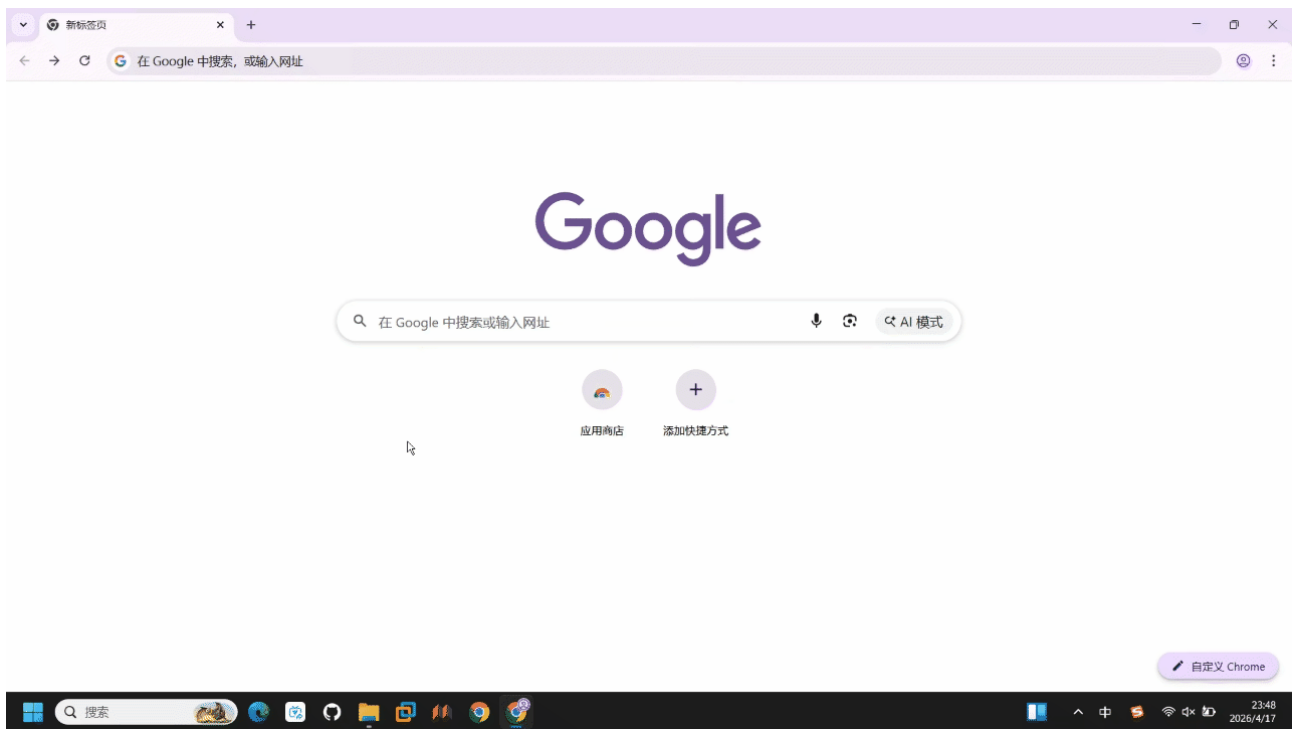
2.1.1 Chrome 安装步骤

第 1 步：打开扩展管理页并启用开发者模式

在 Chrome 地址栏输入以下地址并回车：

`chrome://extensions`

页面 **右上角** 找到”开发者模式”开关，将其打开。



第 2 步：加载插件

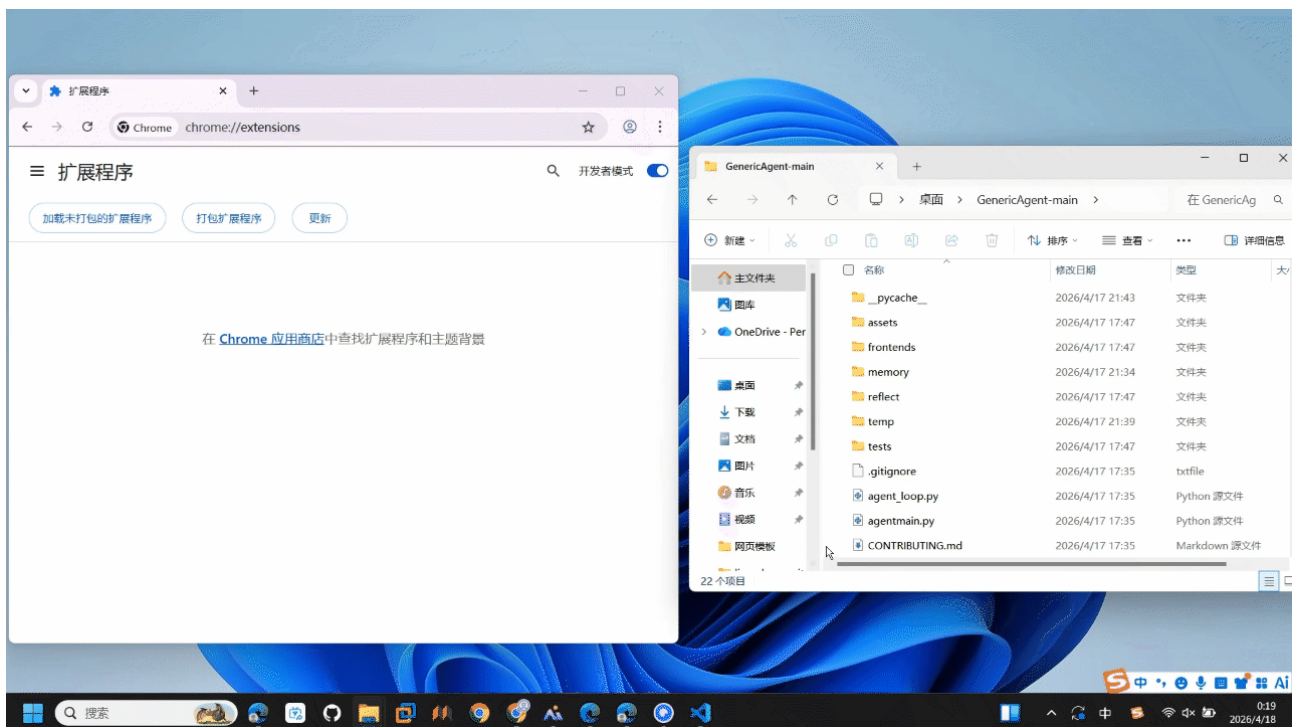
打开资源管理器，定位到以下目录：

GenericAgent\assets\tmwd_cdp_bridge

将 **tmwd_cdp_bridge** 整个文件夹 直接拖入 Chrome 扩展管理页的空白处并释放。扩展列表中会出现 **TMWD CDP Bridge** 卡片，表示安装成功。

[!light_bulb:] 拖拽无效？

也可以点击页面上的“加载已解压的扩展程序”按钮，在弹出的对话框中选择 **tmwd_cdp_bridge** 文件夹。两种方式等价。



2.1.2 Edge 安装步骤

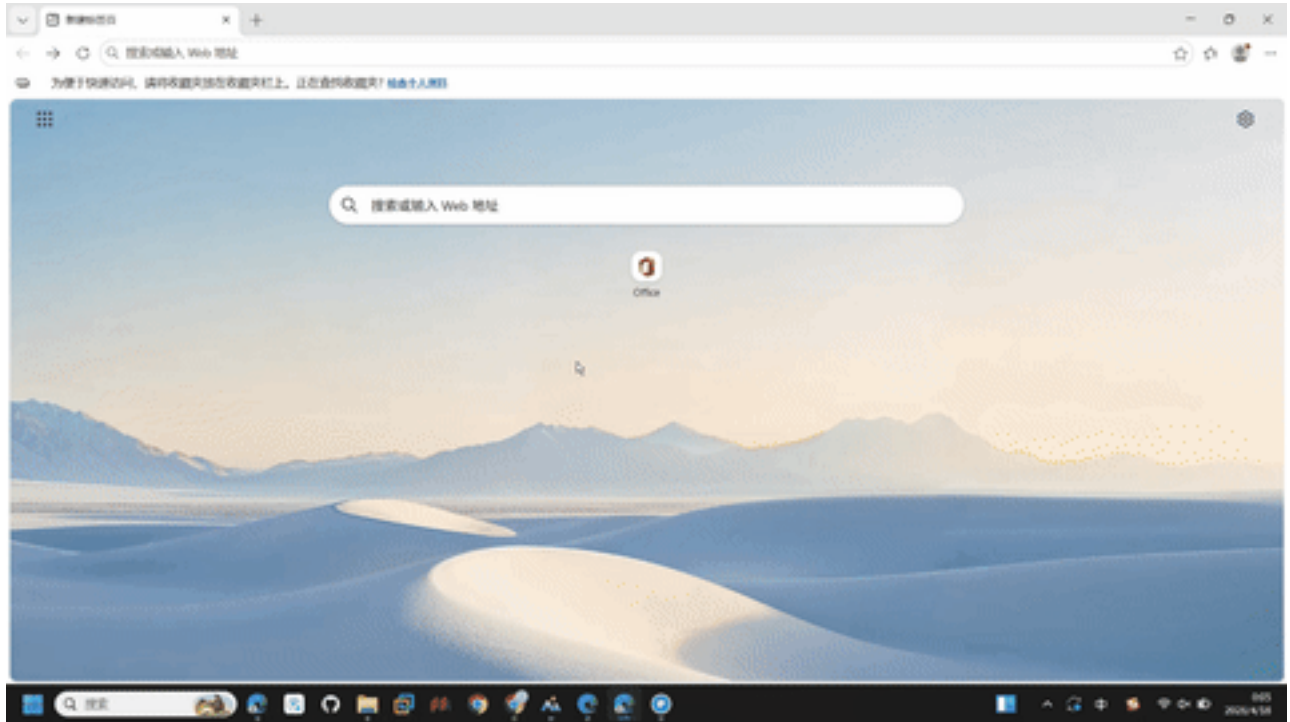
Edge 与 Chrome 底层是同一套引擎，步骤基本一致。

第 1 步：打开扩展管理页并启用开发人员模式

在 Edge 地址栏输入：

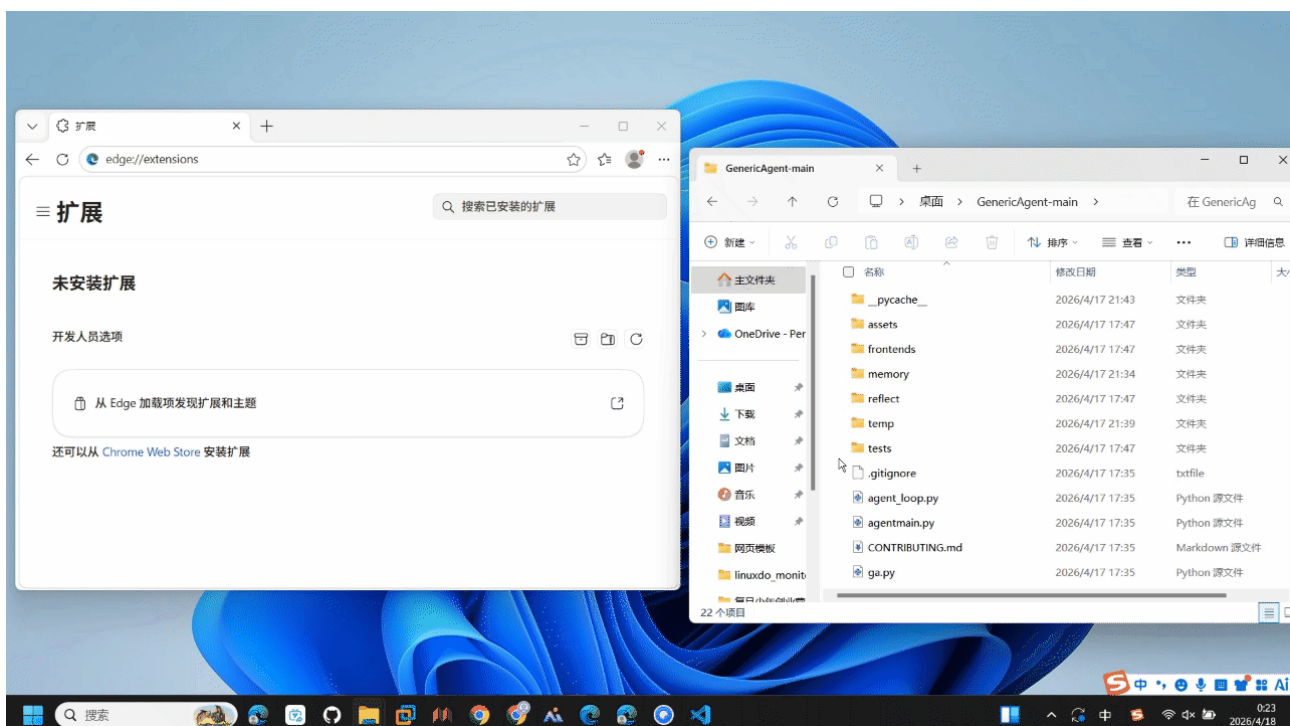
edge://extensions

扩展管理页 左侧导航栏底部 找到 “开发人员模式” 开关，将其打开。



第 2 步：加载插件

将 GenericAgent\assets\tmwd_cdp_bridge 整个文件夹拖入 Edge 扩展管理页窗口并释放，Edge 会自动完成安装。



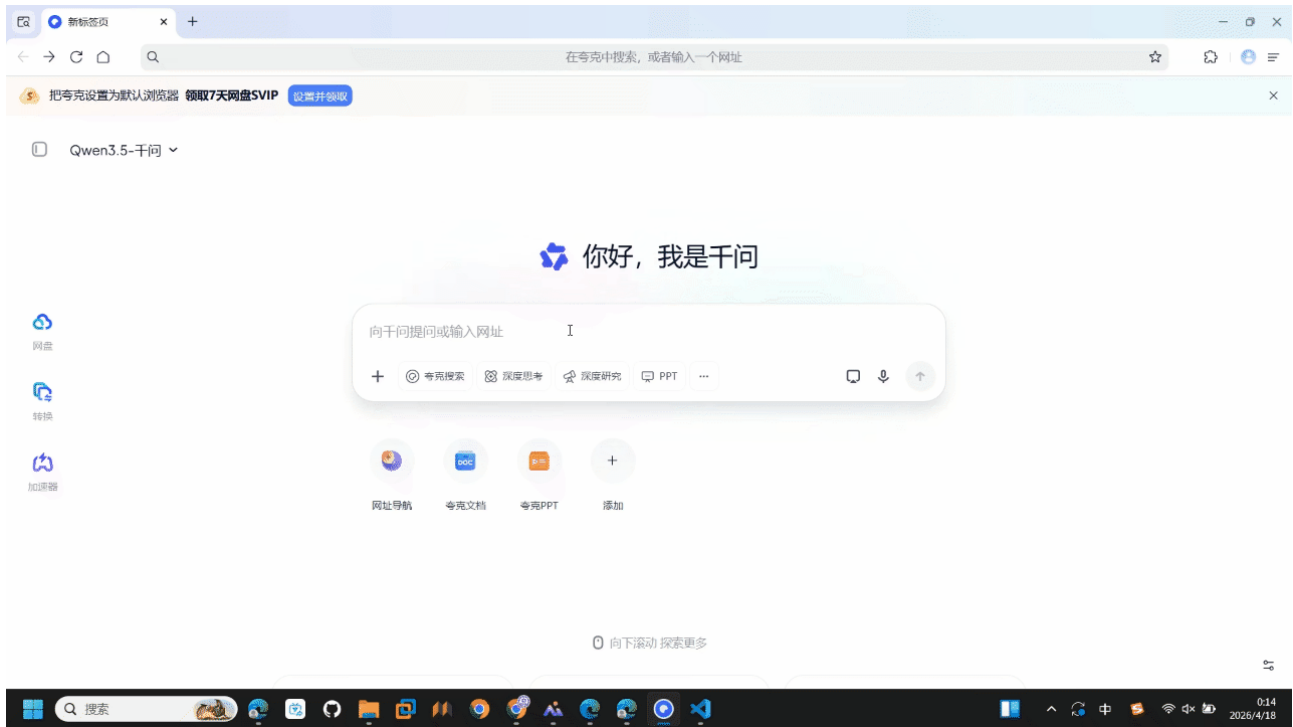
2.1.3 夸克浏览器安装步骤

第 1 步：打开扩展管理页并启用开发者模式

在夸克地址栏输入：

quark://extensions

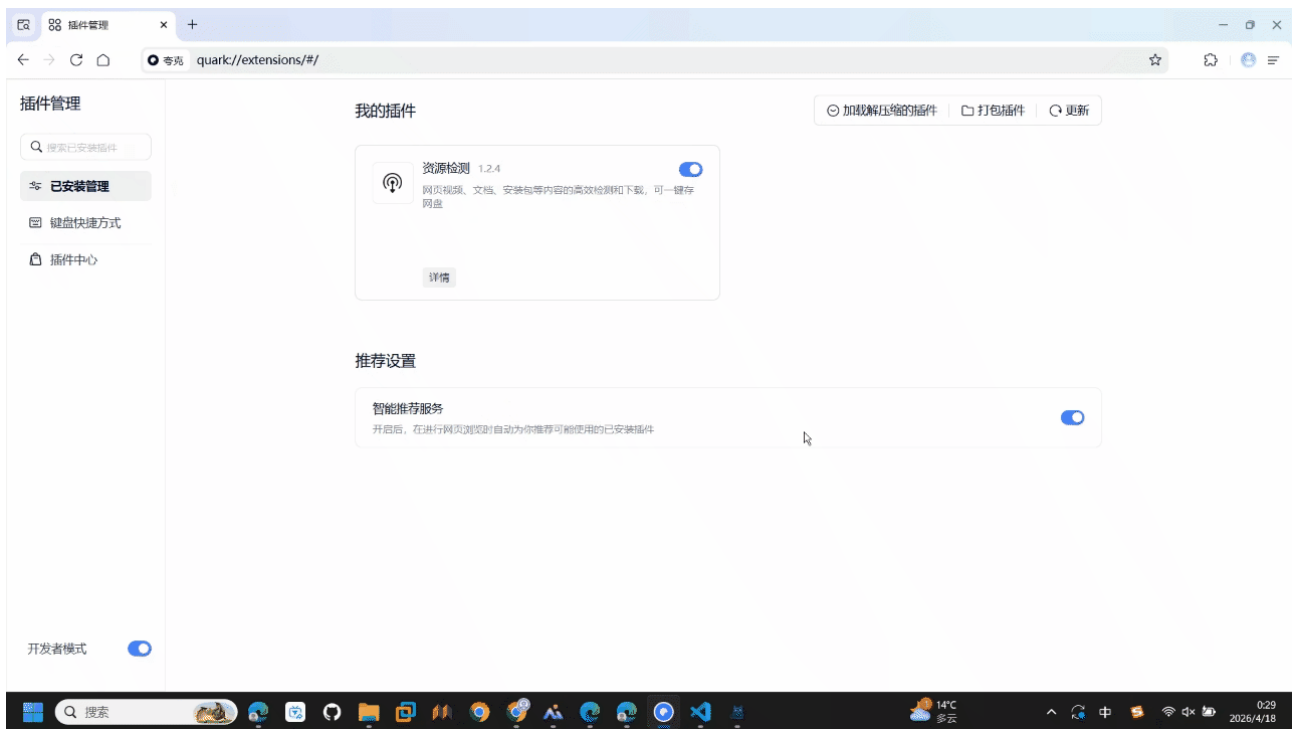
页面 **左侧导航栏底部** 找到 “**开发者模式**” 开关，将其打开。



第 2 步：加载插件

点击 “**加载解压缩的插件**”，在弹出的对话框中选中以下目录：

GenericAgent\assets\tmwd_cdp_bridge



2.2 验证安装

无论使用哪款浏览器，安装完成后的验证方式完全相同。回到 GA 对话界面，输入：
打开百度，搜索"今天天气"
如果浏览器自动打开百度并完成搜索，说明插件已正常工作 [:party_popper:]

2.3 实战示例

插件就绪后，我们可以用自然语言让 GA 完成各种浏览器操作。以下是几个可直接运行的场景：

示例 1：天气查询

告知我北京明日的天气情况

GA 会自动打开天气网站，读取页面内容，把明日的温度与天气状况整理给你。



上图是 GA 正在操控浏览器抓取天气信息的过程。

打开中国天气网，告知我北京明日的天气情况

- 切到中国天气网北京页查明日天气
- 已到北京天气页；提取明日详情
- 用JS精确抓取北京明日天气数据
- 选择器因#7d失效；改用getElementById抓取

LLM Running (Turn 5) ...

<thinking> 已从中国天气网北京城区7天页精确提取到明日天气数据：晴，26°C/15°C，南风，小于3级。现在直接简洁告知用户即可。 </thinking>
<summary>已读到北京明日天气：晴26/15°C</summary>
北京明日天气：晴，最高 26°C，最低 15°C，南风，小于3级。

GA 读取完成后，会直接在对话中返回整理好的天气结果。

示例 2：视频网站浏览

打开哔哩哔哩，看看有哪些热门视频

GA 会打开 B 站热门榜，读取并整理当前热门视频信息。

打开哔哩哔哩，看看有哪些热门视频



未探测浏览器；先查标签并打开B站

已查到7标签；转到B站热门页并读取

已进B站热门页；提取可见热门视频列表

LLM Running (Turn 4) ...

<thinking>已成功打开B站热门页并读取到当前可见的热门视频列表。现在直接向用户简要汇报，不需要进一步操作，除非用户想让我点开某个视频或查看更多。</thinking>

<summary>已读B站热门页；汇总当前可见热视频</summary>

我打开了哔哩哔哩热门页，当前可见的热门视频有这些：

- 盘点嘎子直播带货名场面！经典战役卖酸奶居然还不是最离谱的？！哈哈哈哈哈
- 在 5:20 睡觉 13:14 准时起！！
- 和李现，攀登人生第一座雪山
- 终于来了！大疆Pocket 4上手
- 人生乱套我睡觉

GA 汇总了热门视频的标题、播放量等关键信息，方便你快速了解当前热门内容。

示例 3：购物网站商品推荐

[light_bulb] 使用前需先在浏览器中手动登录淘宝。

我想给好朋友送一本书作为生日礼物，请你打开淘宝看看并给我一些建议吧



GA 会打开淘宝搜索相关商品，并整理推荐结果：

我在淘宝上看到的几类合适选项

1. 想送得高级一点：精装/刷边/礼盒版文学

这类最适合生日礼物，拿得出手。

我看到的比较像礼物的有：

- 《悉达多》精装定制版，约 45.9元
- 《傲慢与偏见》蓝金刷边定制书，约 52.7元
- 《小王子》80周年纪念版 / 珍藏版，约 59~110元
- 《活着》纪念版/印签版，约 16元起，但更建议挑纪念版而不是最普通版本

适合送谁：

- 喜欢拍照、收藏、仪式感强的人
- 平时爱看文学、喜欢书架好看的人

推荐指数：

- 女生朋友：《傲慢与偏见》刷边版 / 《小王子》纪念版

2.4 常见问题

Q1: GA 是否自带浏览器？必须额外安装吗？

GA **不自带**浏览器。它通过 CDP (Chrome DevTools Protocol) 扩展接管你本机已有的 Chrome / Edge / 夸克，沿用你的登录态与浏览器环境。这与 Selenium、Playwright 等需要另外启动一个独立浏览器的自动化方案有本质差异。

Q2: 篡改猴插件要不要装？

当前版本无需安装篡改猴，直接使用 2.1 所述 `tmwd_cdp_bridge` 扩展即可。篡改猴方案仅在 CDP 插件无法使用的特殊环境下作为备选。

Q3: 安装插件时报错怎么办？



在安装浏览器插件之前，必须先运行一遍 GA（见前置条件），否则会缺失配置文件。请回到第 1 章完成首次启动后再安装插件。

Q4: GA 能否操作需要登录的网站？

可以，这也是该方案的核心优势之一。插件直接使用你本机的浏览器，凡是你已登录的站点，GA 均可直接访问，无需提供账号密码或令牌。

Q5: GA 能否自动登录网站？

不能，且不需要。GA 使用的正是你自己的浏览器，因此需要你本人预先完成一次登录（扫码、输密码、二次验证等），此后浏览器会自行保留会话。

实际使用建议：

- **首次使用前：**先打开浏览器，手动登录好你希望 GA 操作的站点（淘宝、Gmail、飞书等）
- **登录过期后：**GA 访问时会看到登录页，此时暂停让你手动重新登录，完成后再继续
- GA **不会也不应**为你输入账号密码——这既涉及账号安全，也无法处理扫码、短信验证码等场景

Q6: 能否让 GA 绕过滑块验证码？

不建议尝试。滑块、图形验证码背后是专门的反爬风控系统（分析鼠标轨迹、设备指纹等），专门用来识别自动化操作；GA 的 CDP 插件定位是“替已登录用户做正常操作”，并非破解工具。

[wrench:] 旧方案：篡改猴（Tampermonkey）安装指南

在 CDP 插件推出之前，GA 采用的是 **篡改猴 + 用户脚本** 方案。目前该方案已不再推荐，主要原因：

- 安装流程较长（安装篡改猴 → 开启“允许用户脚本” → 调整 CSP 设置 → 安装脚本）
- 受浏览器 isTrusted 机制限制，部分按钮点击、文件上传等操作无法完成
- 官方已明确将在后续版本中逐步移除该方案

以下情况仍可能需要旧方案：

- GA 版本较旧，assets/ 目录下仅有 `ljq_web_driver.user.js` 而无 `tmwd_cdp_bridge/`——建议升级至最新版本
- 特殊环境下 CDP 插件不可用（例如企业设备禁用了 Chrome 的 debugger 权限）

回退步骤：向 GA 发送以下指令：

执行 `web setup sop`，解锁 web 工具（按篡改猴方案）

GA 将读取 `memory/web_setup_sop.md` 并引导完成篡改猴的安装与配置。

[information:] 同名指令执行 `web setup sop`，解锁 web 工具在当前版本下默认采用 CDP 插件方案。如需走旧方案，请在指令中附加“按篡改猴方案”。

[open_file_folder:] 相关文件速查

内容	路径
CDP 浏览器桥接插件目录（勿删勿移）	<code>assets/tmwd_cdp_bridge/</code>
旧版篡改猴用户脚本（仅回退时使用）	<code>assets/ljq_web_driver.user.js</code>
浏览器配置 SOP（GA 内部参考）	<code>memory/web_setup_sop.md</code>

[memo:] 小结

- **一个插件解锁浏览器能力：**将 `tmwd_cdp_bridge` 文件夹拖入浏览器扩展页即可完成安装
- **GA 使用你自己的浏览器：**已登录的站点无需重复登录，GA 直接复用你的会话
- **验证方式：**让 GA 执行一次搜索，浏览器自动响应即表示成功

上一章：第 1 章 安装与环境配置 | 下一章：第 3 章 基础使用

第 3 章 基础使用：第一个小时

学完本章，你将熟悉 GA 的界面、解锁视觉与文件搜索等扩展能力，并掌握多轮对话技巧。

[:bullseye:] 学习目标

1. 熟悉 GA 的交互界面，能自如地与 GA 对话
2. 解锁 OCR、视觉、飞书 CLI、Everything CLI 四大扩展能力
3. 掌握多轮对话与上下文管理的最佳实践

[:red_circle:] 注意事项（请务必阅读）：

1. **不要轻易让 GA 修改自己的源代码。**本章的配置过程中，GA 可能会申请修改自身代码。如果确实需要修改，请先让 GA 备份原文件，再进行改动。
2. **任务完成后，务必提醒 GA 更新记忆。**特别是 L1 层索引——如果 L1 没有更新，新窗口中 GA 将不知道自己已经拥有这些能力。

3.1 界面介绍

双击 launch.pyw 启动 GA 后，我们会看到一个简洁的对话界面：

界面主要由以下几个区域组成：

❶ 模型选择区

左上角下拉菜单，用于切换当前使用的大语言模型（LLM）。如果我们在 mykey.py 中配置了多个模型，可以在这里自由切换。

❷ 轮次计数器

显示当前任务已经进行了多少轮对话。每一轮代表 GA 的一次“思考 → 工具调用 → 获得结果”的循环。

[:light_bulb:] 关于轮次的更多细节

每一轮都可以展开查看详细过程。如果 GA 在一轮中调用了多个工具，我们可以展开查看每个工具的输入和输出。

❸ 对话显示区

GA 的回复和工具调用结果都会显示在这里。多轮任务会自动折叠成可展开的摘要，避免历史信息过长。

❹ 输入框

在这里输入指令或问题，用自然语言描述即可。支持多行输入（Shift+Enter 换行），按 Enter 发送。

❺ 发送按钮

点击输入框右侧的上箭头按钮，或直接按 Enter 键发送消息。

[:warning:] **小提示：**如果 GA 正在执行任务，会看到光标闪烁（█），表示正在生成回复。如果离开超过 30 分钟且开启了“自主行动”，GA 会自动执行预设的自动化任务。

3.2 视觉能力

GA 由大语言模型驱动，本身不具备视觉能力。但通过配置 OCR 和视觉扩展，我们可以让 GA “看见”屏幕和图片。

3.2.1 OCR 能力

OCR（光学字符识别）能让 GA 阅读图片上的文字。配置非常简单，只需一句话：

用 rapidocr 配置你的 ocr 能力并存入记忆

配置完成后，GA 就能阅读任何图片上的文字了：

帮我看看 F:\data\to_be_ocrd.png 这张图片写了什么

更实用的是，配置了 OCR 之后，GA 可以了解我们的屏幕内容：

总结我当前屏幕上的内容

3.2.2 视觉能力 (Vision)

OCR 只能识别文字。如果需要 GA 理解界面布局、识别图标和窗口，我们还需要解锁完整的视觉能力。

[warning:] **前置条件：**视觉能力依赖 API 的多模态 (image) 接口。不同 API 提供商对多模态的支持情况不同，需要先确认你当前使用的 API 是否具备这一能力。向 GA 发送：

读取 mykey 中当前使用的 API 配置，写代码探测该 API 是否支持 image（多模态）接口

GA 会根据 mykey.py 中的 API 配置，自动编写探测代码并返回结果。

如果当前 API 支持多模态，直接跳到下一步。

如果当前 API 不支持多模态，也不用担心——GA 内置的 memory/vision_sop.md 中记录了如何通过 [ModelScope](#) 免费使用视觉大模型（如 Qwen3-VL）。

[warning:] **前置条件：**先在浏览器中登录 [ModelScope](#)，注册账号并获取访问令牌（右上角头像 → 访问控制 → 访问令牌，复制 ms- 开头的 token）。

准备好令牌后，向 GA 发送：

读取 memory/vision_sop.md 中 ModelScope 免费后端的集成流程，按照步骤帮我配置

GA 会引导你完成以下操作：1. 在 ModelScope 注册并获取免费的访问令牌（ms- 开头）2. 将令牌写入 mykey.py 3. 自动集成 ModelScope 的免费视觉 API 作为后端

[light_bulb:] ModelScope 提供免费的 API Inference 服务，无需本地显卡，开箱即用。

确认 API 支持多模态后（无论是原有 API 还是刚配置的 ModelScope），向 GA 发送：

根据 memory/vision_sop.py，利用刚才探测成功的 API 重写视觉调用代码，保证能实现相同的功能。重写完

GA 会依次完成以下步骤：

1. 读取现有的 vision_sop.py，用你的 API 重写视觉调用代码
2. 自动测试重写后的代码，确保功能正常
3. 测试通过后，同步更新 vision_sop.md 文档
4. 将视觉能力写入 L1 索引，确保后续会话都能记住这项能力

[warning:] GA 在此过程中可能会请求修改自身代码的授权。由于我们确实希望它获得视觉能力，回复授权即可。GA 给出的授权选项是它自己生成的，并非固定程序，我们可以复制选项发送，也可以用自然语言表达意图。

配置完成后，测试一下：

请看看我的屏幕上有哪些应用、窗口和重要组件

现在 GA 已经拥有完整的视觉能力，可以理解屏幕上的窗口和内容了。

3.3 飞书 CLI

通过飞书 CLI (lark-cli)，GA 不仅能收发飞书消息，还能直接操作日历、云文档、多维表格、邮箱、任务、审批等 22 个飞书功能模块——相当于给 GA 装了一个飞书全能遥控器。

[light_bulb:] 飞书 CLI 和 第 5 章 介绍的飞书 Bot 是两件事：Bot 让你在飞书聊天窗口给 GA 发消息；CLI 让 GA 主动操作你的飞书数据（日历、文档、邮件等）。两者可以同时使用。

[light_bulb:] 飞书 CLI 能做什么

- 日历操作（查看日程、创建会议、查询忙闲）
- 文档操作（创建/读取/更新云文档、搜索文档）
- 多维表格（创建表格、增删改查记录、数据分析）
- 电子表格（读写 Excel、导出数据）
- 任务管理（创建任务、分配成员、设置提醒）
- 邮箱操作（发送邮件、搜索邮件、管理草稿）
- 会议纪要（查询会议记录、获取 AI 总结）
- 审批流程（查询审批、同意/拒绝审批）
- OKR 管理、知识库管理、考勤打卡查询等

安装飞书 CLI

向 GA 发送：

安装并配置飞书 CLI (lark-cli)，使其可以通过 code_run 调用

GA 会自动完成安装：

安装完成后，还需要配置 App 凭证和用户授权，GA 会一步步引导你完成。

配置 App 凭证与用户授权（点击展开完整流程）

配置 App 凭证

GA 会要求提供飞书应用的 App ID 和 App Secret（获取方式见 第 5 章 飞书接入指引）。

如果你已经在 mykey.py 中配置过飞书凭证，可以直接告诉 GA：直接读取我在 mykey.py 中配置的 App ID 和 App Secret。

否则按 GA 提示，选择安全输入方式：

提供 App ID + 手动输入 App Secret

向 GA 发送你的 App ID（请替换为你的实际值）：

app_id: 'cli_XXXXXX' 品牌:feishu

GA 会打开一个命令行窗口，将你的 App Secret 粘贴进去并回车：

看到 OK 后，告诉 GA：已输入 Secret 并回车

授权用户登录

App 凭证只能操作应用级功能。要让 GA 访问你的个人数据（日历、邮件、任务等），还需要完成用户授权：

弹出浏览器授权页面，我来给你授权

浏览器会弹出飞书授权页面，确认授权即可：

授权完成后回到 GA，回复：已授权

效果验证

配置完成，试一下：

帮我看看明天的飞书日程

日程查看、文档整理、任务创建等操作都可以直接用自然语言告诉 GA，无需手动输入 CLI 命令。

3.4 Everything CLI（Windows 文件搜索）

Everything CLI 是 Windows 平台的极速文件搜索工具，能在毫秒级时间内搜索整个硬盘。配置后，GA 可以快速定位任何文件，无需遍历目录。

为什么需要它？

当我们让 GA 读取一个文件时，如果不提供完整路径，GA 可能无法定位。Everything CLI 为 GA 提供毫秒级全盘搜索能力，彻底解决这个问题。

[light_bulb:] 没有 Everything 时的表现

安装配置

向 GA 发送：

安装并配置 Everything 命令行工具进 PATH

[warning:] 安装过程中，系统可能弹出管理员权限请求窗口（有时藏在任务栏后台），请注意查看并授权。

[light_bulb:] 安装过程截图

安装完成后，GA 会确认配置成功：

[light_bulb:] 安装后的建议

- 如果电脑管家提示 Everything 想要开机自启，请允许（不会明显拖慢启动）
- 建议重启电脑以确保 Everything 服务生效
- 可以让 GA 把这个习惯写入记忆：「写入 L1，任何大规模文件查询直接使用 es」

效果验证

再次让 GA 查找文件，这次指定使用 es 搜索：

请读取 to_be_ocred.png 里面的文字，使用 es 全盘搜索这个文件

GA 仅用 1 轮就定位到了文件路径，效率显著提升。

什么是“最佳实践”

走到这里，你已经为 GA 解锁了视觉、飞书 CLI、Everything CLI 三项扩展能力。但这只是 GA 能力的冰山一角——我们不可能替你穷举所有场景，因为一千个人眼里有一千个哈姆雷特，每个人的任务、偏好和工作流都不一样。

所谓最佳实践，不是一套固定的操作手册，而是一个使用习惯：

1. **把需求告诉 GA**——用你自己的话描述，不需要措辞精确
2. **执行过程中敢于打断**——发现方向不对就喊停，给出你的思路 and 判断

3. 成功后让 GA 沉淀记忆——好的经验只需教一次，GA 会永久记住

保持这个习惯，你的 GA 就会越用越懂你，越用越强。所以，现在就打开电脑试起来——你的实践，就是最佳实践。

3.5 多轮对话与上下文

GA 不是一次性工具，而是可以连续对话的助手。它会记住之前说过的话，理解上下文。

3.5.1 / 命令（快捷指令）

在对话框中，我们可以直接输入以 / 开头的快捷指令来控制 GA 的行为，无需用自然语言描述。以下是目前支持的全部 / 命令：

命令	作用	说明
/help	显示帮助	列出所有可用的 / 命令
/status	查看状态	显示 GA 当前是否正在运行，以及正在使用哪个 LLM
/stop	停止当前任务	立即中断正在执行的任务，GA 会停止所有工具调用
/new	清空当前上下文	开始一个全新的对话，清除所有历史记录
/restore	恢复上次对话	直接从日志文件恢复最近一次的对话历史（聊天平台使用）
/resume	恢复历史对话（可选择）	GA 浏览最近几次对话的结尾摘要，让你选择恢复哪一次（GUI 桌面窗口使用）
/llm [n]	查看或切换模型	不带参数：列出所有已配置的模型；带编号：切换到指定模型
/continue	列出可恢复的历史会话	不带参数：显示最近的历史会话列表；/continue N：恢复第 N 个会话

[light_bulb:] 各平台对 / 命令的支持情况

/restore 和 /resume 都能恢复历史对话，但入口不同：

- /restore 在聊天平台前端（飞书、钉钉、QQ、Telegram、企业微信）中实现，直接从日志文件加载上次的对话记录
- /resume 在 GUI 桌面窗口中实现，GA 会列出最近几次对话的摘要让你选择，然后以选中的对话为基础继续聊天

前端	支持的命令
GUI 桌面窗口	/continue、/resume、/llm（部分命令有对应按钮，如停止按钮 = /stop）
飞书、钉钉、QQ、Telegram、企业微信	/help、/status、/stop、/new、/restore、/continue、/llm
微信	/stop、/llm

使用示例：

1. 切换模型：如果当前模型响应太慢，可以快速切换

/llm ← 列出所有可用模型
/llm 2 ← 切换到 2 号模型

2. 恢复历史对话：重启 GA 后想继续之前的任务

/continue ← 列出最近的历史会话（所有前端通用）
/continue 1 ← 恢复第 1 个会话
/resume ← （GUI 桌面窗口）列出最近几次对话摘要，选择后恢复
/restore ← （聊天平台）直接恢复最近一次的对话历史

3. 紧急停止：GA 执行了不符合预期的操作

/stop ← 立即中断，GA 停止当前所有操作

[warning:] **注意：**/restore 恢复的是最近一次的对话历史(保存在 temp/model_responses/ 下的日志文件中)。恢复后 GA 只是获得了之前的上下文，我们需要输入新的问题来继续工作。

3.5.2 什么时候需要开新会话？

情况	说明
切换完全不同的任务	从数据分析切换到浏览器操作 → 建议 /new 或重启
对话太长导致“忘事”	上下文窗口满了，GA 开始遗忘早期内容 → /new 开新会话
想要重新开始	之前尝试失败想换思路 → /new 清空后重来

[light_bulb:] **开新会话方法：**输入 /new 即可清空上下文，无需重启 GA。当然，关闭窗口重新启动也可以。

3.5.3 上下文管理技巧

1. **明确指代：**说“帮我处理刚才那个 Excel 文件”比“帮我处理一下”好得多
2. **分步确认：**复杂任务分成多步，每步确认后再继续
3. **利用记忆：**常用偏好可以让 GA 记住——“以后处理销售数据时，都用这个格式”，GA 会写入记忆系统，下次自动应用
4. **善用 /restore：**意外关闭 GA 后，用 /restore 恢复上下文，无缝继续

3.6 常见问题

Q1: GA 说“文件不存在”，但文件明明在？

可能原因：GA 错误识别了目录，或当前工作目录与我们的认知不一致。

解决方法：

1. 使用绝对路径（包含盘符的完整路径）
2. 右键文件 → 「复制为路径」→ 粘贴到输入框
3. 先告诉 GA 文件位置，再让它操作

Q2: 代码运行结果不对？

解决方法：直接告诉 GA 「结果不对，我预期的是 XXX」。GA 会自己 debug 并修复。

示例：

- [girl:]：帮我统计每个月的销售额
- [robot:]：[返回结果]
- [girl:]：不对，应该是按自然月统计，不是按 30 天
- [robot:]：[重新计算，返回正确结果]

Q3: 对话太长，GA 开始“忘事”？

解决方法：

1. 让 GA 把关键信息写入工作记忆：「请将 xxx 写入 working_checkpoint」
2. 让 GA 把重要偏好写入长期记忆：「记住这个设置，以后都用」
3. 开新会话后让 GA 读取历史：「我们上次聊了什么？」

Q4: GA 安装依赖失败?

解决方法:

1. 检查网络连接, 确保可以访问外网
2. 如果有代理, 确保代理设置正确
3. 尝试手动安装: `pip install 包名`
4. 告诉 GA: 「继续安装剩余的依赖」

Q5: 浏览器控制不生效?

解决方法:

1. 重新执行: 「执行 web setup sop 解锁 web 工具」
2. 检查浏览器扩展管理页面, 确认插件已启用
3. 确保浏览器是最新版本

Q6: GA 响应很慢?

解决方法:

1. 切换到更快的模型 (如果配置了多个)
2. 把复杂任务拆分成多个简单任务
3. 检查网络连接质量

Q7: 如何让 GA 记住我的偏好?

明确告诉 GA 「记住这个设置」或「以后都这样做」, GA 会把偏好写入记忆系统, 下次自动应用。

示例:

- `[:girl:]`: 以后分析数据时, 都用柱状图, 不要用折线图
- `[:robot:]`: 好的, 我会记住这个偏好

`[:open_file_folder:]` 相关文件速查

内容	路径
GA 启动入口	<code>launch.pyw</code>
API 密钥配置	<code>mykey.py</code>
GA 记忆存储目录	<code>memory/</code>

`[:memo:]` 小结

- **界面简洁**: GA 是自然语言交互助手, 输入框打字即可使用
- **扩展能力**: OCR → 视觉 → 飞书 CLI → Everything CLI, 每个都是一句话配置, 永久提升 GA 表现
- **/命令**: `/stop /new /restore /resume /llm` 等快捷指令, 高效控制 GA
- **对话技巧**: 明确指代、分步确认、善用记忆, 让 GA 越用越懂你

上一章: 第 2 章 浏览器能力解锁 | 下一章: 第 4 章 记忆与 Skill 系统

第 4 章 记忆与 Skill 系统：让 GA 越用越强

学完本章，你将掌握 GA 的记忆机制和 Skill 体系——教会 GA 一次，它就能永久记住并复用你的经验。

[:bullseye:] 学习目标

- 1. 理解 GA 四层记忆架构（L1-L4）的作用和管理方式
- 2. 学会让 GA 将成功经验沉淀为可复用的 Skill
- 3. 掌握 Skill Search 和 Skill 迁移，快速扩充 GA 的能力库

4.1 Skill 与记忆概览

GA 把 Skill 作为一种记忆来管理。它们通常以 SOP、Python 脚本或子目录的形式保存在 `./memory/` 下，属于 L3 层。

GA 的四层记忆系统如下：

层级	存储内容	路径
L1 索引层	极简索引（<30 行），每轮启动注入，让 GA 知道自己有哪些知识	<code>./memory/global_mem_insight.txt</code>
L2 事实层	环境事实，如路径、系统配置、用户偏好	<code>./memory/global_mem.txt</code>
L3 技能层	具体 Skill：SOP / 脚本 / 目录	<code>./memory/*.md</code> 、 <code>./memory/*.py</code>
L4 归档层	压缩归档后的历史会话	<code>./memory/L4_raw_sessions/</code>

名称	修改日期	类型	大小	
autonomous_operation_sop	2026/4/18 13:51	文件夹		L3记忆
L4_raw_sessions	2026/4/18 13:51	文件夹		L4记忆
skill_search	2026/4/18 13:51	文件夹		
adb_ui.py	2026/4/18 13:51	Python 源文件	4 KB	L3记忆
autonomous_operation_sop.md	2026/4/18 13:51	Markdown 源文件	3 KB	
github_contribution_sop.md	2026/4/18 13:51	Markdown 源文件	5 KB	
global_mem.txt	2026/4/18 13:58	文本文档	1 KB	L2记忆
global_mem_insight.txt	2026/4/18 13:58	文本文档	2 KB	L1记忆
keychain.py	2026/4/18 13:51	Python 源文件	2 KB	
ljqCtrl.py	2026/4/18 13:51	Python 源文件	6 KB	
ljqCtrl_sop.md	2026/4/18 13:51	Markdown 源文件	4 KB	
memory_cleanup_sop.md	2026/4/18 13:51	Markdown 源文件	3 KB	
memory_management_sop.md	2026/4/18 13:51	Markdown 源文件	6 KB	
ocr_utils.py	2026/4/18 13:51	Python 源文件	4 KB	
plan_sop.md	2026/4/18 13:51	Markdown 源文件	13 KB	
procmem_scanner.py	2026/4/18 13:51	Python 源文件	5 KB	
procmem_scanner_sop.md	2026/4/18 13:51	Markdown 源文件	5 KB	L3记忆
scheduled_task_sop.md	2026/4/18 13:51	Markdown 源文件	2 KB	
subagent.md	2026/4/18 13:51	Markdown 源文件	4 KB	
tmwebdriver_sop.md	2026/4/18 13:51	Markdown 源文件	9 KB	
ui_detect.py	2026/4/18 13:51	Python 源文件	5 KB	
verify_sop.md	2026/4/18 13:51	Markdown 源文件	3 KB	
vision_sop.md	2026/4/18 13:51	Markdown 源文件	3 KB	
web_setup_sop.md	2026/4/18 13:51	Markdown 源文件	5 KB	

总结：Skill 保存在 L3（`./memory/`），并通过 L1（`./memory/global_mem_insight.txt`）帮助 GA 在每次启动时想起它们。

[warning:] **注意：**1. 第一次成功启动后，L1 文件会自动创建 2. 每次 git pull 更新代码后，如果 ./memory/ 下有新文件且你已有 L1，需要让 GA 同步：

看 ./memory/ 下的文件，依据 memory_management_sop.md 更新 L1 索引

4.2 创建新 Skill

GA 完成任务后，我们可以让它把刚才的经验总结成 Skill。这样只需教会 GA 一遍，它后续就能参照 Skill 流畅高效地完成整套流程。用一段时间以后，你的 GA 就会拥有专属于你的一整套 Skill。

操作流程

1. **直接告诉 GA 你的任务**——如果 GA 找不到现成经验，会先逐步试错
2. **发现问题随时打断**——点击页面右上角的 Stop 按钮，指出错误并补充正确方向
3. **任务成功后，让 GA 总结为 Skill：**

依据 memory_management_sop.md，提取成功经验总结为 skill 写入 L3，并更新 L1 索引

[warning:] **注意事项：**- GA 有时会忘记更新 L1，导致新窗口不知道这个 Skill 存在。记得提醒它 - 如果单个窗口对话太长（上下文约 30K），GA 会遗忘早期内容。可以让它读日志恢复记忆：

在 ./model_responses/ 下找到你的日志文件，依据 memory_management_sop.md，提取经验总结为 skill 写

- **不是所有经验都值得沉淀。**L1 中存放低价值索引会干扰 GA 查找。值得沉淀的任务特征：需要多轮试错、以后会重复做、能总结成稳定步骤
-

4.3 Skill Search：搜索外部 Skill

GA 内置了 **Skill Search**——“寻找 Skill 的 Skill”，适合在本地没有现成经验时找外部参考。

使用方式

1. **直接向 GA 描述需求：**

帮我找一个用来……的 skill

2. **GA 会自动执行搜索流程：**

- 搜索：利用内置 Skill Search API，在 10 万 + skills 中自动搜索
- 选择：优先选评分高、环境匹配、风险低的 Skill
- 验证：在你的环境里小步试运行

3. **任务成功后，总结为自己的 Skill：**

依据 memory_management_sop.md，提取经验改写 skill 写入 L3，并更新 L1 索引

[warning:] Skill Search 的语义匹配对中文支持较差，GA 会自动用英文搜索。手动搜索时务必使用英文关键词。

[wrench:] 自定义 Skill Hub

GA 默认连接官方 Hub，也可以指定其他开源 Skill 网站或私有 Hub。直接用提示词告诉 GA 即可：

把 skill search 的搜索源改为 http://your-hub.com:port

4.4 迁移已有 Skill

如果你在其他平台（Claude Code、OpenClaw 等）已经积累了 Skill，可以迁移到 GA 中使用。

[package:] 从 Claude Code / OpenClaw 迁移

先确认 Skill 的真实来源和实际生效目录，再把其中可复用的机制迁移到 GA 的 L3 层。

Skill 目录位置： - 个人 Skill（所有项目生效）：~/.claude/skills/<skill-name>/ - 项目 Skill（仅单个项目生效）：项目根目录下 .claude/skills/<skill-name>/ - 插件市场 Skill：~/.claude/plugins/marketplaces/<marketplace-name>/skills/<skill-name>/

手动迁移步骤：

1. 提取 Skill 的有效内容。除 SKILL.md 外，也需要关注目录下的其他文件：

```
<skill-name>/
├── SKILL.md           # 核心文件（必需）
├── agents/            # subagent 定义
├── scripts/           # 可执行脚本
├── references/        # 参考文档
├── assets/            # 模板、数据文件
└── examples/         # 使用示例
```

2. 删除敏感信息和旧系统专属内容（包装语、旧路径、特定格式）

3. 整理成 SOP / 脚本 / 目录，放进 ./memory/

4. 小范围试跑，确认新 SOP 在当前环境下可执行

5. 补充 L1 索引（手动编辑或让 GA 更新）：

看 ../memory/ 下的新增文件，依据 memory_management_sop.md 更新 L1 索引

让 GA 帮你迁移：

读取 [skill 路径] 中的内容，提取可复用的机制，删除专属依赖，依据 memory_management_sop.md，改写

[warning:] 迁移野生 Skill（社区来源）

1. 先检查安全性：

帮我检查 [skill 路径] 这个 skill 是否有安全隐患

2. 小任务验证：

用这个 skill 完成 [任务描述] / 帮我想一个测试案例验证这个 skill

3. 成功后正式存入 L3：

依据 memory_management_sop.md，提取经验改写 skill 写入 L3，并更新 L1 索引

4.5 记忆系统详解

核心公理

- **无行动不记忆：**只有工具调用成功后的结果才能写入 L1/L2/L3
- **已验证数据不可删改：**重构时只能迁移不能丢失
- **禁止存储易变状态**（时间戳、PID 等）

L1 索引层压缩原则

- **存在性编码**：只需让 GA 意识到知识存在，不存具体知识
- 用 ROI 公式评估每行索引的价值，压缩低价值内容

[light_bulb:] 记忆的详细写入和清理规则见 ./memory/memory_management_sop.md 和 ./memory/memory_cleanup_sop.md。

L4 自动压缩归档

L4 用于保存较早的历史会话，避免原始日志不断堆积。

[open_file_folder:] L4 启用方法与触发条件

启用方式（需以 reflect/scheduler 模式启动）：

```
# 用 agentmain.py 启动
python agentmain.py --reflect reflect/scheduler.py

# 用 launch.pyw 启动
python launch.pyw --sched
```

[warning:] 直接双击 launch.pyw 默认**不会**启用 L4 自动压缩。

触发条件（需同时满足）：

1. 使用 reflect/scheduler 模式启动
2. Agent 进程在本次运行中已持续超过约 12 小时
3. 会话（日志文件）2 小时内未更新且大于 4.5KB

产出目录：

```
./memory/L4_raw_sessions/
├── all_histories.txt      ← 所有会话历史的工作记忆汇总（追加写入）
├── 2026-04.zip           ← 月度压缩包
├── 2026-03.zip
└── ...
```

[light_bulb:] 也可以跳过 scheduler, 直接让 GA 用 L4_raw_sessions/compress_session.py 手动压缩日志。

检索历史对话：

到 L4 下找对话记录，[描述对话（对话时间是 XXX / 内容是 XXX）]

4.6 内置 Skill 一览

GA 仓库中内置了以下 Skill，启动后可以直接使用：

Skill	描述
web_setup_sop	首次使用时配置浏览器自动化环境
skill_search	从 105K+ 技能卡中搜索技能
autonomous_operation_sop	指导 Agent 闲置时自主行动
scheduled_task_sop	设置定时任务
plan_sop	针对复杂任务（3 步以上 / 多文件 / 条件分支）的规划模式
subagent	启动并行子 Agent，上下文隔离，可与主 Agent 互相通信
ljqCtrl	Windows 系统键鼠物理控制（macOS 用户可适配）

Skill	描述
adb_ui.py	安卓手机界面控制（点击、滑动、截图）
ocr_utils	图片文字提取，支持多语言 OCR
github_contribution_sop	开源项目贡献工作流：fork → 修改 → 测试 → PR
keychain	管理本地加密密钥

GA 大多数情况下会自动选择合适的 Skill。想主动使用时直接描述需求即可，例如：

开启 subagent 做任务 A，你同时做另一个任务 B

4.7 常见问题

Q1: memory 文件里都是 SOP，和 Skill 的关系是什么？

在 GA 中，Skill 就是以 SOP（或脚本）形式存在的。SOP 是 Skill 的载体——一份 SOP 文档就是一个 Skill。GA 不像其他平台有独立的” Skill 格式”，它直接把经验写成可执行的 SOP 步骤或 Python 脚本，存放在 ./memory/ 下（L3 层）。所以你看到的 SOP 文件，本身就是 GA 的 Skill。

Q2: GA 有没有像 OpenClaw 那样的 skills？

有。GA 采用分层记忆架构，其中 L3 层即为可执行 Skill（SOP 或脚本形式）。GA 有内置 Skill（见 4.6），外部 Skill 可通过 Skill Search 检索（见 4.3），或从 Claude Code / OpenClaw 迁移（见 4.4）。

Q3: Skill 怎么安装？能让 GA 自己装 Skill 吗？

不需要手动”安装”。直接描述需求即可，GA 会自动通过 Skill Search 寻找。详见 4.3。

Q4: Claude Code 的 Skill 能直接迁移到 GA 吗？

不能直接复制使用。GA 会读取外部 Skill 的内容，在你的环境中实际运行验证，成功后再将经验沉淀为自己的 SOP/脚本存入 L3。也就是说，外部 Skill 经过「读取 → 实践 → 沉淀」后才会变成 GA 自己的 Skill。详见 4.4。

Q5: 社区 Skill 迁移后不管用？

社区 Skill 不是为你的环境定制的，先让 GA 改造、试跑后再总结为你自己的 Skill，存入记忆。详见 4.4。

Q6: GA 做完任务但下次还是重新摸索？

任务完成后需提示 GA 将经验总结为 Skill，并按照 memory_management_sop.md 写入 L3 和 L1。详见 4.2。

依据 memory_management_sop.md，提取成功经验总结为 skill 写入 L3，并更新 L1 索引

Q7: Skill Search 搜不到想要的？

任何 Skill 库都不可能覆盖所有场景。如果官方 Hub 搜不到，可以尝试换英文关键词重试；也可以将 GA 的 Skill Search 指向其他开源 Skill 网站或自建的私有 Skill Hub（见 4.3 自定义 Skill Hub），GA 会到你指定的 Skill 库中检索。实在没有现成 Skill，也可以让 GA 从零探索，成功后再总结成你自己的 Skill。

Q8: 窗口关闭后怎么恢复历史对话？

当前会话间不直接共享记忆，但可以让 GA 读历史日志恢复：

去 model_responses 和 L4 下找对话记录，[描述对话（时间是 XXX / 内容是 XXX）]，读取记录恢复上下文

若日志尚未归档至 L4，可在 `./temp/model_responses/` 下按时间戳手动查找。

Q9: 怎么开多个会话？GUI 只有一个对话框

直接新开实例即可——再打开一个终端运行启动命令，或再次双击 `launch.pyw`。

Q10: 对话太长能压缩上下文吗？

可以让 GA 用 `L4_raw_sessions/compress_session.py` 压缩历史日志。目前没有在线压缩功能，上下文到 30K 后会被直接截断。建议：

- 长任务使用 `plan_sop` 分阶段执行，缓解失忆
- 关键结论及时沉淀到 L2/L3，避免依赖长上下文

Q11: GA 的记忆（L1-L4）必须手动添加吗？

几乎不需要。GA 会自动更新或在简单提示下完成：

- **L4**：用 `reflect/scheduler` 模式自动归档，或让 GA 主动压缩
- **L3**：多种方式详见 4.2、4.3、4.4，可提醒 GA 根据 `memory_management_sop` 自动总结
- **L2/L1**：GA 会依据提示词和 SOP 自动管理

Q12: `global_mem.txt` 是空文件，正常吗？

正常。`global_mem.txt` 是 L2 层（环境事实）存储文件，首次使用时为空。GA 在探索过程中发现环境事实后会自动写入。也可以主动提醒：

请查阅 `memory_management_sop.md`，将已发现的环境事实写入 L2 层

Q13: GA 如何在不同系统中保持成长进度 / 迁移？

GA 日常运行和自进化几乎只涉及 `memory` 和 `temp` 两个目录，迁移这两个目录即可。

[`:open_file_folder:`] 相关文件速查

内容	路径
L1 索引层，每轮启动注入	<code>./memory/global_mem_insight.txt</code>
L2 事实层，环境配置和用户偏好	<code>./memory/global_mem.txt</code>
L3 技能层，Skill 存放位置	<code>./memory/*.md / ./memory/*.py</code>
L4 归档层，历史会话压缩包	<code>./memory/L4_raw_sessions/</code>
记忆管理规范（写入/清理规则）	<code>./memory/memory_management_sop.md</code>
Skill Search 工具目录	<code>./memory/skill_search/</code>
当前会话原始日志	<code>./temp/model_responses/</code>

[`:memo:`] 小结

- **四层记忆**：L1 索引 → L2 事实 → L3 技能 → L4 归档，层层递进、自动管理
- **Skill 沉淀**：任务成功后一句话提示 GA，即可将经验固化为可复用 Skill
- **Skill 获取**：内置 11 个开箱即用，Skill Search 提供 10 万 + 外部技能检索，还可从 Claude Code 等平台迁移

上一章：第 3 章 基础使用 | 下一章：第 5 章 聊天平台接入

第 5 章 聊天平台接入

学完本章，你可以把 GA 接入微信、飞书、钉钉等聊天平台，随时随地用手机给 Agent 下任务。

[:bullseye:] 学习目标

1. 理解为什么要把 GA 接入聊天平台，以及支持哪些平台
2. 完成至少一个平台的 Bot 接入，让 GA 能在聊天窗口中回复消息
3. 掌握多平台并行运行的方法

5.1 为什么要接入 Bot

GA 默认通过本地窗口交互，必须守在电脑前。把它接入聊天平台后，我们可以获得几个明显好处：

- **随时可用**：打开聊天框就能开始交互，不必一直守在终端前
- **手机可用**：在外面也能继续下发任务、查看状态
- **融入工作流**：很多信息本来就在聊天中产生，Bot 接入后可直接嵌入沟通场景
- **降低门槛**：非技术用户也能通过熟悉的聊天界面使用 Agent

5.2 支持的平台一览

平台	启动文件	核心依赖	配置项 (mykey.py)
微信个人号	frontends/wechatapp.py	requests, qrcode, pycryptodome	wxbot
飞书	frontends/fsapp.py	lark-oapi	fs_app_id, fs_app_secret
钉钉	frontends/dingtalkapp.py	dingtalk-stream	dingtalk_client_id, dingtalk_client_secret
QQ 机器人	frontends/qqapp.py	qq-botpy	qq_app_id, qq_app_secret
Telegram	frontends/tgapp.py	python-telegram-bot	tg_bot_token
企业微信	frontends/wecomapp.py	wecom-aibot-sdk	wecom_bot_id, wecom_secret

[:light_bulb:] 所有平台的接入流程遵循相同模式：**安装依赖 → 在对应平台创建应用 → 获取密钥填入 mykey.py → 启动对应的 app.py**。选择你最常用的平台，跟着下面的指引操作即可。

5.3 微信个人号接入

微信是最常见的使用场景，我们提供两种接入方式。

方法一：让 GA 自己搞定（推荐）

不想手动操作？直接在 GA 对话窗口中输入一句话：

查看frontends/wechatapp.py，帮我接入微信 bot，最后记得生成一个二维码让我登录

GA 会自动完成依赖安装和启动流程，你只需在最后扫码即可。

扫码后，终端显示连接成功，你就可以在微信中和 GA 对话了。

方法二：手动接入

第 1 步：安装依赖

```
pip install pycryptodome qrcode requests
```

第 2 步：启动微信前端

```
cd /path/to/GenericAgent/frontends # 替换为你本地的 frontends 目录  
python wechatapp.py
```

第 3 步：扫码登录

启动后会自动弹出二维码，用微信扫码即可连接。

Q4: 没有弹出二维码？

打开一个新的终端窗口，手动查看二维码图片：

```
open ~/.wxbot/wx_qr.png
```

如果显示二维码过期，重新运行 `python wechatapp.py` 即可。

第 4 步：确认连接成功

扫码后，终端显示连接成功，你就可以在微信中和 GA 对话了。

5.4 其他平台接入

以下平台的接入流程已折叠，点击展开你需要的平台即可。

[[:blue_book:](#)] 飞书接入指引

安装依赖

```
pip install lark-oapi python-socks
```

创建飞书应用

1. 打开 [飞书开放平台](#)，登录账号
2. 点击「创建企业自建应用」，填写应用信息：
 - 应用名称：我的 Agent 助手（可自定义）
 - 应用描述：个人 AI 助手
 - 应用图标：可选
3. 进入应用详情页，左侧菜单选择「添加应用能力」→ 找到「机器人」→ 点击「添加」
4. 左侧菜单「权限管理」→ 「开通权限」，搜索并开通以下权限：
 - `im:message` — 获取与发送单聊、群组消息
 - `im:message:send_as_bot` — 以应用身份发送消息
 - `contact:user.id:readonly` — 获取用户 ID
 - `im:message.p2p_msg:readonly`

获取密钥并配置

5. 左侧菜单「凭证与基础信息」，记录 App ID 和 App Secret



6. 在 mykey.py 中填入：

```
fs_app_id = 'cli_xxxx'           # 替换为你的 App ID
fs_app_secret = 'xxxxxxx'       # 替换为你的 App Secret
fs_allowed_users = [""]         # 空列表表示允许所有用户

387 # qq_allowed_users = ['your_user_openid'] # 留空或 ['*'] 表示允许所有 QQ 用户
388 fs_app_id = 'cli_xxxxxxxxxxxxxxxx'
389 fs_app_secret = 'xxxxxxxxxxxxxxxxxx'
390 fs_allowed_users = [''] # 留空或 ['*'] 表示允许所有飞书用户
391 # wecom_bot_id = 'your_bot_id'
```

启动服务

7. 启动飞书前端，看到「等待消息…」表示服务已就绪：

```
cd /path/to/GenericAgent/frontends
python fsapp.py
```

```
/Users/liaoying/miniconda3/envs/tlly/lib/python3.11/site-packages/lark_oapi/ws/pb/google/__init__.py:2: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  __import__('pkg_resources').declare_namespace(__name__)

=====
飞书 Agent 已启动（长连接模式）
App ID: cli_a932a86dc7b8dbb4
等待消息...
=====
[Lark] [2026-03-16 23:05:07,643] [INFO] connected to wss://msg-frontier.feishu.cn/ws/v2?fpid=493&aid=552564&device_id=7617869682841996253&access_key=64d02688e639677c2a6e3f55fd9f87a7&service_id=33554678&ticket=25f7fdc3-9e02-42c6-9142-4b29ffbe596d [conn_id=7617869682841996253]
```

配置事件订阅

8. 回到飞书开放平台的应用配置页面：

- 点击「回调配置」→ 订阅方式选择「**使用长连接接收事件**」
- 点击「事件配置」→ 同样选择「**长连接**」→ 点击「添加事件」→ 搜索添加 `im.message.receive_v1`

发布应用

9. 填写版本信息 → 保存 → 确认发布

开始对话

10. 下载 [飞书客户端](#)，登录后搜索你的 Agent 名称，即可开始对话

搜索框搜索你建立的个人助手名字

在对话框中输入问题，手机版飞书也能继续对话

[[:orange_book:](#)] 钉钉接入指引

安装依赖

```
pip install dingtalk-stream
```

创建钉钉应用

1. 登录 [钉钉开发者后台](#)（需企业管理员账号）
2. 进入 **应用开发** → **企业内部应用** → **钉钉应用** → **创建应用**，填写应用名称、描述、图标

获取密钥并配置

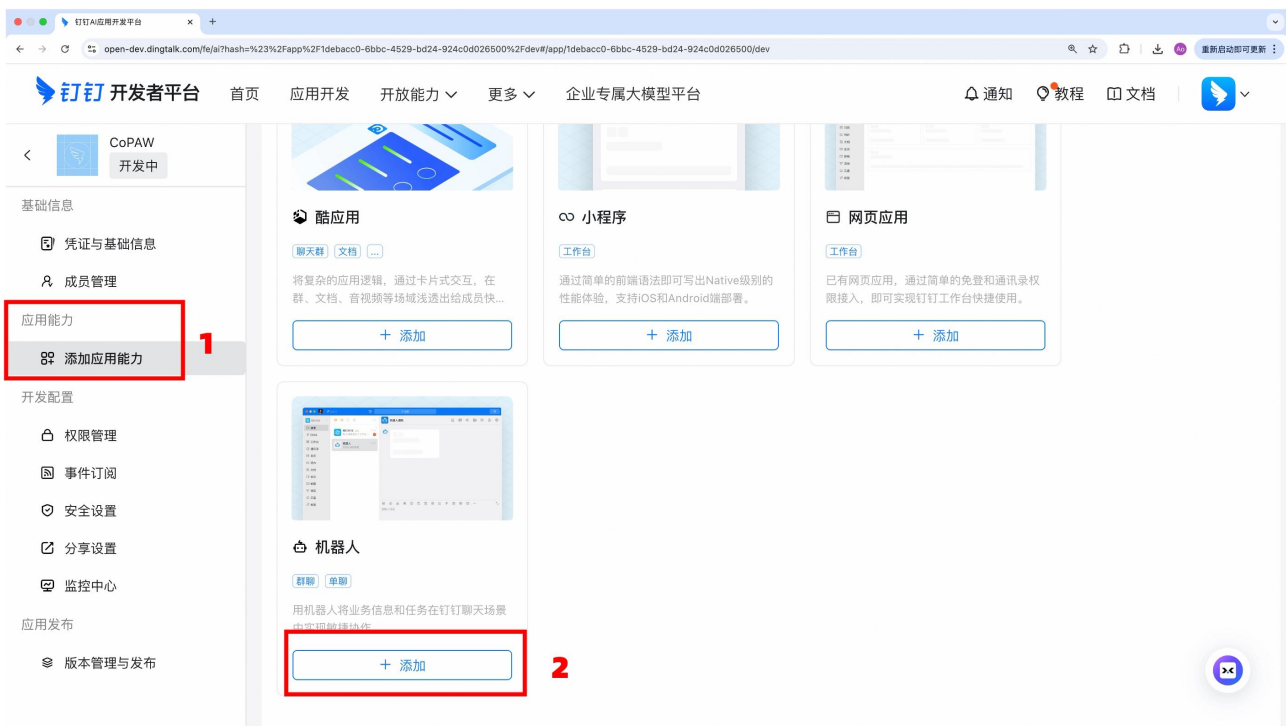
3. 左侧菜单「基础信息 → 凭证与基础信息」，找到：

- **Client ID** = AppKey（公开标识）
- **Client Secret** = AppSecret（私密密钥）

4. 在 `mykey.py` 中填入上述 ID 和 Secret

添加机器人能力

5. 应用详情 → **应用能力** → **添加应用能力** → **机器人**

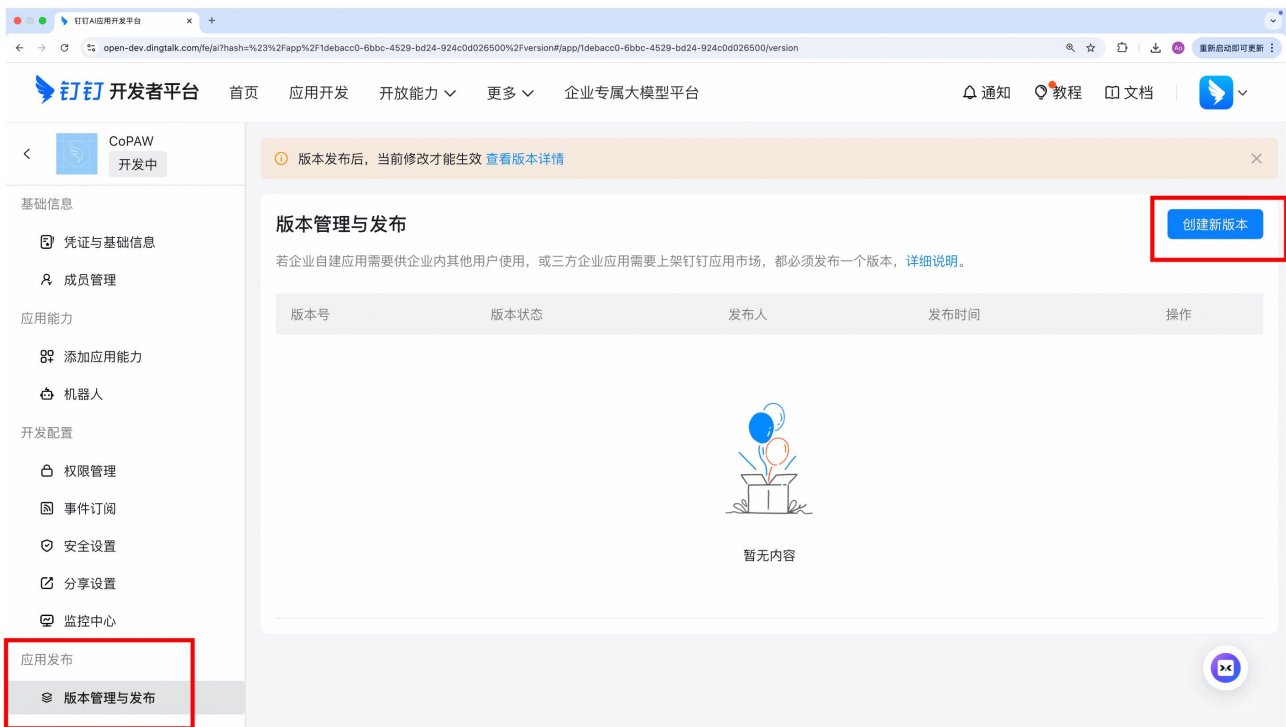


6. 配置机器人信息，底部消息接收模式选择 Stream 模式（流式接收）



发布并启动

7. 应用发布 → 版本管理与发布 → 创建新版本 → 发布，等待审批通过



8. 启动钉钉前端：

```
cd /path/to/GenericAgent/frontends
python dingtalkapp.py
```

9. 在钉钉客户端搜索 Agent 名称，即可开始对话

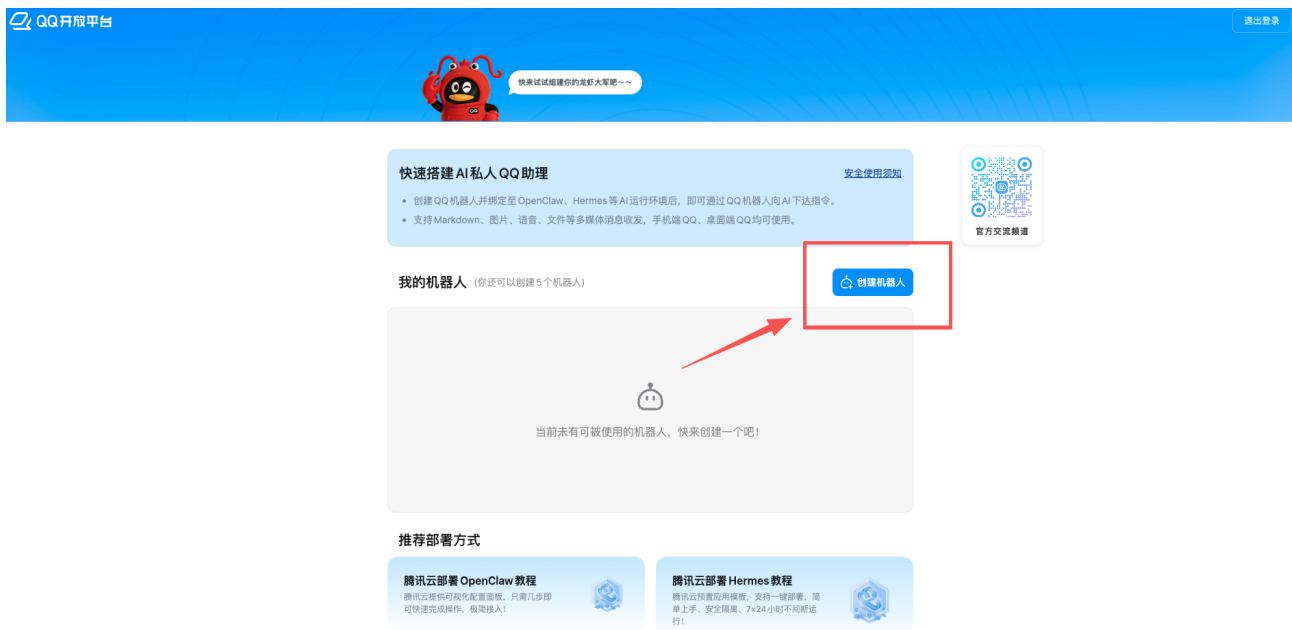
[:green_book:] QQ 机器人接入指引

安装依赖

```
pip install qq-botpy
```

创建 QQ 机器人

1. 打开 [QQ 开放平台](#)，扫码登录 → 选择「机器人」选项
2. 创建机器人



获取密钥并配置

3. 获取 **AppID** 和 **AppSecret** (见 step1), QQ 扫码进入机器人对话窗口 (见 step2)



4. 在 `mykey.py` 中填入上述 ID 和 Secret

```

384 # tg_allowed_users = [6806...]
385 qq_app_id = 'xxxx'
386 qq_app_secret = 'xxxx'
387 qq_allowed_users = [''] # 留空或 ['*'] 表示允许所有 QQ 用户
388 fs_app_id = 'cli xxxxxxxxxxxxxxxxx'

```

启动并对话

5. 启动 QQ 前端：

```

cd /path/to/GenericAgent/frontends
python qqapp.py

```

6. 开始对话

20:56

71%

< 机器人 在线 >



20:25

介绍 genericagent 的核心能力

思考中...

LLM Running (Turn 1) ...

GenericAgent 的核心能力可以概括为“能规划、能调用工具、能持续执行、能闭环验证”。按能力层次看，主要包括：

1. 任务理解与规划

- 能把用户需求拆成可执行步骤
- 复杂任务会先做阶段划分、风险评估和依赖分析
- 支持长程任务的持续跟进，而不是只做一次性回答

2. 文件与代码操作

- 读取、修改、创建文件
- 执行脚本进行自动化处理
- 能在项目目录中探索代码结构、定位问题、做局部修复

3. 浏览器与网页自动化

- 可进行网页扫描与 JS 注入执行



[closed_book:] Telegram 接入指引

安装依赖

```
pip install python-telegram-bot
```

创建 Telegram Bot

1. 在 Telegram 中搜索 **@BotFather**，发送 /newbot
2. 按提示设置 Bot 名称，获取 **Bot Token**

配置并启动

3. 在 mykey.py 中填入：

```
tg_bot_token = "1234567890:ABCxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"  
tg_allowed_users = [123456789] # 你的 Telegram 用户 ID
```

4. 启动 Telegram 前端：

```
cd /path/to/GenericAgent/frontends  
python tgapp.py
```

5. 在 Telegram 中搜索你的 Bot 名称，开始对话

[notebook:] 企业微信接入指引

安装依赖

```
pip install wecom_aibot_sdk
```

注册与下载

1. 打开 [企业微信官网](#)，注册账户（或登录已有账户），下载桌面端

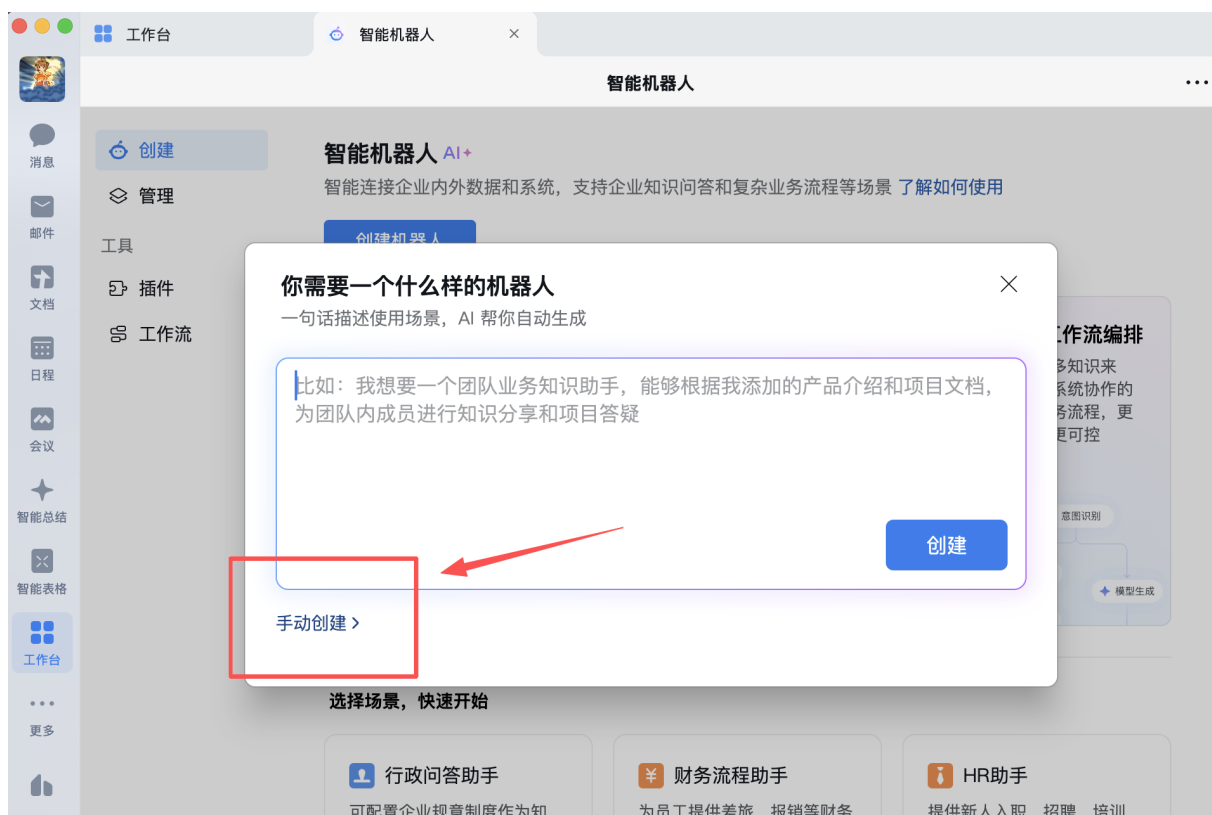


创建智能机器人

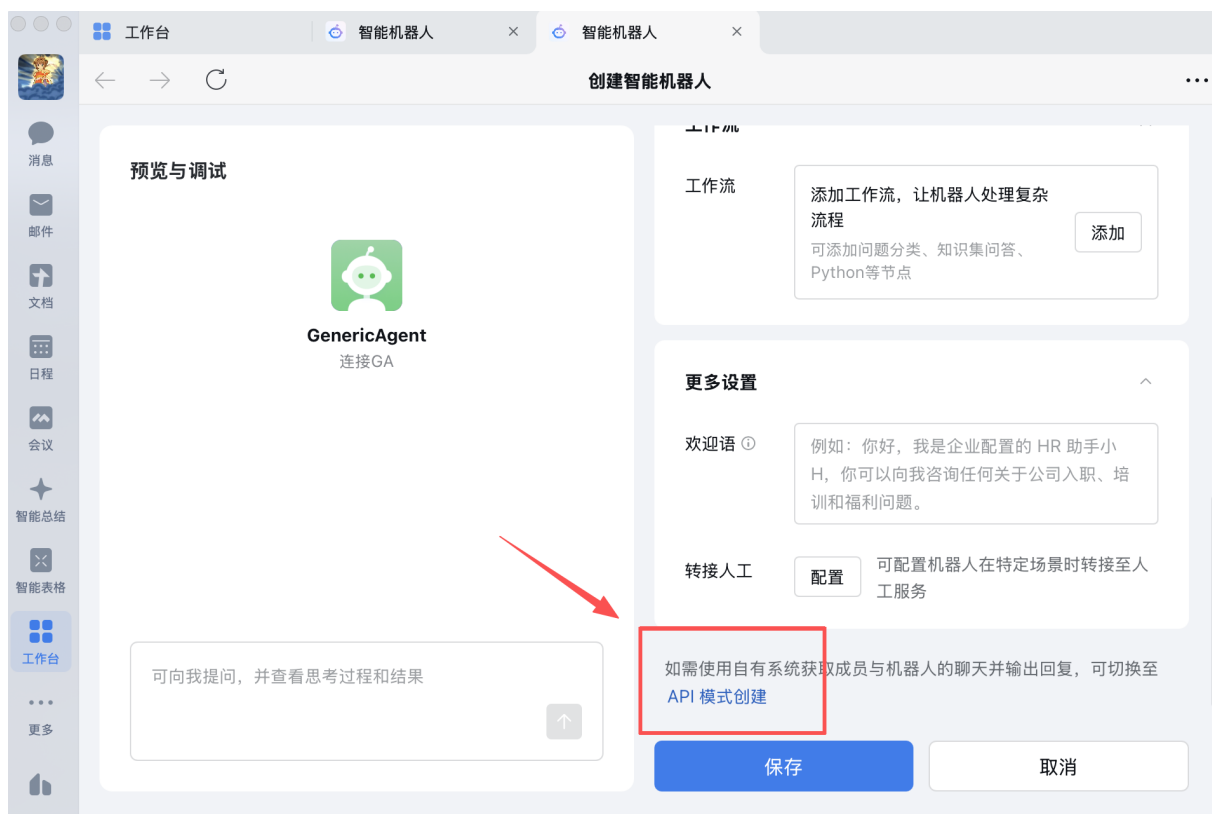
2. 登录企业微信客户端，创建智能机器人

3. 设置机器人：

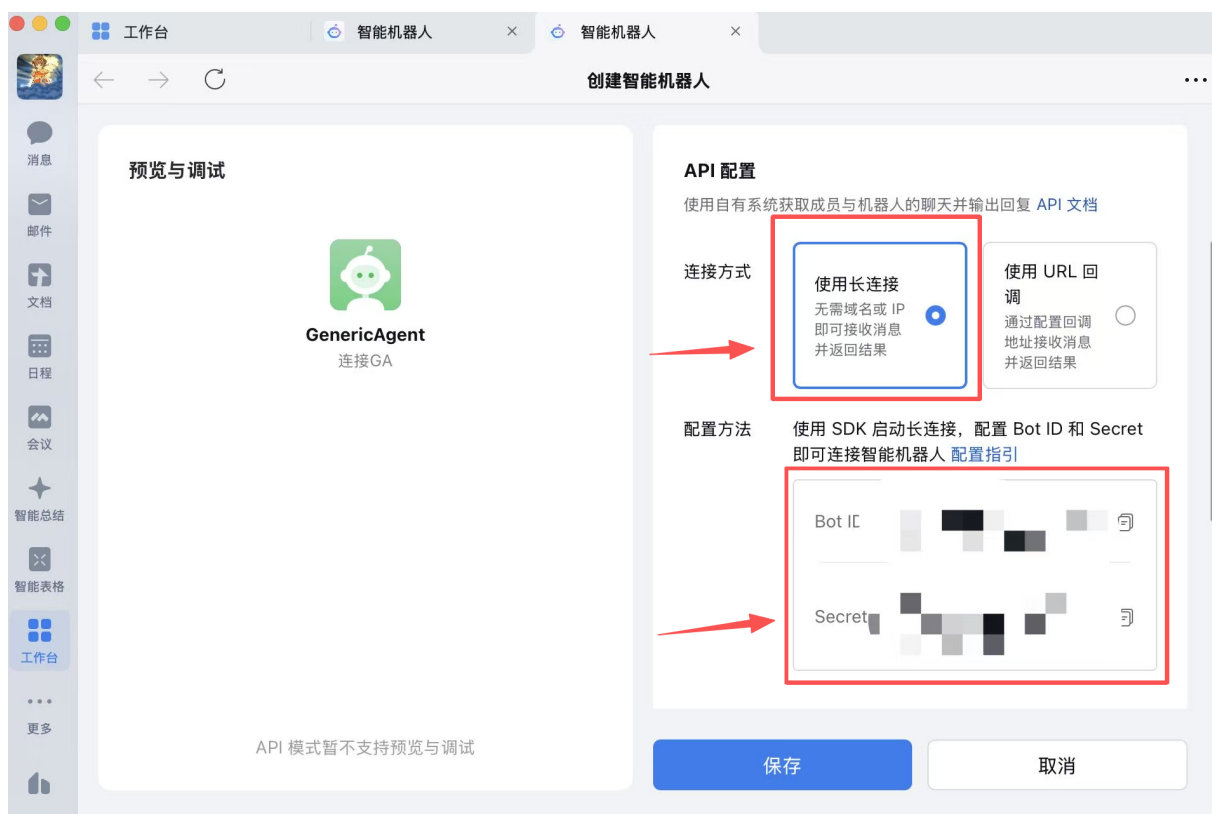
- 选择手动创建



- 滑到底部，选择 API 模式创建



- 选择**长连接**，获取 **Bot ID** 和 **Secret**



配置并启动

4. 在 `mykey.py` 中填入上述 Bot ID 和 Secret

```

390     is_allowed_users = [ ] # 留空或 ['*'] 表示允许所有企业微信用户
391     wecom_bot_id = 'your_bot_id'
392     wecom_secret = 'your_bot_secret'
393     wecom_allowed_users = [ ] # 留空或 ['*'] 表示允许所有企业微信用户
394     wecom_welcome_message = '你好，我在线上。'
395     dingtalk_client_id = 'your_app_key'

```

5. 启动企业微信前端：

```

cd /path/to/GenericAgent/frontends
python wecomapp.py

```

```
frontends — python wecomapp.py — 64x39
(claw) songkun ~ frontends % python wecomapp.py
[Info] Load mykeys from genericAgent/mykeys.py

=====

企业微信 Agent (长连接模式)
端口锁: 19531 | 允许用户: 全部

=====

终端命令: help | status | stop | exit
[WeCom] starting ...
[WeCom] connected
[WeCom] authenticated, 等待消息中 ...

[14:39:03] 📧 SongKun: 你好!
[14:39:08] ✅ Done (SongKun) - 用户打招呼, 回应
```

6. 在企业微信中找到你的机器人，开始对话

5.5 常见问题

Q1: 关闭 GenericAgent 窗口会影响 Bot 连接吗?

不影响。用户启动的是对应平台的 app.py (如 wechatapp.py)，它是独立进程，关闭 GenericAgent 窗口不会中断连接。

Q2: 怎么关闭 Bot 连接?

在运行 app.py 的终端中按 Ctrl + C 即可中断连接。中断期间发出的消息不会丢失——重新运行 python wechatapp.py 后，会按顺序回复中断期间收到的消息。

Q3: 多个平台可以同时运行吗?

可以。每个平台是独立进程，互不影响。例如同时运行微信和飞书：

```
python frontends/wechatapp.py &
python frontends/fsapp.py &
```

[[:open_file_folder:](#)] 相关文件速查

文件 / 目录	作用
frontends/wechatapp.py	微信个人号前端启动脚本
frontends/fsapp.py	飞书前端启动脚本
frontends/dingtalkapp.py	钉钉前端启动脚本
frontends/qqapp.py	QQ 机器人前端启动脚本
frontends/tgapp.py	Telegram 前端启动脚本
frontends/wecomapp.py	企业微信前端启动脚本
mykey.py	所有平台密钥的统一配置文件

[memo:] 小结

- **接入流程统一**：安装依赖 → 创建平台应用获取密钥 → 填入 mykey.py → 启动 app.py
- **推荐从微信开始**：流程最简单，扫码即用，适合快速体验
- **多平台可并行**：每个平台独立进程，互不干扰，按需启用

上一章：第 4 章 记忆与 Skill 系统 | 下一章：第 6 章 进阶功能

第 6 章 进阶功能

学完本章，你将掌握 GA 的五种进阶能力——自主行动、定时任务、反射模式、子代理和 Plan 模式。

[:bullseye:] 学习目标

- 1. 学会让 GA 在你离开时**自主执行任务**，回来看报告
- 2. 掌握**定时任务**的创建与管理
- 3. 理解**反射模式**的三要素，能让 GA 创建自定义反射脚本
- 4. 学会用**子代理**并行处理独立任务
- 5. 知道 **Plan 模式**何时触发，能审查和跟踪执行计划

6.1 自主行动模式（Autonomous）

睡前给 GA 留一张任务清单，醒来报告就写好了——这就是自主行动模式。

三步上手

第一步：写任务清单

直接用自然语言告诉 GA：

你：帮我在自主行动的 TODO 里添加一条：整理桌面上所有的 PDF 文件，按年份分到不同文件夹。

GA 会把任务写入 temp/TOD0.txt。你可以一次添加多条，但 GA **每次只做一条**，按顺序从上往下取第一条未完成的。

任务描述写得越具体越好。“整理文件”太模糊，“把桌面 PDF 按年份分到 ~/Documents/PDF/2024、2025 子文件夹”才够清晰。

[:light_bulb:] TODO.txt 长什么样

- [] 整理桌面上所有的 PDF 文件，按年份分到不同文件夹
- [] 把 Downloads 里超过 30 天的文件归档到 Archive
- [x] 清理 temp 目录下的过期缓存文件

第二步：离开电脑

不需要额外操作。GA 检测到**超过 30 分钟没收到你的消息**，就会自动从 TODO 取任务开始执行。

第三步：回来看报告

回来后直接问 GA “你刚才做了什么”，或者去 temp/autonomous_reports/ 看报告文件。报告自动编号，比如 R3_ 整理 PDF 文件.md，包含执行摘要、操作详情和遇到的问题。

权限边界

可以自主做	写在报告里等你确认	绝对不会做
读取分析文件	修改 GA 的记忆和 SOP	读取密钥文件
在临时目录写文件、跑脚本	安装新软件	修改 GA 核心代码
调用已有的 Skill	调用外部 API	不可逆的危险操作
写执行报告	删除非临时文件	

遇到拿不准的事，GA 会记在报告里等你回来决定，**不会擅自做主**。

6.2 定时任务 (Scheduled Task)

每天早上 8 点自动抓新闻、每周五整理文件——设好时间，GA 准时开干。

创建一个定时任务

直接用自然语言告诉 GA：

帮我创建一个定时任务，每天早上 9 点整理 Downloads 文件夹里的新文件，按类型分类。

GA 会在 sche_tasks/ 目录下生成一个 JSON 配置文件。

[] 配置文件长什么样

```
{
  "schedule": "09:00",
  "repeat": "daily",
  "enabled": true,
  "prompt": "整理 Downloads 文件夹里的新文件，按文件类型分到对应子文件夹里",
  "max_delay_hours": 3
}
```

每个字段的含义：

字段	说明	示例
schedule	每天几点触发	"09:00"
repeat	重复方式	见下方表格
enabled	是否启用	true / false
prompt	GA 要执行的指令	你想让它做什么，写清楚
max_delay_hours	最大延迟窗口	默认 6，超过这个时间没开机就跳过

repeat 支持以下几种写法：

值	含义
"daily"	每天
"weekday"	仅工作日（周一到周五）
"weekly"	每周
"monthly"	每月
"once"	只执行一次
"every_3h"	每 3 小时
"every_30m"	每 30 分钟
"every_2d"	每 2 天

max_delay_hours：任务定在 8 点但你 12 点才开机，设了 3 就会跳过，避免执行过时任务。

管理定时任务

问 GA " 我的定时任务都什么情况？ " 可以查看所有任务状态。

告诉 GA " 暂停整理文件那个定时任务 " 即可暂停，GA 会把 JSON 里的 enabled 改为 false。恢复、删除同理，直接说就行。

查看执行报告

每次定时任务执行完，报告会自动保存在 sche_tasks/done/ 目录下，文件名格式：

sche_tasks/done/2026-04-17_0900_整理文件.md

报告里包含本次执行了什么、结果如何。你可以直接去这个目录翻看，也可以问 GA " 上次定时任务的报告呢？ "。

注意事项

- 1. **GA 必须在运行**：关了 GA 或电脑关机，定时任务不会执行
- 2. **多窗口不冲突**：开了多个 GA 窗口也只有一个调度器在工作
- 3. **任务 JSON 在 sche_tasks/**，报告在 sche_tasks/done/，不要和 temp/autonomous_reports/ 搞混

6.3 反射模式（Reflect）

条件满足时自动触发任务，不用你盯着——这就是反射模式。

反射脚本的三个要素

所有反射脚本放在 reflect/ 文件夹下，每个都是一个 Python 文件。打开最简单的 reflect/autonomous.py：

```
# reflect/autonomous.py
INTERVAL = 1800    # 每 1800 秒（30 分钟）检查一次
ONCE = False       # 持续检查，不是只查一次

def check():
    return "[AUT0][:robot:] 用户已经离开超过 30 分钟，作为自主智能体，请阅读自动化 sop，执行自动任务"
```

要素	作用	示例值
INTERVAL	多久检查一次（秒）	1800 = 30 分钟，600 = 10 分钟
ONCE	True = 触发一次就停，False = 持续监测	False
check()	返回字符串 → 触发任务；返回 None → 本轮跳过	见上方代码

[light_bulb:] 前面学的**自主行动**（6.1）和**定时任务**（6.2）本质上都是反射脚本——autonomous.py 和 scheduler.py 就在 reflect/ 目录里。你可以用 ls reflect/ 查看所有现有脚本。

创建自定义反射脚本

直接告诉 GA 你想监测什么：

你：帮我写一个反射脚本，每 10 分钟检查桌面有没有新的 .csv 文件，有的话自动做数据分析，然后启动它。

GA 会自动完成创建和启动，你不需要自己写代码或敲命令。脚本支持**热更新**：修改后不用重启，下次检查时自动加载新版本。

6.4 子代理模式（Subagent）

把任务分给独立的”分身”去做，主 GA 只管分配和收结果。

启动一个子代理

最简单的方式——直接告诉 GA：

你：帮我用子代理分析 ~/Documents/report.pdf，提取关键结论写成摘要。

GA 会自动创建工作目录、写入任务描述、启动子代理。你不需要手动操作。

[!light_bulb:] GA 底层实际执行了什么

```
python3 agentmain.py --task analyze_report --input " 分析 report.pdf，提取关键结论" --bg
```

参数	作用
--task 名称	指定任务名，同时决定工作目录 temp/analyze_report/
--input " 描述"	任务描述（短文本直接传，长文本先写 input.txt）
--bg	后台运行，主 GA 不用等它做完
--llm_no N	指定用第几个模型配置（可选）

查看进度和干预

直接问 GA 就能查看子代理的进度，比如 " 分析报告那个子代理做完了吗？ "。

如果需要干预，也可以告诉 GA，比如 " 让分析报告的子代理停下来" 或 " 告诉那个子代理重点关注第三章"。

[!light_bulb:] 子代理的工作目录结构

子代理的所有文件都在 temp/{任务名}/ 目录下：

```
temp/analyze_report/
├── input.txt      # 任务描述（GA 自动生成）
├── output.txt     # 执行日志和结果（看到 [ROUND END] 表示完成）
├── context.json  # 文件路径清单（可选，用于复杂任务）
└── reply.txt     # 你可以写追加指令，子代理会继续执行
```

GA 通过在工作目录下创建特殊文件来干预子代理：

文件	效果
_stop	当前轮结束后停止
_keyinfo	写入重要信息，子代理下轮会读到

文件	效果
_intervene	写入新指令，子代理会调整方向

并行限制

多个子代理可以同时运行，但有一条硬约束：

- [:check_mark_button:] **纯文件处理**（分析、生成、转换）→ 放心并行
- [:cross_mark:] **需要浏览器或键鼠** → 必须排队，一个做完再做下一个

[:light_bulb:] 写好子代理输入的技巧

好的输入只需要四样东西：1. **做什么**：任务目标和产出物 2. **约束**：格式要求、工具限制等 3. **输入文件**：用**绝对路径** 4. **输出位置**：结果写到哪

不要写执行步骤——子代理和主 GA 一样聪明，会自己规划怎么做。

6.5 Plan 模式（复杂任务规划）

步骤多、有依赖、要并行——GA 会先写计划给你确认，再按步骤逐个执行。

什么时候会触发

你不需要手动开启 Plan 模式。当任务足够复杂时，GA 会自动进入。比如：

你：帮我把项目里所有 JavaScript 文件迁移到 TypeScript，保持功能不变，更新所有 import 路径。

这种任务涉及多文件、有依赖关系，GA 会自动走 Plan 模式。一般来说，满足以下任一条件就会触发：

- 步骤超过 3 个，且步骤之间有先后依赖
- 需要同时读取或修改多个文件
- 包含条件分支（“如果 X 就做 Y，否则做 Z”）

简单的 1-2 步任务，GA 会直接做，不走 Plan 模式。

Plan 模式的五个阶段

探索 → 规划 → 你确认 → 执行 → 验证

① **探索**：GA 派一个子代理去摸底——有什么文件、缺什么条件、有什么风险。子代理只看不改，把结论写成一份探索报告。

② **规划**：基于探索报告，GA 写出 plan.md，列出每一步做什么、用什么 SOP、依赖哪一步。

③ **你确认**：GA 把计划展示给你，**等你说“可以”才开始执行**。你可以要求修改步骤、调整顺序、补充遗漏。

④ **执行**：逐步执行，每完成一步打 [✓] 标记。搬砖量大的步骤会自动委托给子代理。

⑤ **验证**：全部执行完后，GA 启动一个独立的验证子代理来“挑毛病”。验证通过才算完成；失败则自动修复后重新验证。

审查计划时看什么

GA 给你展示 plan.md 后，重点关注三件事：

1. **有没有遗漏？** 你想做的事是否都覆盖了
2. **顺序对不对？** 有依赖关系的步骤是否排在前面
3. **风险步骤有没有标出？** 不可逆操作是否有保护措施

觉得哪里不对，直接告诉 GA 你的想法：

你：第 3 步应该在第 2 步之前，而且缺少备份这一步。

GA 会修改计划后再次让你确认。

跟踪执行进度

执行过程中 GA 不会主动停下来汇报。想知道做到哪了，随时问：

你：现在执行到哪一步了？

GA 会读取 plan.md，告诉你哪些步骤已经打了 [✓]、当前在做哪一步。

[open_book:] plan.md 里的步骤标记含义

标记	含义
[]	待执行
[D]	委托给子代理执行（搬砖量大的步骤）
[P]	并行执行（同时启动多个子代理）
[?]	条件分支（根据结果选路径）
[✓]	已完成
[✗]	已失败并跳过
[SKIP]	因依赖步骤失败而跳过
[FIX]	验证失败后追加的修复步骤
[VERIFY]	强制验证步骤，不可跳过

[open_book:] 验证失败后会怎样

验证子代理会给出三种判定之一：

- **PASS**：结果可用，任务完成
- **FAIL**：有问题。GA 会把修复步骤追加到 plan.md（标记 [FIX]），只修复失败项，修完后重新验证。最多重复 2 轮，仍然失败则请你介入
- **PARTIAL**：部分通过。GA 会告诉你哪些没过，由你决定是否可接受

[open_file_folder:] 相关文件速查

内容	路径
自主行动	
任务清单	temp/TODO.txt
执行报告	temp/autonomous_reports/R1_xxx.md
行动 SOP	memory/autonomous_operation_sop.md
定时任务	
任务配置	sche_tasks/任务名.json

内容	路径
执行报告	sche_tasks/done/2026-04-17_0900_ 任务名 .md
调度日志	sche_tasks/scheduler.log
任务 SOP	memory/scheduled_task_sop.md
反射模式	
反射脚本目录	reflect/
执行日志	temp/reflect_logs/
子代理	
任务描述	temp/{任务名}/input.txt
执行日志和结果	temp/{任务名}/output.txt
干预文件	temp/{任务名}/ 下的 _stop / _keyinfo / _intervene
子代理 SOP	memory/subagent.md
Plan 模式	
执行计划	plan_XXX/plan.md (XXX 为任务英文短名)
探索报告	plan_XXX/exploration_findings.md
验证上下文	plan_XXX/verify_context.json
Plan 模式 SOP	memory/plan_sop.md

[:memo:] 小结

- **自主行动**：写 TODO → 离开 30 分钟 → GA 自动干活 → 回来看报告
- **定时任务**：一句话创建，JSON 配置，sche_tasks/done/ 看报告
- **反射模式**：三要素（INTERVAL + ONCE + check()），自主行动和定时任务的底层机制
- **子代理**：独立”分身”并行处理，纯文件任务可同时跑，浏览器任务要排队
- **Plan 模式**：复杂任务自动触发，探索 → 规划 → 确认 → 执行 → 验证

上一章：第 5 章 聊天平台接入

第 7 章 上下文信息密度——GenericAgent 的第一性原理

在第一部分的应用篇中，我们已经学会了如何使用 GenericAgent (GA) 来完成各种实际任务——从文件管理到网页操作，从定时任务到子代理协作。但你是否好奇过：GA 的底层到底是怎么设计的？为什么它能在长时间、多步骤的复杂任务中依然保持稳定的表现？

从本章开始，我们将进入原理篇，深入剖析 GA 的设计哲学与核心机制。而一切的起点，需要从**长期运行 Agent 面临的两大根本挑战**说起。

两大挑战：为什么现有 Agent 不够用？

当 LLM 从单轮问答的“文本生成器”转变为在终端、文件系统、浏览器中持续运作的**目标驱动型 Agent**，两个根本性的系统挑战随之浮现：

挑战一：上下文爆炸 (Context Explosion)。随着 Agent 与环境交互，提示词不断膨胀——工具定义、检索到的记忆、中间观察结果、历史轨迹、网页原始内容层层堆叠。虽然这些组件在产生时各有用途，但它们对下一步决策的价值参差不齐。那些与当前推理无关的内容不仅被浪费，还会主动**稀释模型的注意力**，引发约束遗漏、状态混淆和幻觉等错误。

挑战二：经验无法有效积累与复用 (Experience Accumulation)。在真实的长期运行环境中，大量有价值的知识并非一开始就存在，而是通过与特定用户的文件系统、工作流程和外部服务的反复交互中逐渐浮现的。如果 Agent 无法蒸馏和保留这些经验，每次遇到类似任务都得以从头探索——耗费大量 token 却毫无能力增长。久而久之，系统陷入一种**停滞循环 (Stagnation Loop)**：尽管持续交互，性能却纹丝不动。

那么，要同时应对这两大挑战，关键的设计支点是什么？答案是一个看似简单却极其关键的问题：**LLM 的上下文到底应该塞什么？**

7.1 一个反直觉的事实：上下文越长，表现越差

7.1.1 直觉理解

想象你正在参加一场闭卷考试。你被允许带一张 A4 大小的“小抄”进考场。你会怎么写这张小抄？

- 方案 A：把课本的每一页都缩印上去，字小到几乎看不清
- 方案 B：只写你最容易忘记的公式和关键概念，字大清晰

显然，大多数人会选择方案 B。虽然方案 A 包含了“所有信息”，但在考试的有限时间内，你根本来不及从密密麻麻的文字中找到需要的内容。更糟糕的是，过多的无关信息反而会干扰你的判断。

LLM Agent 面临的处境与此极其相似。虽然现代 LLM 的上下文窗口越来越大（从 4K 到 200K 甚至更长），但**有效上下文远小于名义窗口**。塞进去的信息越多，不代表模型的决策质量越高——恰恰相反，大量无关信息会显著降低模型的推理准确率。这个答案或许反直觉，但已被多项研究证实。接下来我们看看背后的三个技术原因。

7.1.2 深入理解：LLM 的三重上下文陷阱

研究者们发现，LLM 在处理长上下文时存在三种相互叠加的失效模式：

第一重陷阱：位置偏差 (Lost-in-the-Middle)

LLM 对上下文中不同位置的信息关注度并不均匀。它们倾向于更好地利用上下文开头和结尾的信息，而**中间部分的信息往往被“遗忘”**。这意味着，即使关键信息确实存在于上下文中，只要它恰好落在中间位置，模型就可能视而不见。

这个现象被形象地称为” Lost-in-the-Middle” ——信息迷失在上下文的中间地带。Liu et al. (2024) 的实验表明，当关键信息从上下文的开头/结尾移到中间位置时，模型的准确率会显著下降。

第二重陷阱：注意力稀释（Attention Dilution）

LLM 的注意力机制是一种有限资源。当上下文中充斥着大量无关内容时，模型分配给每条关键信息的注意力就会被稀释。这就好比在嘈杂的鸡尾酒派对上试图听清一个人说话——背景噪音越大，越难捕捉到有价值的信号。需要强调的是，无关内容不仅仅是被浪费了——它会**主动干扰**模型对关键证据的利用。

第三重陷阱：有效窗口远小于名义窗口

一个标称 128K token 上下文窗口的模型，其”有效利用”的上下文可能远小于这个数字。随着上下文增长，模型的推理能力会逐渐退化。换句话说，**上下文窗口的”名义容量”和”有效容量”之间存在巨大鸿沟。**

7.1.3 恶性循环：三重陷阱如何相互强化

这三重陷阱并非简单并列——它们在实践中形成一个**相互强化的恶性循环**。我们可以追踪这个循环的因果链条：

- 1. **上下文不断增长** → 无关内容占比上升
- 2. **无关内容增多** → 注意力被稀释，模型更难从中间位置检索关键信息（位置偏差加剧）
- 3. **位置偏差加剧** → 模型越来越依赖开头和结尾的信息，有效窗口进一步收缩
- 4. **有效窗口收缩** → 为补偿信息损失，系统倾向于注入更多”可能有用”的上下文
- 5. **回到第 1 步** → 上下文进一步膨胀，循环加剧

这个恶性循环的核心启示是：**超过某个临界点后，增加更多上下文不仅无法提升性能，反而会降低表现。**

这一洞察对 Agent 系统设计有直接启示。现有的一些 Agent 框架采用整块式提示词组装策略——把脚手架文本、控制指令和大量边际相关的交互历史一股脑塞进上下文。这种做法不是在暴露当前决策真正需要的信息，而是把最需要的证据**挤出了**有效上下文窗口。结果就是：上下文在体量上更大了，但在效用上反而更低了，同时推理成本还在持续攀升。

那么，“上下文爆炸”到底有多严重？我们可以看一组实际数据。在安装了 20 个技能并经过长时间使用后，仅对一个最简单的“Hello”请求，各系统的完整提示词长度如下（数据来源：论文 Table 7）：

系统	完整提示词长度 (tokens)
OpenClaw	43,321
CodeX	23,932
Claude Code	22,821
GA	2,298

GA 的提示词长度只有其他系统的 **1/10 到 1/20**。这就是系统性控制上下文信息密度的效果——即使经过长期使用和大量技能积累，GA 的上下文也不会失控膨胀。

7.2 GenericAgent 的核心设计约束：上下文信息密度最大化

7.2.1 直觉理解

既然”塞得多”不等于”用得好”，那 GA 的设计哲学就很清晰了：**不追求上下文的长度，而追求上下文的信息密度（Information Density）。**

什么是信息密度？我们可以用一个简单的比喻来理解：

- **低信息密度上下文** = 一份冗长的会议记录，包含了每个人的寒暄、跑题讨论和重复发言
- **高信息密度上下文** = 一份精炼的会议纪要，只保留了决策事项、关键结论和行动计划

GA 的所有设计决策，都围绕一个核心目标展开：**在有限的上下文预算内，最大化每一个 token 对当前决策的贡献**。我们可以用一个形式化的视角来表达这个设计目标：

$$\max_{\mathcal{C}} D(\mathcal{C}) = \frac{\text{决策相关信息量}(\mathcal{C})}{\text{上下文总长度}(\mathcal{C})}$$

其中：

- $\mathcal{C}$ 是当前上下文的内容组合
- $D(\mathcal{C})$ 是上下文的信息密度
- 分子衡量“有用信息有多少”——对应完备性
- 分母衡量“总共塞了多少”——对应简洁性

需要说明的是，这个公式是教程为便于理解而构造的形式化表达，用于直觉地传达 GA 的设计理念。在实际系统中，GA 并不显式地计算信息密度，而是通过多层机制从不同角度来逼近这个目标。

7.2.2 深入理解：完备性与简洁性的权衡

从上下文工程（Context Engineering）的视角来看，上下文质量可以用两个主要需求和一个辅助约束来刻画：

完备性（Completeness）：当前决策所需的所有信息都必须**显式地**出现在上下文中，防止模型依赖隐含假设或产生幻觉补全。如果关键信息被遗漏，模型就会“巧妇难为无米之炊”。

简洁性（Conciseness）：无关和冗余的信息必须被清除，使注意力始终聚焦在决策关键的信号上。每多一条无关信息，就多一分注意力稀释。

自然性（Naturalness）：上下文应当保持合理的自然语言形态和语义可读性。但需要指出，自然性是一个**重要但次要的约束**——在大多数 Agent 失败场景中，主要的失败模式不是“措辞不够自然”，而是完备性或简洁性的丧失。

图 7-1：完备性与简洁性定义了上下文设计的核心权衡，而自然性作为有效表征的辅助约束。左图完备但过度冗长，关键信息被淹没；右图简洁但遗漏关键信息；中间为 GA 的平衡方案——完备且简洁，自然性自然跟随。

上下文工程的核心挑战，在于平衡完备性与简洁性。其他常被讨论的维度——如时效性（Recency）和相关性（Relevance）——可以在这个框架下自然理解：时效性同时服务于完备性（最新状态往往是当前决策所必需的）和简洁性（过时信息通常不再相关）；相关性则是一个横切关注点，完备性和简洁性都需要基于“什么是相关的”来做判断。

7.2.3 核心洞察：结构性张力而非预算问题

理解了完备性和简洁性之后，我们来到本章最关键的洞察。

完备性和简洁性之间存在一个**根本性的张力**。你可能会以为：这个矛盾只是因为上下文窗口太小了——等模型有了无限大的窗口，问题就解决了。但事实并非如此。**即使上下文窗口是无限大的，这个张力依然存在**。这是一个**结构性矛盾**，而非仅仅是资源预算问题。

为什么？至少有三个原因：

1. **加入更多可能相关的信息，提升了完备性，却削弱了简洁性**——因为“可能相关”并不等于“当前需要”，多余的信息必然稀释注意力
2. **摘要和压缩提升了简洁性，却可能损害完备性**——任何压缩都有信息损失的风险，可能恰好丢弃了下一步决策需要的关键细节

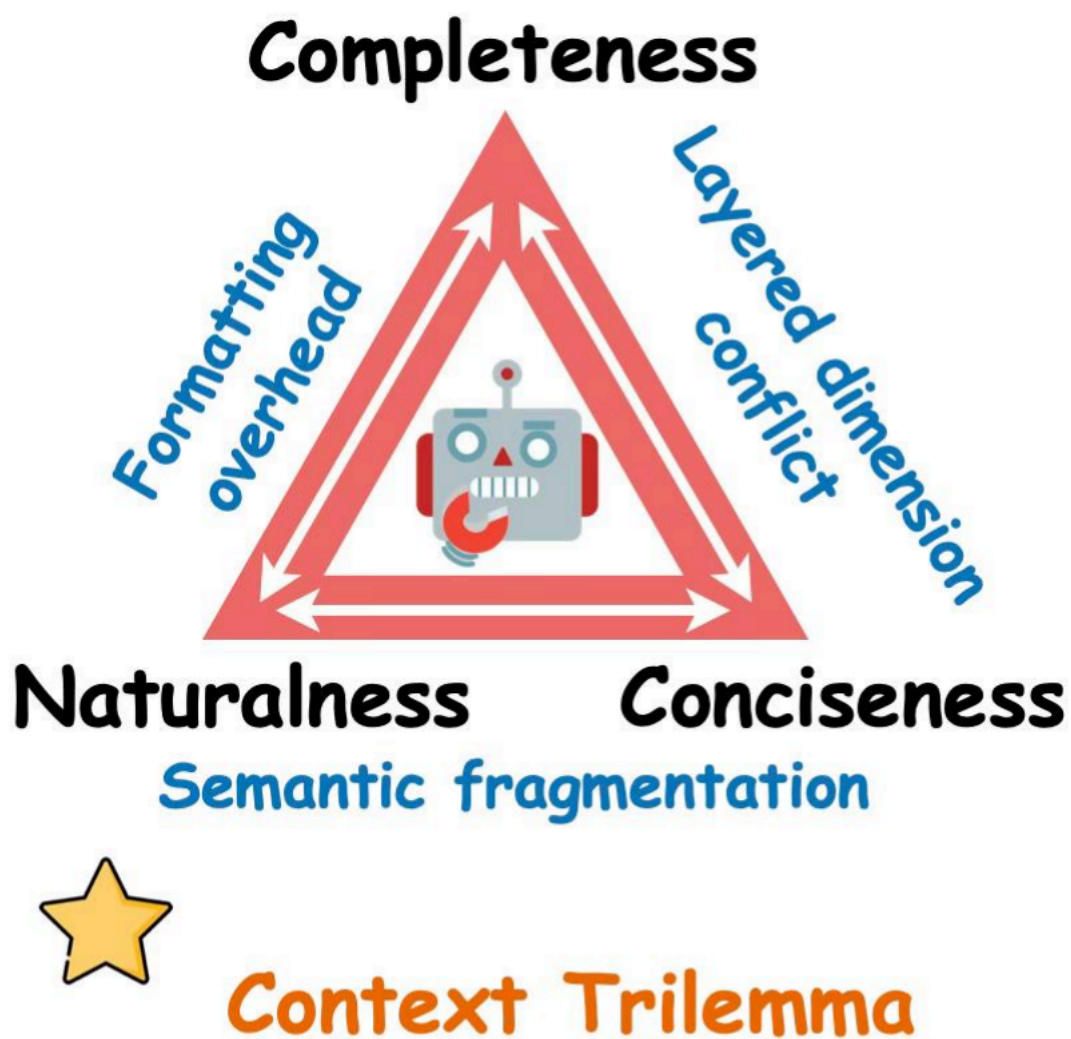


Figure 1: 完备性与简洁性的核心权衡

3. 自然性在某些情况下会加剧这个权衡——高度压缩或人工编码的表示可能更难被模型解读，但这通常是次要效应而非主要约束

有限的有效上下文窗口 把这个结构性张力进一步激化为一个硬约束——但底层的冲突是结构性的，而非单纯的资源驱动。在这种结构性张力下，上下文工程不应被看作完备性-简洁性-自然性的三元等价博弈，而是以**平衡完备性与简洁性为核心**、以自然性为辅助表征约束的**约束优化问题**。

7.3 四层信息密度优化：GA 的系统性解法

那么，GA 具体是如何在系统层面应对完备性与简洁性的结构性张力，最大化上下文信息密度的？答案是通过**四个核心机制**，分别在信息生命周期的不同阶段进行优化：

机制一：最小原子工具集——减少”先天噪声”

在每一轮对话中，LLM Agent 都需要在上下文中包含工具定义（告诉模型有哪些工具可用、每个工具的参数是什么）。工具越多，这些定义占用的 token 就越多——而且这些 token **每轮都会重复出现**。

GA 的策略是极端克制：只保留**9 个原子工具**，覆盖文件操作、代码执行、网页交互、记忆管理和人机协作五大能力类。通过最小化工具数量，GA 从源头上减少了上下文中的”固定开销”——在任务执行开始之前，就让低价值的接口信息不再占用上下文预算。

我们将在第 9 章详细展开工具集的设计原理。

机制二：分层按需记忆——只加载”当前需要的”

传统的 Agent 要么不保留历史（每次从零开始），要么把所有历史都塞进上下文（信息爆炸）。GA 采用了一种分层记忆架构：将长期知识按重要性和类型分为多个层次，只在需要时按需加载到上下文中。

这就好比一个图书馆：你不会把所有书搬到桌上才开始工作，而是先查目录（索引层），再去书架取需要的那本（事实层/SOP 层）。这种设计决定了**哪些信息随时间持久保留**，而不是让所有保留的信息都自动成为提示词的一部分。

我们将在第 10 章详细介绍分层记忆的设计。

机制三：上下文截断与压缩——主动”瘦身”

即使工具定义和记忆加载都经过了精心控制，随着对话轮次的增加，上下文仍然会不断膨胀。GA 通过一套四阶段的压缩流水线，在工具返回值截断、历史轮次压缩、消息驱逐、工作记忆锚点注入等多个层面主动”瘦身”，确保活跃上下文在对话过程中始终保持简洁和任务相关。

我们将在第 11 章详细剖析这套压缩机制。

机制四：反思驱动的自我进化——让”未来的上下文”更精炼

GA 最独特的机制之一是自我进化：它能将成功的任务经验蒸馏为可复用的 SOP、代码和技能，存入长期记忆。这意味着，当下次遇到类似任务时，GA 不需要在上下文中重新探索整个解决方案，而是直接调用之前积累的精炼经验——将交互轨迹逐步转化为紧凑的、结构化的能力。

这正是对”挑战二——经验无法积累”的系统性回应。没有进化机制时，Agent 在重复任务上面临两难：要么重放更长的探索过程（削弱简洁性），要么从更短但信息不足的提示词开始（削弱完备性）。自我进化创造了一种打破这个两难的途径。

我们将在第 12 章深入探讨自我进化的原理。

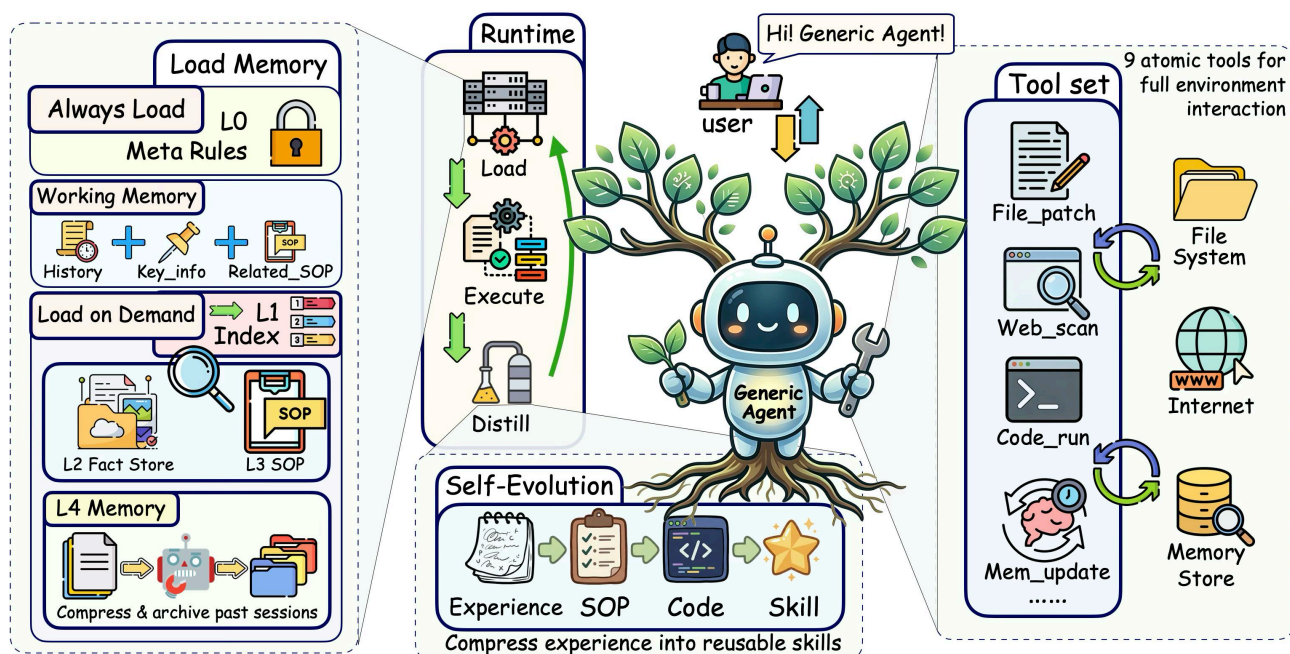


Figure 2: GA 整体框架

下面这张图展示了四个机制如何在信息生命周期的不同阶段协同工作：

图 7-2：GenericAgent 整体框架。GA 围绕统一 Agent 循环构建，从当前任务和记忆中组装执行上下文，生成输出或工具调用，并通过结构化反馈更新系统状态。四大核心机制——最小原子工具集（右侧）、分层记忆（左侧）、自我进化（底部）和上下文截断与压缩（贯穿运行时）——在信息生命周期的不同阶段协同优化信息密度。

7.4 从” Prompt Engineering” 到” Context Engineering”

在看过四大机制的全景之后，我们可以从更高的视角来理解 GA 的设计——它代表的是一种范式转变。

在 LLM 应用的早期阶段，大家关注的是**提示工程（Prompt Engineering）**——如何写出更好的提示词来引导模型输出。这在单轮问答或简单任务中非常有效。

但在 Agent 场景下，仅靠一段精心设计的提示词已经远远不够。Agent 的上下文不仅包含用户指令，还包含工具定义、历史对话、记忆内容、工具返回值等多种组件。如何系统性地管理这些组件的注入、压缩和替换，才是决定 Agent 长期表现的关键。

这种更高层次的设计思维，就是从” Prompt Engineering” 到**上下文工程（Context Engineering）**的转变。GA 的整个设计理念，正是” Context Engineering” 思想的一种系统性实践。

如果说 Prompt Engineering 是在优化”一句话怎么说”，那么 Context Engineering 就是在优化”每一轮对话中，模型看到的所有信息应该是什么”。正如我们在 7.2 节中所分析的，这本质上是一个以平衡完备性与简洁性为核心的约束优化问题。GA 通过工具层、记忆层、压缩层和进化层四个维度，将 Context Engineering 落实为具体的系统机制。

7.5 本章小结

本章介绍了 GenericAgent 最核心的设计原则——**上下文信息密度最大化**。

核心洞察是：完备性与简洁性之间的张力是**结构性的**——即使上下文窗口无限大，这个矛盾依然存在。GA 的所有设计都是围绕如何在这一结构性张力下最大化信息密度展开的。

围绕这一原则，GA 设计了四个核心机制，分别在信息生命周期的不同阶段优化信息密度：

机制	作用阶段	核心策略	详见章节
最小原子工具集	工具定义注入时	用 9 个工具覆盖全部能力，减少固定开销	第 9 章
分层按需记忆	记忆加载时	按层次存储，按需检索，不预加载	第 10 章
上下文截断与压缩	对话过程中	四阶段流水线，主动”瘦身”	第 11 章
反思驱动自我进化	跨任务经验蒸馏	经验蒸馏为 SOP/代码/技能，优化未来上下文	第 12 章

在接下来的章节中，我们将首先俯瞰 GA 的系统全貌，然后逐一深入每个机制的设计原理。那么，让我们先从一个问题开始：GA 整体长什么样？

第 8 章 GenericAgent 系统全貌

在上一章中，我们建立了 GenericAgent (GA) 的第一性原理——**上下文信息密度最大化 (Contextual Information Density Maximization)**，并了解了 LLM 在处理长上下文时面临的三重陷阱。我们还预告了 GA 通过四个核心机制来优化信息密度。

那么，GA 究竟是如何围绕这个原则构建整个系统的？在深入每个机制之前，我们先来鸟瞰 GA 的全貌——看看它的整体架构长什么样，各模块之间如何协作。本章的目标是为你建立一张“系统地图”，后续章节则相当于逐个区域深入探索。

GA 的一个重要设计决策是**模型无关 (Model-Agnostic)**：底层的推理引擎 (Claude、GPT、Gemini 等) 可以替换，而不影响执行逻辑、工具接口或记忆架构。这意味着，当更强的模型出现时，整体架构不需要改变，只需替换推理引擎即可获得能力提升。GA 的系统设计与具体模型的能力完全解耦。

8.1 统一 Agent 循环

8.1.1 直觉理解：像人一样工作

在理解 GA 的架构之前，我们先想想一个人是怎么完成一项复杂工作的：

1. **接到任务**：老板说“帮我整理一份季度报告”
2. **回忆相关经验**：你想起上次做报告时用了哪个模板、数据放在哪个文件夹
3. **思考策略**：先收集数据，再填模板，最后排版
4. **动手执行**：打开电脑，查找文件，复制数据
5. **观察结果**：看看填进去的数据对不对，格式是否正确
6. **决定下一步**：如果数据有误，回去重查；如果没问题，继续下一节

这个“接任务 → 调记忆 → 想策略 → 动手做 → 看结果 → 下一步”的循环会一直重复，直到任务完成。

GA 的工作方式与此极其相似。它的核心是一个**统一 Agent 循环 (Unified Agent Loop)**——一个连续的“推理 → 行动 → 观察”闭环，驱动 GA 完成从简单文件操作到复杂多步任务的一切工作。

8.1.2 深入理解：执行上下文的构建

让我们更精确地看看这个循环的每一步发生了什么。

第一步：构建执行上下文 (Execution Context)

每一轮循环开始时，GA 将**全局记忆**与**当前任务规格**组合，构建执行上下文。这个上下文不是简单地把用户输入转发给 LLM，而是由多个组件精心组装而成：

执行上下文 = 系统提示词 (System Prompt)

- + 始终在线记忆 (Always-on Memory: 元记忆 + L1 索引 + 工作目录上下文)
- + 工具定义 (9 个原子工具的 Schema)
- + 当前任务规格 (用户指令 + 约束条件)
- + 历史对话 (经过压缩的交互记录)
- + 工作记忆 (当前目标、约束、进度)

注意这里的每个组件都经过了信息密度的考量：始终在线记忆只注入索引层而非全量知识，工具定义只有 9 个而非几十个，历史对话经过了截断和压缩。这正是第 7 章所讲的“上下文信息密度最大化”原则在实践中的体现。

第二步：LLM 推理与决策

构建好的执行上下文被送入 LLM。模型基于这些信息进行推理，产出两种可能的结果：

- **直接输出**：模型认为可以直接回答用户问题，不需要使用工具（比如解释一个概念）

- **工具调用 (Tool Call)**：模型决定调用一个或多个工具来获取信息或执行操作（比如读取一个文件、运行一段代码）

第三步：工具执行与结构化反馈

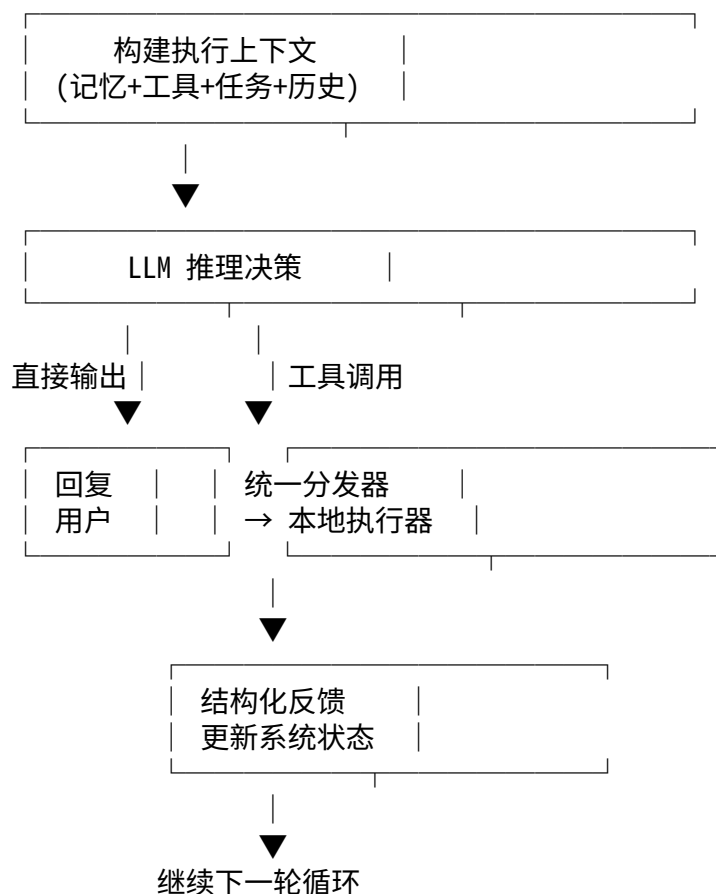
如果模型发出了工具调用请求，**统一分发器 (Unified Dispatcher)** 会将请求路由到对应的本地执行器。执行完成后，结果以**结构化信号 (Structured Signal)** 的形式返回，更新系统状态。

这里的”结构化”很关键——工具不是简单地返回一段原始文本，而是返回经过格式化的结果，包含执行状态（成功/失败）、核心输出内容、以及可能的元数据（如页面变化提示）。这使得模型可以快速判断执行结果，而不需要从一大段原始输出中”猜”发生了什么。

第四步：状态更新与循环继续

工具返回的结构化信号被追加到对话历史中，成为下一轮执行上下文的一部分。模型据此决定是否需
要继续执行（再调一个工具）、调整策略（换一种方法），还是任务已经完成（向用户汇报结果）。任务完成后，执行轨迹会被压缩为结构化的长期表征，存入共享记忆，使积累的经验可以通过压缩和复用来改善未来的执行。

这个循环创建了推理与环境反馈之间的持续交互，论文将其称为**工具锚定的迭代执行 (Tool-Grounded Iterative Execution)**。我们可以用一个流程图来概括：



这个循环看起来简洁，但它的威力在于**通用性**：无论是读写文件、操作浏览器、执行代码、还是管理记忆，所有任务都通过同一个循环来驱动。不需要为不同类型的任务设计不同的执行流程。

8.2 两种执行模式

GA 的统一循环支持两种不同的启动方式，论文称之为**两种执行模式 (Execution Modes)**：

8.2.1 交互模式 (Interact Mode)

这是最常见的模式——**用户发起任务，GA 响应执行**。你在第一部分应用篇中使用的所有场景都属于交互模式：

- 用户说” 帮我把桌面的 PDF 按年份整理到文件夹”
- GA 接收指令，开始执行 Agent 循环
- 每完成一步，GA 可能向用户汇报进度或请求确认
- 任务完成后，GA 返回结果

交互模式的特点是**用户驱动**：需要用户指令来触发，涵盖代码执行、信息检索、文件操作等任务。GA 在执行过程中可以随时通过 ask_user 请求人类介入。

8.2.2 反射模式 (Reflect Mode)

这是 GA 的另一面——**无需用户指令，自动触发的后台任务**。反射模式持续监控环境变化，当检测到特定条件或事件时自动触发相应任务，包括两种触发方式：

1. **看门狗 (Watchdog)**：监控环境变化（如新文件出现、错误日志产生），一旦检测到就立即触发 GA 任务
2. **定时任务 (Scheduled Task)**：基于时间规则在特定间隔或精确时间点生成 GA 任务

反射模式的实现机制非常简洁：一个轻量级脚本定期检查条件，当规则被触发时，脚本将返回的字符串作为标准任务分发给 GA CLI。

关键设计决策：两种模式共享同一套基础设施。交互模式和反射模式使用完全相同的 Agent 循环和记忆系统，仅在触发方式和输出处理上有所不同。这意味着：

- 反射模式可以使用与交互模式完全相同的工具来读写文件、执行代码
- 反射模式产生的结果和记忆更新，会在下一次交互执行中被自动利用
- 不需要维护两套独立的系统

8.2.3 执行边界与可控性

为了支持长期运行，GA 定义了明确的**执行边界 (Execution Bounds)**，在保持可中断性和范围约束的同时确保可控性：

- **轮次上限**：每次任务执行有最大轮次限制，超过后强制终止或请求用户介入
- **故障升级策略**：当执行遇到失败时，GA 遵循三步渐进升级——① 分析错误信息，做小幅修正后重试；② 若持续失败，切换到全新策略或搜索环境中缺失的信息；③ 若所有自动化尝试均失败，暂停并请求人类介入
- **可中断性**：用户可以随时中断执行，GA 会保存当前进度到工作记忆

这些边界确保了 GA 在拥有强大自主能力的同时，始终处于可控范围内。

8.3 四大核心机制预览

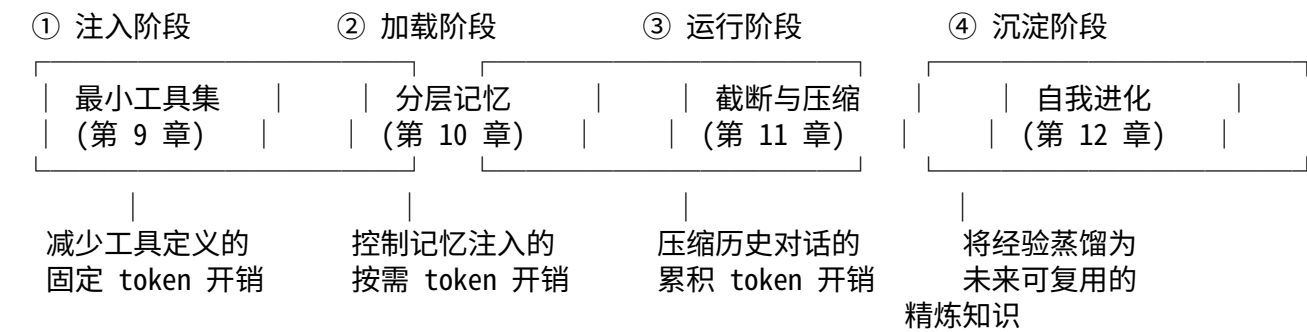
在统一循环和两种执行模式的基础上，GA 围绕” 上下文信息密度最大化” 这一核心目标，设计了四个协同工作的核心机制。下表给出每个机制的一句话定位：

机制	核心问题	一句话定位	详见章节
最小原子工具集	Agent 需要多少工具？	用 9 个原子工具覆盖完整能力环，从源头减少上下文固定开销	第 9 章
分层记忆架构	如何管理跨任务知识？	四层分级存储 + 按需加载，让” 记住更多” 不再等于” 上下文更长”	第 10 章

机制	核心问题	一句话定位	详见章节
上下文截断与压缩	对话越来越长怎么办？	四阶段流水线主动“瘦身”，确保每轮上下文都在预算之内	第 11 章
反思驱动的自我进化	能不能越用越聪明？	将成功经验蒸馏为可复用的 SOP 和代码，让未来的上下文更精炼	第 12 章

这四个机制**不是独立拼凑的功能模块，而是围绕同一目标协同设计的有机整体**。它们分别在信息生命周期的不同阶段发挥作用：

信息的生命周期



下面这张图展示了 GA 的整体架构，以及四个核心机制如何在统一循环中协同工作：

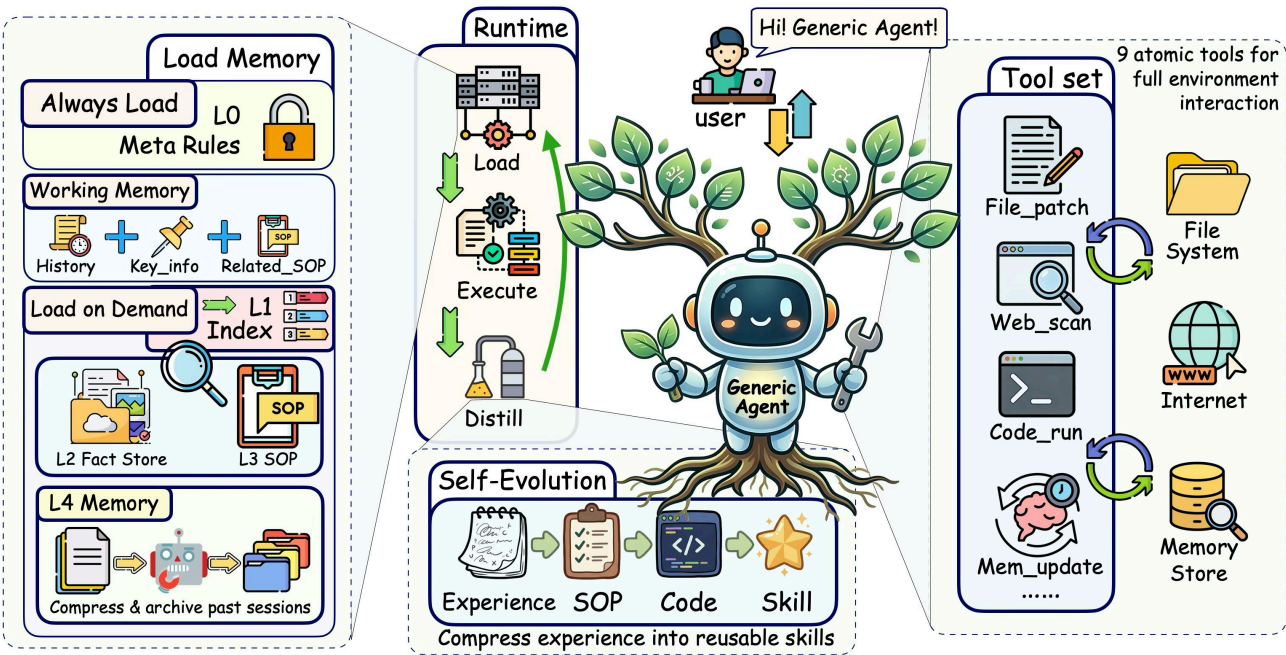


Figure 3: GA 整体框架

图 8-1: GenericAgent 整体框架。GA 围绕统一 Agent 循环构建，从当前任务和记忆中组装执行上下文，生成输出或工具调用，并通过结构化反馈更新系统状态。四大核心机制——最小原子工具集（右侧）、分层记忆（左侧）、自我进化（底部）和上下文截断与压缩（贯穿运行时）——在信息生命周期的不同阶段协同优化信息密度。（来源：论文 Figure 2）

8.4 从全貌到细节：一个具体例子

为了让上面的抽象架构更加具体，我们来追踪一个完整的任务执行过程，看看各个模块是如何协作的。

场景：用户说” 帮我从 HuggingFace 下载 IMDB 数据集”。

第一轮循环：

1. **构建上下文：**系统注入始终在线记忆（L1 索引中包含” HuggingFace 下载” 相关的 SOP 指针）、9 个工具的 Schema、用户指令
2. **LLM 推理：**模型发现 L1 索引中有相关 SOP，决定先读取它
3. **工具调用：**`file_read(path="memory/L3/huggingface_download_sop.md")`
4. **结构化反馈：**返回 SOP 内容（经 `file_read` 截断控制，只返回相关部分）

这里模型通过 L1 索引中的指针定位到 L3 层的 SOP 文件，然后用 `file_read` 按需加载——这就是第 10 章将详细介绍的 L1→L3 路由链。

第二轮循环：

1. **构建上下文：**上一轮的工具返回结果被追加到历史中
2. **LLM 推理：**模型根据 SOP 中的步骤，决定执行下载代码
3. **工具调用：**`code_run(script="from datasets import load_dataset; ds = load_dataset('imdb')...")`
4. **结构化反馈：**返回执行结果（成功/失败 + 输出摘要）

第三轮循环：

1. 模型确认下载成功，向用户汇报结果
2. 任务完成

幕后（自我进化）：

- 如果这是 GA 第一次执行 HuggingFace 下载任务，GA 会在任务成功完成后通过 `start_long_term_update` 将这次成功的执行经验蒸馏为一条新的 SOP，存入 L3 记忆层
- 下次遇到类似任务时，L1 索引会路由到这条 SOP，GA 可以直接按 SOP 执行，跳过探索阶段

注意在这个例子中，四个机制都参与了工作：**工具集**提供了 `file_read` 和 `code_run` 两个原子操作；**分层记忆**让 GA 快速定位到相关 SOP；对话历史经过**截断与压缩**控制在合理范围内；**自我进化**将成功经验沉淀为可复用的 SOP 知识。

8.5 本章小结

本章从全局视角介绍了 GA 的系统架构。我们了解到：

1. **统一 Agent 循环**是 GA 的执行引擎——“构建上下文 → LLM 推理 → 工具执行 → 结构化反馈”的闭环驱动所有任务
2. **两种执行模式**（交互模式 + 反射模式）共享同一套基础设施，分别负责用户驱动的任务执行和环境驱动自动触发
3. **四大核心机制**（工具集、记忆、压缩、进化）围绕信息密度最大化协同设计，在信息生命周期的不同阶段分别发挥作用

现在，你已经拥有了 GA 的”系统地图”。从下一章开始，我们将沿着信息生命周期的顺序，逐一深入每个核心机制。

第一站，是 GA 最引人注目的设计选择：**为什么只用 9 个工具就够了？**

第 9 章 最小原子工具集——用 9 个工具覆盖完整能力环

在前面的章节中，我们建立了 GA 的第一性原理——**上下文信息密度最大化**（第 7 章），并从全局视角了解了 GA 的系统架构和四大核心机制的协作方式（第 8 章）。从本章开始，我们将沿着信息生命周期的顺序，逐一深入每个核心机制。

第一站，是信息进入上下文之前的第一道关口：**最小原子工具集（Minimal Atomic Toolset）**。工具定义是上下文中**每轮都要重复支付的固定成本**——在任务执行开始之前，这些定义就已经占据了宝贵的上下文预算。因此，工具层的设计直接决定了上下文的”先天信息密度”。

一个很自然的问题是：既然 Agent 需要与外部世界交互，那为什么不给它尽可能多的工具？工具越多，能力不就越强吗？

9.1 工具膨胀的代价

9.1.1 直觉理解

想象你走进一家五金店。你需要拧一颗螺丝，店员递给你一本 200 页的工具目录，里面有 53 种不同的螺丝刀——十字的、一字的、六角头的、带棘轮的、电动的、可弯曲的……你需要先花 10 分钟翻阅目录，理解每种螺丝刀的区别，然后才能做出选择。

而另一家店只给你一把通用螺丝刀和一个万能批头套装。你拿起来就能干活。

对于 LLM Agent 来说，“翻阅工具目录”的代价远比你想象的大。因为**每一轮对话**中，所有可用工具的定义都会被注入到上下文中。工具越多，这些定义占用的 token 就越多——而且这是**每轮都要重复支付的固定成本**。

9.1.2 深入理解：工具膨胀的两层代价

论文指出，工具膨胀在两个层面同时产生系统性的负面影响：

第一，提示词层面（Prompt Level）的上下文预算挤占。每增加一个工具，就需要在上下文中注入该工具的 Schema（名称、描述、参数类型、约束说明）和相关指令。在多轮交互中，这些开销**每轮累积**，直接挤占了本应留给任务信息的有效上下文预算。以 Claude Code 为例，53 个工具的 Schema 可能消耗数千甚至上万 token——这还不包括伴随的使用说明文本。

第二，策略层面（Policy Level）的决策复杂度增长。每增加一个工具，就扩大了模型的动作空间，增加了工具选择时的歧义性。比如，FileReadTool 和 GrepTool 都可以读取文件内容，BashTool 也可以通过 cat 命令实现同样的效果。模型需要理解这三个工具的微妙差异才能做出正确选择。这种歧义会**削弱工具使用模式的稳定性，增加执行错误和不必要重试的概率**。

这两层代价共同作用的结果，可以从实验数据中直观看到。即使暴露了几十个工具，真正被频繁调用的往往只是少数几个：

Agent 系统	工具总数	最常用工具及占比
Claude Code	53 个	AgentTool (50.4%)、WebFetchTool (22.1%)、FileReadTool (10.6%)
OpenClaw	18 个工厂	browser (32.5%)、exec (20.5%)、web_fetch (15.7%)
GA	9 个	code_run (34.4%)、file_read (31.2%)、update_working_checkpoint (17.2%)

Claude Code 虽然暴露了 53 个工具，但超过 80% 的调用集中在仅仅 3 个工具上，剩余 47 个工具几乎从不被使用。这意味着这 47 个工具的 Schema **每轮都在消耗上下文预算，却几乎不贡献决策价值**——这正是上下文信息密度的反面。

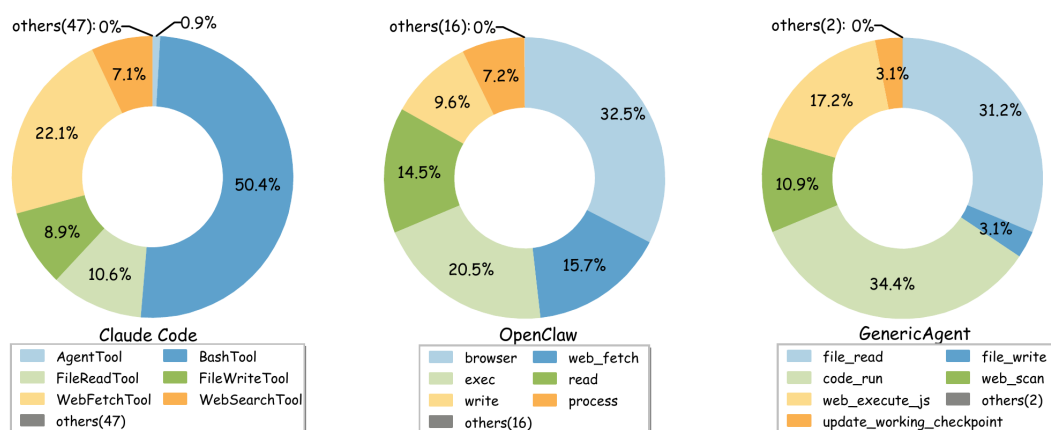


Figure 4: 工具使用分布

图 9-1: Claude Code、OpenClaw 和 GA 的工具使用分布。尽管 Claude Code 和 OpenClaw 暴露了数十个工具，但实际调用高度集中于少数几个。GA 将这个”活跃工具环”显式化为 9 个原子工具，消除了长尾浪费。(来源：论文 Figure 3)

论文的结论非常明确：工具最小化不仅仅是一种节俭策略，而是一个**更优的运行点 (a better operating point)**——它同时降低了提示词开销和决策复杂度。

那么，GA 如何解决这个问题？答案出奇地简单：**只保留 9 个工具**。

9.2 GA 的 9 个原子工具

9.2.1 直觉理解：乐高积木而非瑞士军刀

GA 的工具设计哲学可以用一个类比来概括：**不要瑞士军刀，要乐高积木**。

瑞士军刀为每种功能设计了一个专用部件——开瓶器、剪刀、锯子、锉刀……虽然功能多，但每个部件都是固定形态，无法组合。而乐高积木只有几种基本形状的砖块，却可以通过组合搭建出无限可能。

GA 的 9 个工具就是 9 块”乐高积木”。论文指出，工具选择必须满足两个条件：

- **原子性 (Atomicity)**：每个工具被约束为**不可再分的原始能力 (irreducible primitive capability)**，执行单一职责，工具之间不存在功能重叠
- **组合泛化 (Compositional Generalization)**：复杂行为通过原子工具的**序列组合**来实现，而非通过引入新的专用接口

换句话说，GA 通过**组合而非枚举**来获取能力：少量原子工具作为可复用的原语 (reusable primitives)，复杂行为从它们的组合中涌现。

9.2.2 深入理解：五大能力类与九大原子工具

GA 将 Agent 与外部世界的所有交互归纳为**五大能力类 (Capability Classes)**，每个能力类下放置 1-3 个原子工具：

GA 原子工具集架构		
能力类	工具	
文件操作	file_read • file_patch • file_write	

代码执行	code_run	
网页交互	web_scan • web_execute_js	
记忆管理	update_working_checkpoint	
	start_long_term_update	
人机协作	ask_user	

论文指出，每个工具维持单一职责，这些工具共同构成了一个完整的循环，覆盖**状态观测**、**动作执行**、**上下文保持**和**干预请求**四个环节：

- **状态观测 (State Observation)**: file_read 读取文件内容，web_scan 扫描网页信息——让 Agent 能“看见”外部世界的状态
- **动作执行 (Action Execution)** : file_patch / file_write 修改文件，code_run 执行代码，web_execute_js 操作浏览器——让 Agent 能“改变”外部世界
- **上下文保持 (Context Preservation)** : update_working_checkpoint 维护短期工作记忆，start_long_term_update 触发长期记忆蒸馏——让 Agent 能“记住”重要信息
- **干预请求 (Intervention Request)**: ask_user 在需要人类决策时请求介入——让 Agent 知道自己的**能力边界**

在第一部分的应用篇中，我们已经实际使用过这些工具。本章的重点不是“怎么用”，而是“理解”为什么是这 9 个，为什么不多也不少”。

9.2.3 关键工具的设计细节

每个原子工具都经过了面向**执行效率**的专门优化。论文指出，这些优化的目标不仅是压缩单次工具输出，更是通过在每一步提供更精确、更完整的反馈，来**减少后续轮次中的冗余探测和重试**，从而在多轮交互中降低整体 token 消耗。我们来看几个典型例子：

file_read：精准读取，拒绝”全文搬运”

file_read 不是简单的”把整个文件内容返回”。它支持分段读取 (start/count)、关键词锚定 (keyword) 和行号输出，使 Agent 只获取”当前决策需要的那几行代码”，而不是把一个 5000 行的文件全部塞入上下文。

模式 1: 指定范围读取——只读文件的第 100-120 行

```
file_read(path="main.py", start=100, count=20)
```

模式 2: 关键词锚定——跳到包含 "def train" 的位置

```
file_read(path="main.py", keyword="def train")
```

file_patch：唯一匹配，拒绝”沉默错误”

file_patch 要求 old_content 参数在目标文件中**恰好匹配一处**。如果匹配零处（内容不存在）或多处（存在歧义），工具会直接报错而非静默执行。这种**快速失败 (Fail-Fast)** 设计避免了”改错位置”的沉默错误——在 Agent 自主运行的场景下，沉默错误比报错更危险，因为它可能在很多步之后才被发现。

web_scan：语义提取，拒绝”原始 DOM”

web_scan 是优化最激进的一个工具。它不是简单地返回网页的 HTML 源码，而是执行了一套布局分析算法 (layout-analysis algorithm)：

1. 克隆当前页面的 DOM 树
2. 计算每个元素的可见性
3. 通过叠加分析和区域划分，将区域分类为”主要内容”和”非核心区域”（导航栏、侧边栏、广告等）
4. 移除被遮挡或隐藏的元素
5. 序列化为结构化的精简文本

论文指出，这套处理将 token 消耗降低了一个数量级——一个原始 DOM 可能包含数万个 token，而经过 web_scan 处理后通常只剩几千个 token，且关键信息完整保留。

web_execute_js：操作结果 + 页面变化观测

web_execute_js 在返回操作结果的同时，还会附带**页面变化的观测信息**。这意味着很多工作流无需再发起一次完整的 web_scan 就能继续推进——减少了一轮完整的“扫描 → 分析”循环。

code_run：一轮一次，观察后再行动

code_run 被限制为**每轮只能调用一次**。这不是任意的约束，而是一种执行纪律：确保每次代码执行的结果都被模型观察和评估后，才能决定下一步行动。这种“先看结果再行动”的节奏，防止了模型在没有反馈的情况下连续发起多个可能相互冲突的操作。

一个关键视角：论文指出，理论上 Agent 可以**仅凭 code_run 一个工具完成所有任务**——通过编写自定义脚本，它能复制其他每个工具的功能。因此，其余 8 个工具的存在不是为了扩展 Agent 的能力边界，而是作为**专用快捷方式 (specialized shortcuts)** 来降低决策成本和操作开销。论文将这种设计称为**工具脚手架 (harness)**——它让 Agent 专注于高效解决问题，而不必为每个简单操作都从头编写代码。

这是“上下文信息密度最大化”原则在工具层的直接体现：同样是“看网页”，返回原始 HTML 是低信息密度的（大量标签、样式、脚本），返回结构化摘要则是高信息密度的。

9.3 能力边界与执行纪律

工具不仅是“能力”的体现，也是“权限”的边界。论文将这种设计称为**能力边界与执行纪律 (Capability Boundaries and Execution Discipline)**：Agent 的能力范围严格由注入的工具集定义，每个工具的能力范围严格有界且互不重叠。

9.3.1 直觉理解

这就好比一个公司的权限管理系统。每个员工的操作权限由管理员在系统中配置，员工不能自己给自己加权限。但管理员可以提前给某些员工配置“受控提权通道”——比如 IT 人员可以通过特定审批流程获得服务器的临时管理员权限。

在 GA 中，工具形成了一个从低风险到高风险的**权限层级**：- **只读工具** (file_read、web_scan) 只能观察，不能修改——Agent 可以看到文件内容和网页信息，但无法改变任何状态 - **编辑工具** (file_patch、file_write) 可以修改文本，但受严格的参数约束——file_patch 必须精确匹配，file_write 必须指定完整内容 - **执行工具** (code_run、web_execute_js) 可以触发真实的系统动作——这是风险最高的工具，但也是能力最强的

所有交互都通过统一分发器返回结构化输出，这提升了可控性、降低了操作风险，并确保**每一个动作的完全可追溯性**。

9.3.2 深入理解：声明、调用与分发

GA 的所有工具通过一套统一的 **Schema 契约 (Schema Contract)** 来定义。每个工具声明其名称、描述和参数（以 JSON Schema 格式描述），统一使用 type: function 格式：

```
{
  "type": "function",
  "function": {
    "name": "file_read",
    "description": "Read file content with optional range and keyword anchoring",
    "parameters": {
      "type": "object",
```

```

    "properties": {
      "path": {"type": "string"},
      "start": {"type": "integer"},
      "count": {"type": "integer"},
      "keyword": {"type": "string"}
    },
    "required": ["path"]
  }
}

```

在运行时，这些 Schema 被注入到模型的上下文中。模型根据 Schema 生成结构化的**工具调用请求 (tool_use call)**，然后由**统一分发器 (Unified Dispatcher)** 将请求路由到对应的本地执行器。分发器还负责执行前后的处理和结果返回管理，实现了**声明、调用和执行的清晰分离**。

这种分离意味着模型只需要知道”工具能做什么”和”参数格式是什么”，不需要知道工具的具体实现方式。9 个工具的 Schema 开销远小于 53 个工具——这是工具最小化在提示词层面最直接的收益。

9.4 工具组合的威力：实验印证

理论上讲，9 个原子工具覆盖了完整的能力环。但在实践中，它们真的能替代拥有几十个专用工具的系统吗？论文 Section 4.2 的实验数据给出了令人信服的回答。

9.4.1 能力覆盖：专用工具 → 原子组合

论文的 Table 11 展示了具体的替代案例——每个专用工具的功能都可以通过少量原子工具的组合来实现：

原系统	专用工具	代表性任务	GA 替代方案	替代原理
Claude Code	GlobTool	文件模式匹配	code_run	文件模式匹配可通过通用代码执行完成
Claude Code	GrepTool	文本搜索定位	code_run + file_read	文本搜索可通过通用执行与文件读取的组合实现
Claude Code	WebFetchTool	网页抓取与摘要	web_scan + web_execute_js + code_run	网页抓取、清洗和摘要可通过浏览器原语与代码后处理的组合重构
Claude Code	AgentTool	信息提取	file_read + code_run + update_working_checkpoint	显式的子 Agent 委派可通过记忆中存储的子 Agent 策略加通用执行来近似
Claude Code	WebSearchTool	在线搜索	web_execute_js + web_scan	在线搜索可通过浏览器交互加网页读取完成
Claude Code	NotebookEditTool	Jupyter 检查	file_read + code_run	读取.ipynb 的 JSON 结构并执行解析脚本
OpenClaw	nodes (结构化数据)	CSV 统计	code_run	CSV 统计可通过通用代码执行完成，无需专用结构化数据工具

论文的关键论点是：GA 不是试图逐一复制这些专用工具，而是证明了**许多基准能力可以通过少量原子工具的短序列组合来重构**。

9.4.2 效率对比：更少的工具，更低的成本

在 5 个长链复杂任务（包括 PDF/PPT 制作、SQL 副驾驶、实验分析报告撰写等）上的对比实验中：

指标	Claude Code (53 工具)	GA (9 工具)	OpenClaw (18 工厂)
任务成功率	100%	100%	80%
总 Token 消耗	537,413	188,829	633,101
平均请求数	32.6	11.0	15.0
平均工具调用数	22.6	12.8	16.6

GA 在保持 100% 任务成功率的同时，Token 消耗仅为 Claude Code 的 **35.1%**、OpenClaw 的 **29.8%**。请求数从 32.6 降至 11.0，工具调用数从 22.6 降至 12.8。论文指出，这些结果表明 GA 的优势来自**更低的上下文和工具流程开销**，而非牺牲成功率。

这些数据直观地印证了第 7 章的核心论点：上下文信息密度的提升直接转化为效率的提升。当上下文中每个 token 都更”有用”时，模型需要更少的轮次就能做出正确决策。

9.5 故障升级：工具执行失败时的应对

在实际运行中，工具调用并不总是成功的——代码可能报错、文件可能不存在、网页可能加载失败。为了防止 Agent 陷入错误动作的无限循环，GA 引入了一个**分阶段故障升级机制（Staged Escalation Mechanism）**。

当任务执行失败时，GA 通过一个清晰的**三步渐进恢复流程**来处理：

- 1. **局部修正**：分析当前错误信息，做小幅定向调整后重试
- 2. **策略切换**：如果失败持续，放弃当前方案，强制切换到全新策略或搜索环境中缺失的信息
- 3. **人类介入**：如果所有自动化尝试均失败，暂停执行并请求人类干预

这个结构化设计防止了 Agent 盲目重复同样的错误。通过触发**渐进增强的纠正动作**，它确保 Agent 能够突破死胡同，使长期学习保持在正确的轨道上。

故障升级与工具最小化的关系在于：9 个原子工具的清晰边界（如 file_patch 的快速失败、code_run 的一轮一次）使得失败原因更容易诊断。当动作空间小且工具语义明确时，Agent 更容易定位问题并选择正确的恢复策略。

9.6 本章小结

本章深入分析了 GA 最小原子工具集的设计原理。核心要点：

- 1. **工具膨胀在两个层面产生代价**：提示词层面挤占上下文预算，策略层面增加决策复杂度。实验数据表明，大量工具中真正被使用的只是少数几个
- 2. **工具选择遵循两个条件**：原子性（每个工具是不可再分的原始能力）和组合泛化（复杂行为通过原子工具的序列组合涌现）
- 3. **9 个原子工具覆盖五大能力类**，形成完整的状态观测-动作执行-上下文保持-干预请求循环
- 4. **code_run 是万能原语**，理论上可以完成所有任务；其余 8 个工具是降低决策成本的专用快捷方式（harness）
- 5. **实验验证**：GA 在保持 100% 任务成功率的同时，Token 消耗仅为 Claude Code 的 35.1%

论文还指出了工具最小化的一个深层价值：由于保留的操作知识自然表达为**少量通用原语的可复用组合序列**，而非新的任务专用接口，工具最小化为**后续的经验沉淀（experience consolidation）创造了有利条件**。换句话说，工具越少、越通用，过去的经验就越容易被压缩为可复用的 SOP——这正是第 12 章”自我进化”机制得以高效运作的前提。

在下一章中，我们将深入 GA 的另一个核心机制——分层记忆架构。如果说工具集解决的是“Agent 能做什么”的问题，那么记忆系统解决的是一个更根本的问题：**Agent 如何”记住”跨任务的经验，又不让过往记忆淹没当前决策？**

第 10 章 分层记忆架构——让 Agent 拥有持久且有序的知识

在上一章中，我们看到了 GA 如何通过 9 个原子工具同时降低提示词层面的上下文开销和策略层面的决策复杂度。但工具层控制的是**每轮固定成本**，更大的挑战在于**跨任务的知识积累——记忆**。

一个只有短期记忆的 Agent，每次启动都从零开始，重复犯同样的错误，重复探索同样的路径。而一个拥有良好记忆系统的 Agent，能够把过去的经验转化为未来的效率。那么，GA 如何在”记住更多”和”上下文不爆炸”之间取得平衡？

10.1 为什么 Full-History Append 行不通

10.1.1 直觉理解

想象你有一个笔记本，从工作第一天起，你就把每天的所有工作内容——会议记录、邮件、临时想法、代办事项——全部按时间顺序写进去，从不整理、从不删除。

10 年后，你需要查找一条关于某个客户的关键信息。你打开这本笔记，面对的是几万页密密麻麻的文字。即使信息确实存在其中，要找到它的成本也是巨大的——你可能需要翻阅数百页才能定位到那一行关键记录。

这就是**全量历史追加（Full-History Append）**策略的困境：记住了一切，却等于什么都没记住。

10.1.2 深入理解：上下文预算的系统性侵蚀

论文指出，在传统 Agent 框架中，prior interactions、intermediate states 和 execution traces 会随着任务推进持续累积，**逐步消耗上下文预算，将决策关键信息埋藏在低价值内容之下**，从而降低每一步的推理质量。

这个问题的严重性可以从实验数据中直观看到。在安装了 20 个技能并经过大量使用之后，不同 Agent 系统对一条最简请求（“Hello”）产生的**完整提示词长度（Full Prompt Length）**差异巨大：

系统	完整提示词长度（tokens）
OpenClaw	43,321
CodeX	23,932
Claude Code	22,821
GA	2,298

GA 的提示词长度仅为 Claude Code 的 **10%**，OpenClaw 的 **5%**。这意味着当其他系统已经被历史记忆”淹没”时，GA 的上下文依然保持着极高的信息密度。论文的结论是：**分层检索严格隔离了空闲记忆与活跃提示词，消除了上下文爆炸的风险**。

这个 10 倍以上的差距不是因为 GA “记住了更少的东西”，而是因为它**换了一种方式来记忆**。接下来我们就来看看这种方式是什么。

10.2 GA 的四层记忆架构

GA 将记忆组织为四个层次。论文明确指出：在这个层级中，**L1 构成始终在线记忆，L2 和 L3 共同构成长期记忆，L4 提供持久存储用于追溯和审计**。每层有明确的定位和准入标准。我们用一个图书馆的类比来帮助理解：

10.2.1 直觉理解：图书馆的四个区域

想象 GA 的记忆系统是一座图书馆：

- **L1 索引层**就像图书馆的**目录检索台**——你不需要走进书架，只需查目录就能知道你要的书在哪。它很小（通常只有几百个 token），但能快速路由到任何需要的知识
- **L2 事实层**就像**百科全书区**——存放经过验证的、长期稳定的事实。比如”服务器 IP 是 192.168.1.100”或”项目使用 Python 3.11”。准入门槛很高，只有经过执行验证且跨任务可复用的信息才能写入
- **L3 SOP 层**就像**操作手册区**——存放可复用的流程知识，包括任务 workflows、前置条件、关键执行步骤、常见故障案例及对应的调试恢复策略。比如”如何从 HuggingFace 下载数据集”就是一条典型的 L3 SOP
- **L4 归档层**就像**档案室**——存放历史执行记录的压缩存档。它不用于日常注入，平时大门紧闭。只有当需要追溯过往决策、审计历史操作时才会打开

10.2.2 深入理解：四层之间的路由机制

这四层记忆之间并非孤立存在，而是通过一条 **L1→L2/L3 路由链**协同工作：

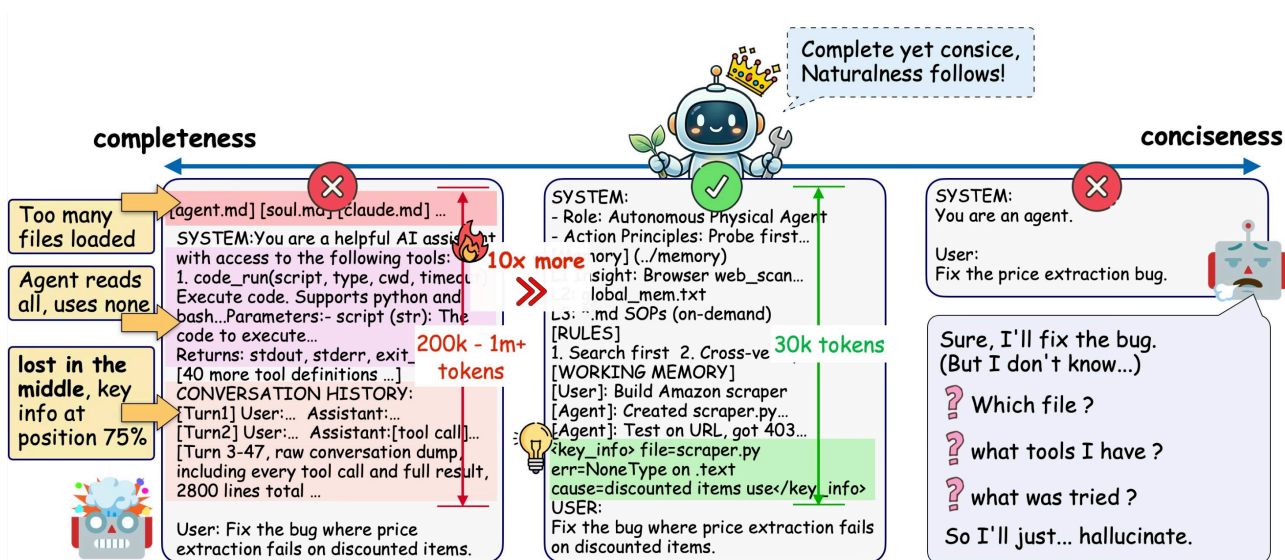


Figure 5: GA 记忆架构

图 10-1：GA 分层记忆架构。始终加载区域（Always Load）包含元记忆规则和工作记忆；按需加载区域（Load on Demand）通过 L1 索引路由到 L2 事实层和 L3 SOP 层；L4 归档层存储压缩后的历史会话，用于追溯和审计。自我进化机制将执行经验蒸馏为可复用的 SOP、代码和技能，写回长期记忆。（来源：论文 Figure 2）

路由过程是这样的：

1. 任务启动时，GA 默认只注入**元记忆**和 **L1 索引层**到上下文中。元记忆提供记忆系统的组织规则和更新边界，L1 提供紧凑的导航指针（高频入口、关键词映射等）
2. 当 LLM 在推理过程中发现需要某类知识时，它沿着 **L1→L2/L3 路由链**，通过 file_read 按需加载更深层的事实或 SOP 内容
3. 加载的内容进入当前轮的上下文，随着对话推进，会被第 11 章的四阶段压缩流水线处理

注意这里的关键设计：**路由通过工具调用和提示策略来强制执行（enforced），而非硬编码的强制流水线**。模型根据任务需要自主决定是否读取更深层的记忆，无关内容不会进入活跃上下文。

10.2.3 每层的准入标准与内容定义

四层记忆的质量靠**严格的准入标准**来保证：

L2 事实层：- **内容定义**：存储经过验证的、长期稳定的事实信息 - **准入标准**：信息必须经过执行验证（不是“听说”，而是“确认”），且具有跨任务可复用性 - **排除清单**：瞬态状态（transient states）、一次性事件（one-time events）、未验证假设（unverified hypotheses）一律排除

L3 SOP 层：- **内容定义**：存储可复用的流程知识，包括任务 workflow、前置条件、关键执行步骤、常见故障案例及调试恢复策略 - **准入标准**：流程必须来自经过成功工具执行验证的操作（论文称之为“**No Execution, No Memory**”规则——不是“应该这样做”，而是“这样做了，而且成功了”），且具有跨任务可复用性

这种严格的准入机制就是 GA 防止**记忆污染（Memory Pollution）**的核心手段。试想，如果任何信息都可以随意写入长期记忆，那么错误的经验、过时的配置、一次性的调试信息都会混入其中，最终让记忆变得不可信。

10.2.4 实验印证：精炼记忆优于全量记忆

记忆消融实验（Memory Ablation）直接验证了“精炼记忆优于全量记忆”的设计假设。实验在 SOP-Bench 的危险品分类子集（dangerous goods）上进行，使用**任务成功率（Task Success Rate, TSR）**衡量行为表现，使用**记忆大小**衡量上下文成本。

实验对比了四种记忆配置：

- **No-Memory**：不使用任何外部记忆，仅依赖任务输入和工具描述
- **Full-Memory**：注入完整的、未经编辑的原始 SOP 作为外部知识
- **Redundant-Memory**：在 Condensed Memory 的基础上扩展了背景描述、定义和弱相关信息
- **Condensed Memory**：仅保留高密度的、行动导向的规则

记忆配置	记忆大小（tokens）	任务成功率（%）
No-Memory	0	13.87
Full-Memory	575	52.44
Redundant-Memory	288	66.48
Condensed Memory	165	66.48

结果表明：No-Memory 的表现远差于所有使用记忆的配置，说明外部程序性记忆对该任务是必要的。Condensed Memory 用 **165 个 token**（不到全量记忆的三分之一）达到了与 Redundant-Memory 相同的最高成功率，且远超 Full-Memory。Redundant-Memory 尽管使用了更多 token，却没有比 Condensed Memory 带来任何额外收益——背景描述、定义和解释性文本增加了上下文负担，却没有贡献额外的行为价值。

论文的结论是：GA 从中受益最大的是**那些只存储模型尚未掌握的、关键的、能直接改变行为的少量信息**的记忆。

10.3 元记忆：记忆的管理规则

在四层记忆之上，GA 还有一个**元记忆层（Meta-Memory Layer）**——它不存储具体知识，而是定义记忆系统本身的运行规则。

10.3.1 直觉理解

如果四层记忆是图书馆的书架，那么元记忆就是图书馆的**管理章程**——它规定了：- 图书馆有哪些区域（记忆地图）- 每个区域收什么类型的书（准入规则）- 新书怎么入库、旧书怎么更新（操作规范）- 什

么书不能收（排除清单）

10.3.2 深入理解：元记忆的功能

论文指出，元记忆定义了**整体记忆地图 (memory map)**、**核心规则 (core rules)** 和 **更新边界 (update boundaries)**，在执行开始之前就为模型提供了一个共享的参考框架，使其了解记忆的组织方式、各层用途和更新规范。这一设计的直接效果是减少了三类问题：

- **随意写入 (arbitrary writes)**：模型不会在缺乏规范的情况下向任意层级写入未经验证的信息
- **历史误读 (historical misreads)**：模型在读取记忆时有明确的层级语义，不会混淆事实（L2）和流程（L3）
- **跨任务泄漏 (cross-task leakage)**：不同任务的上下文信息不会错误地污染长期记忆

一个容易被忽略的细节是：**元记忆的完整内容 (meta-SOP)** 也是通过 `file_read` 按需加载的，而非**全部预加载**。始终注入的只是元记忆的核心规则摘要，这确保了始终在线层的轻量化。

10.4 工作记忆：短期上下文的锚点

除了长期记忆（L1-L4）和元记忆，GA 还维护一个**工作记忆 (Working Memory)**，负责在单次任务执行过程中保持上下文的连贯性。

10.4.1 直觉理解

工作记忆就是你桌上的**草稿纸**。当你在做一项复杂任务时，你会在草稿纸上写下当前进度、关键发现、待办事项。这张草稿纸不会被归档到图书馆——任务结束后它就可以丢掉了，除非上面有值得永久保存的内容。

10.4.2 深入理解：工作记忆的机制

论文 § 2.3.2 指出，工作记忆**在每一轮中持续注入**，只携带最小必要的任务状态：当前目标、约束和进度，使 Agent 无需每次重建完整上下文即可维持执行连续性。

具体来说，工作记忆锚点 (working-memory anchor) 包含三个组成部分：

1. **20 条最近的单行轮次摘要**——每条约 100 字符，提供近期操作的紧凑回顾
2. **当前轮次编号**——让模型了解任务执行到了哪一步
3. **持久 key_info 块**——由 Agent 通过 `update_working_checkpoint` 工具主动维护，记录当前目标、关键发现和下一步计划

这个锚点在**每次工具调用后自动附加到下一条用户消息中**。当早期消息被第 11 章的压缩机制驱逐后，这个锚点就成为长期记忆的**唯一来源**，确保任务关键信息始终留在活跃上下文中。

工作记忆与长期记忆的关键区别：

维度	工作记忆	长期记忆 (L2-L4)
生命周期	单次任务	跨任务持久
注入方式	每轮自动注入	按需加载
晋升关系	不自动晋升 为长期记忆	经过验证后才能写入
内容特征	当前任务的进度和发现	可复用的事实和流程

这里的”不自动晋升”是一个重要的设计决策。论文明确指出，GA 在最低记忆层级（L4）保存原始执行轨迹，但**不会自动将这些轨迹提升到更高的长期记忆层级（L2/L3）**。可复用的 L3 SOP 只能通过显式的蒸馏步骤生成——这个过程将在第 12 章详细讨论。

10.4.3 三种记忆功能的分离

概括来看，论文将 GA 的记忆系统划分为三种功能不同的记忆类型：

GA 记忆系统全景			
工作记忆 (草稿纸)	始终在线记忆 (随身卡片)	长期记忆 (图书馆)	
轮次摘要 轮次编号 key_info 块	元记忆核心 L1 索引	L2 事实 + L3 SOP + L4 归档	
每轮注入	启动时注入	按需加载	

论文对这三种类型的定位非常明确：

- **工作记忆**在每轮持续注入，只携带最小必要的任务状态
- **始终在线记忆**始终可见，但被刻意压缩为最轻量的导航信息——只提供记忆布局和索引，而非完整知识
- **长期记忆**保持在默认上下文之外，通过显式的任务后蒸馏过程管理；只有经过验证的稳定知识才会被写回

论文指出，这种划分直接体现了上下文信息密度最大化原则：**每种类型的记忆只在必要的时刻、以必要的粒度出现在活跃上下文中。**

10.5 按需读取 vs 默认注入

10.5.1 核心策略：知道在哪，但不提前加载

论文的核心设计思路可以用一句话概括：**避免将所有保留的信息默认视为提示词常驻内容——只有一个小型的始终在线层保持可见，更深层的记忆仅在需要时访问。**

这与大多数 Agent 系统的做法截然不同。典型的做法是在每次任务启动时，把所有”可能相关”的记忆内容全部注入上下文——这就是为什么 Claude Code 和 OpenClaw 的提示词长度高达 2 万以上。而 GA 只注入一个紧凑的索引，等到模型真正需要某条知识时，才通过工具调用去读取。

10.5.2 L1 有界性：为什么索引不会失控膨胀

一个自然的疑问是：随着 L2 和 L3 不断积累新知识，L1 索引是否也会无限增长？

论文指出，L1 的有界性是整个架构的一个**关键设计不变量 (critical design invariant)**：即使 L2 和 L3 持续扩展，L1 仍然保持有界。

原因在于：L1 中每条记录只编码某个知识类别的**存在性 (existence)**，而非其实质内容。只有当真正出现新的知识类别时，才会引入新的 L1 条目。因此，L1 的总体描述长度趋近于知识集分类结构的**Kolmogorov 复杂度**——这是一个理论上的下界。

这种极致的压缩之所以可行，是因为 **LLM 自身充当了解码器**：一旦模型从 L1 中推断出某个相关的能力或事实存在，它就可以花费工具调用和 token 来从更深层检索完整内容。因此，一个最小化的存在性信号就足以实现准确的路由。

这就是 GA 提示词长度能始终保持在 2,298 token 的根本原因——不是通过”少记”，而是通过将 L1 约束为一个**与知识总量无关的有界索引**。

10.5.3 运行时的按需加载策略

按需加载（On-Demand Loading）的具体流程：

1. 任务启动时，注入元记忆核心 + L1 索引（即”始终在线记忆”）
2. LLM 在推理过程中发现需要某个 SOP → 调用 file_read 读取对应的 L3 文件
3. SOP 内容进入当前轮上下文 → 模型根据 SOP 执行任务
4. 随着对话推进，早期加载的内容会被第 11 章的四阶段压缩流水线（工具输出截断 → 标签级压缩 → 消息驱逐 → 工作记忆锚点）处理

这种策略确保了上下文预算始终分配给**当前任务真正需要的信息**，而不是”以防万一”预加载的大量冗余内容。

10.5.4 长期整合的触发提交机制

当一次任务执行产生了值得长期保存的新知识时，GA 不会立即写入记忆，而是采用**触发提交（Triggered Commit）**机制：

1. **识别阶段**：在任务执行过程中，模型识别出潜在有价值的新信息
2. **验证阶段**：检验信息的有用性和可复用性——它是否经过了执行验证？它是否只适用于当前任务？（回顾 § 10.2.3 的 “No Execution, No Memory” 规则）
3. **提交阶段**：只有通过验证的信息才被写入 L2 或 L3，采用**小幅增量更新**而非全量覆盖
4. **索引更新**：相应地更新 L1 索引，确保新知识可被未来任务发现

这种”先验证再提交”的机制，防止了记忆以不受控的方式随时间增长。

10.5.5 实验印证：长期事实记忆

分层记忆的按需加载策略不仅控制了上下文膨胀（如 § 10.1 所示的提示词长度对比），还在长期事实记忆任务上表现出色。

实验在 LoCoMo 基准测试上进行，对比了两类方法：**- 嵌入式方法（embedding-based）**：Mem0 和 A-MEM, 使用 text-embedding-3-small 作为嵌入模型 **- 非嵌入式方法（non-embedding）**：OpenClaw 和 GA，不依赖任何向量数据库或嵌入模型

系统	多跳推理 F1	多跳推理 BLEU-1	时序推理 F1	时序推理 BLEU-1	开放域 F1	开放域 BLEU-1	单跳查询 F1	单跳查询 BLEU-1
Mem0 (嵌入式)	39.32	28.43	50.03	43.33	18.32	13.80	40.32	32.33
A-MEM (嵌入式)	29.03	20.48	46.83	38.84	13.11	12.94	44.68	37.01
OpenClaw	21.43	18.41	22.56	20.43	9.56	9.03	23.44	24.21
GA	43.33	39.96	52.23	51.11	20.41	15.31	45.69	40.66

GA 在所有四个类别上都取得了最高的 F1 和 BLEU-1 分数。论文指出，GA 的优势在多跳推理和时序推理任务上尤为明显——这说明 GA 不仅能保持长期事实记忆，还能在跨时间和跨事实链的推理中有效利用这些记忆。即使在所有方法都表现最差的开放域类别中，GA 仍然取得了最高分。

论文的核心结论是：**长期事实记忆可以通过精确的记忆组织和过滤来实现，GA 不需要额外的嵌入模型或向量数据库来构建这种能力。**

10.6 本章小结

本章深入剖析了 GA 的分层记忆架构。核心要点：

1. **Full-History Append 行不通**：传统框架中，历史交互不断累积，逐步消耗上下文预算，将决策关键信息埋在低价值内容之下
2. **四层长期记忆**（L1 索引 → L2 事实 → L3 SOP → L4 归档）通过严格的准入标准和 “No Execution, No Memory” 规则，在 “记住更多” 和 “保持精炼” 之间取得了平衡
3. **L1 有界性**是整个架构的关键不变量——L1 只编码知识类别的存在性，其描述长度趋近于知识集分类结构的 Kolmogorov 复杂度，确保索引不会随知识积累而失控膨胀
4. **元记忆**定义了记忆地图、核心规则和更新边界，防止随意写入、历史误读和跨任务泄漏
5. **工作记忆**通过轮次摘要、轮次编号和 key_info 块，在每轮注入最小必要的任务状态
6. **按需加载 + 触发提交**确保了上下文预算始终用在刀刃上
7. **精确的记忆组织和过滤使 GA 不需要额外的嵌入模型或向量数据库**，就能在长期事实记忆任务上超越专门的向量检索系统

但记忆分层解决的是 “什么信息该进入上下文” 的问题。当任务执行到中后期，即使有了分层记忆，单次对话内累积的历史消息依然会越来越长。GA 如何在不丢失关键信息的前提下，主动控制每一轮的上下文大小？这就是下一章的主题——**上下文截断与压缩**。

第 11 章 上下文截断与压缩——主动管理每一轮的信息预算

在上一章中，我们看到了 GA 如何通过分层记忆管理跨任务的知识——只注入索引，按需加载详情。但即使有了记忆分层，**单次任务内**的对话依然会越来越长：每一轮工具调用都会返回新的数据，推理过程会产生新的思考记录，历史轮次不断累积。

许多 Agent 框架选择依赖扩展的上下文窗口（高达 1M token），假设更多上下文意味着更好的推理。但实践表明，这种做法不仅成本高昂，还会引入更多幻觉。论文将当前模型在不产生幻觉的前提下能有效利用的上下文上限称为**无幻觉上下文长度 (hallucination-free context length)**，其量级大约比名义窗口小一个数量级。因此，GA 选择了一条截然不同的路径：**将预算控制在 30k token 以内，投资于压缩而非扩展——把高密度信息装进更小的窗口，优于把稀释的内容灌入更大的窗口。**

这一战略选择决定了本章的核心主题：GA 如何在”不丢关键信息”和”不超预算”之间取得平衡？

11.1 上下文预算的量化模型

11.1.1 直觉理解：信用卡额度

把上下文窗口想象成一张信用卡的额度。每一轮对话都是一笔”消费”：用户消息、模型推理、工具调用、工具返回——每一项都在消耗额度。如果不主动管理，几十轮下来就会”刷爆额度”，模型将无法接收新的信息。

GA 的做法不是”等到刷爆再处理”，而是**主动记账、提前瘦身**——在每一笔大额消费发生时就控制支出，在余额不足时主动清理旧账单。

11.1.2 深入理解：字符域预算公式

由于 GA 无法直接获得精确的 token 计数，它使用一个**字符域启发式公式 (Character-Domain Heuristic)** 来管理上下文预算：

$$C_H = \sum_{m \in H} \text{len}(m)$$

$$B = \alpha \cdot W_{\text{tokens}}, \quad \alpha \approx 3$$

其中：- C_H 是当前对话历史 H 中所有消息 m 的字符总长度（JSON 序列化后）- W_{tokens} 是配置的 token 窗口大小 - $\alpha \approx 3$ 是字符/token 的经验转换系数 - B 是字符域预算 - 当 $C_H > B$ 时，触发压缩或驱逐机制

所有预算比较都在字符域完成，不会转换回 token 数。这使得预算管理完全独立于具体的 tokenizer 实现，支撑了 GA 的模型无关设计。

关于 $\alpha \approx 3$ 的设计权衡：对于英文为主的内容，实际的字符/token 比率约为 4:1，这意味着 $\alpha = 3$ 会略微高估 token 消耗，导致**提前驱逐**——这是一种安全的失效模式（宁可少放一些，也不要超出窗口）。但对于中日韩（CJK）内容，每个字符通常消耗 1-2 个 token， $\alpha = 3$ 反而会**低估** token 消耗 3-6 倍，存在延迟驱逐的风险。这是当前实现的一个已知局限。

11.2 四阶段压缩流水线

GA 的上下文管理不是单一策略，而是一条**四阶段流水线 (Four-Stage Pipeline)**，从细粒度到粗粒度依次处理。我们用一个比喻来理解：

想象你在整理一个越来越满的行李箱：- **Stage 1**：先把每件衣服卷紧（减少单件体积）- **Stage 2**：把旧的、不常穿的衣服折叠得更小（压缩旧物品）- **Stage 3**：实在装不下了，把最旧的衣服拿出来（移除旧物品）- **Stage 4**：在行李箱外面贴一张清单，记录被拿出去的东西是什么（保留索引）

11.2.1 Stage 1：工具级截断——控制每件“物品”的大小

第一道防线作用在**每个工具的返回值**上。在工具执行结果进入对话历史之前，GA 会根据预设的阈值进行截断。

截断策略采用**对称头尾保留 (Symmetric Head-Tail Policy)**：当输出超过阈值 L 时，保留前 $L/2$ 和后 $L/2$ 字符，中间部分替换为省略号。

为什么保留头和尾而不是只保留头部？因为在很多场景下，**尾部信息同样关键**：- 代码执行的错误信息通常出现在输出末尾 - 文件内容的结尾可能包含关键的总结或签名 - 日志的最新条目在底部

各工具的截断阈值如下：

工具 / 模式	截断阈值 L (字符)	说明
code_run	10,000	标准代码输出截断
web_execute_js	8,000	使用 save_to_file 时 全量写磁盘，仅短 预览入上下文
web_scan (纯文本)	10,000	头尾截断
web_scan (HTML)	35,000	DOM 级子树裁剪， 非头尾截断
file_read	~1,280/行；总计 20,000	行级 + 总量双重控制

注意 web_scan 的 HTML 模式使用的是 **DOM 级字符预算**而不是简单的头尾截断。这是因为 HTML 是树形结构，简单的字符截断会破坏标签的完整性。GA 在 DOM 层面进行子树裁剪，确保返回的 HTML 始终是结构完整的。我们将在 11.3 节详细讨论浏览器层的选择性提取。

11.2.2 Stage 2：Tag 级压缩——压缩旧轮次的“包装”

即使每个工具返回值都被截断了，随着对话推进，历史消息中仍然会累积大量**结构化标签内容**：推理痕迹 (<thinking>)、工具调用记录 (<tool_use>)、工具返回结果 (<tool_result>)、以及过期的工作记忆快照 (<history>、<key_info>)。

Tag 级压缩大约每 **5 轮** 执行一次（以摊销计算成本）。这个间隔还带来了**一个额外收益**：未被修改的旧消息在相邻轮次之间保持不变，使得约 **80% 的轮次**能够命中 prompt-cache，显著降低了推理成本。处理方式分两类：

第一类：替换过期的工作记忆块。 <history> 和 <key_info> 标签的内容只有最新的一份是有意义的——旧的快照已经被新快照覆盖。因此，旧轮次中的这些标签会被替换为简短的占位符 [...]。

第二类：截短推理和工具标签。 旧轮次中的 <thinking>、<tool_use>、<tool_result> 标签内容会被截断到约 **800 字符**的头尾窗口。这保留了推理的开头（通常包含策略判断）和结尾（通常包含结论），丢弃中间的详细推导过程。

关键保护机制：最近的消息（默认 **10 条**）不受 Stage 2 压缩的影响。这确保了模型对最近几轮交互拥有完整的上下文，不会因为压缩而丢失刚刚发生的关键信息。

11.2.3 Stage 3：消息驱逐——当“瘦身”不够时“断舍离”

当 C_H 在追加新消息后超过字符预算 B 时，Stage 3 启动。它的执行分两步：

第一步：更激进地重跑 Stage 2。这次只豁免最近 4 条消息（而非正常的 10 条），对更多历史消息进行 Tag 级压缩。

第二步：FIFO 驱逐。如果压缩后仍然超预算，按先进先出顺序从最早的消息开始移除，直到 C_H 降到 $0.6B$ 以下，或者只剩 5 条消息。

为什么目标是 $0.6B$ 而不是刚好 B ？这个 60% 目标是刻意的”过度修剪”，目的是为后续轮次预留空间。如果每次都只修剪到刚好不超预算，那么下一轮新消息进来后立即又要触发驱逐，导致频繁的压缩开销。60% 的目标提供了约 40% 的缓冲区。

驱逐后的结构修复：消息被移除后，剩余的历史可能出现结构问题——比如开头是一条孤立的 tool_result 消息，却没有对应的 tool_use。GA 会进行轻量级的结构修复：- 重新对齐历史，确保以用户消息开头 - 将孤立的 tool_result 转换为纯文本，避免 API 协议错误

一个关键的设计选择：Stage 3 不做摘要。被驱逐的消息就是被丢弃了，GA 不会尝试用 LLM 生成一段”之前发生了什么”的摘要。这是因为：- 摘要需要额外的 LLM 调用，增加延迟和成本 - 摘要本身也占用上下文空间 - 摘要的质量难以保证，可能引入错误信息

但论文强调，**被驱逐的内容并不会完全丧失：**Stage 4 的工作记忆锚点保留了关键的任务状态，而 Stage 2 的压缩机制确保了即使在不到 30k token 的上下文窗口内，也能容纳数十轮历史交互的压缩信息。GA 正是用这两层机制来弥补驱逐造成的信息损失。

11.2.4 Stage 4：工作记忆锚点——驱逐后的唯一长程记忆

Stage 4 是整个压缩流水线的”安全网”。在每次工具调用之后，GA 会在下一条用户消息中注入一个**锚点提示 (Anchor Prompt)**，包含三部分内容：

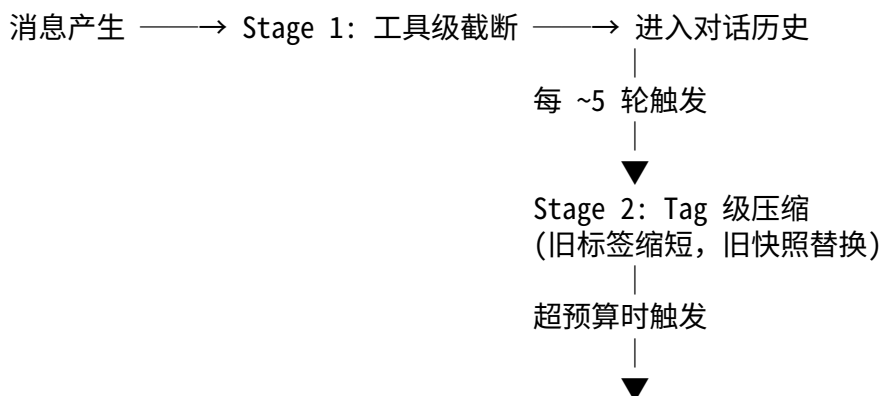
1. **最近 20 条单行轮次摘要：**每条约 100 字符，记录了”第 N 轮做了什么、结果如何”。摘要的来源有两种：优先使用模型在推理过程中提供的 <summary> 字段；如果模型未提供，则从该轮的实际工具操作中自动生成一条简要描述
2. **当前轮次编号：**让模型知道任务已经进行了多少轮
3. **持久化 key_info 块：**通过 update_working_checkpoint 工具维护的结构化笔记，包含当前目标、关键发现、约束条件等。这里有一个重要的机制区分——key_info 是由 **Agent 显式更新**的（模型主动调用工具写入），但写入后由**系统自动携带**到后续每一轮的上下文中。它不是后台自动改写的机制，而是”写一次，自动传播”的设计

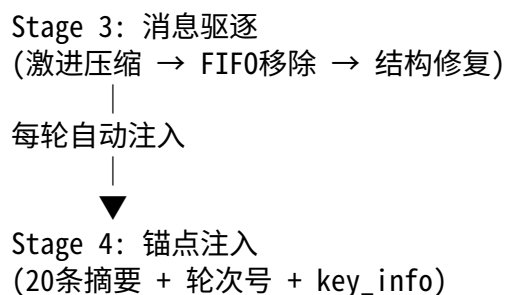
这个锚点是**消息驱逐后唯一的长程记忆保留机制**。当 Stage 3 把早期的对话全部清除之后，模型依靠这个锚点来”知道之前发生了什么”。

注意这里的巧妙之处：因为锚点被注入到每条用户消息中，所以 Stage 2 会自动将**旧的锚点副本**压缩为占位符 [...], 只保留**最新的一份完整内容**。这避免了锚点自身造成的上下文膨胀。

11.2.5 四阶段协作的全景

让我们把四个阶段放在一起，看看它们如何协作：





Stage 1 控制增量（每条新消息的大小），**Stage 2** 控制存量（旧消息的膨胀），**Stage 3** 控制总量（整体预算），**Stage 4** 补偿损失（驱逐后的记忆恢复）。四个阶段形成了一个完整的信息生命周期管理系统。

11.3 浏览器层的选择性提取

在四阶段流水线之外，GA 还在**信息进入系统之前**就进行了一次重要的过滤——这就是浏览器工具 web_scan 的**语义结构化处理**。

11.3.1 为什么不能直接把网页塞进上下文

一个典型网页的完整 HTML 可能有几十万甚至上百万字符，其中大部分是对 Agent 决策毫无价值的内容：CSS 样式、JavaScript 代码、隐藏元素、广告栏、页脚导航等。如果把原始 HTML 直接送入上下文，信息密度将极其低下。

11.3.2 DOM 语义压缩的效果

正如第 9 章 § 9.2.3 所介绍的，web_scan 在返回页面内容之前会执行一套 DOM 级的选择性提取——克隆 DOM 树、计算元素可见性、区分主/非主要内容区域、移除遮挡和隐藏元素，最终在 35,000 字符的 DOM 级预算内优先保留主要内容。

从上下文压缩的角度来看，论文指出这种处理将 token 消耗降低了一个**数量级**——原始 DOM 可能包含数十万字符，经过 web_scan 的语义提取后通常压缩到数万字符以内，且关键信息完整保留。

这里体现了 GA 信息密度最大化原则的又一个实例：**不是在信息进入上下文后再压缩，而是在源头就只提取有价值的信息**。Stage 1 的工具级截断是“入口把关”，而 web_scan 的 DOM 处理则是“源头净化”——两者共同构成了信息进入上下文之前的两道过滤。

11.4 工具 Schema 省略优化

除了对话内容的压缩，GA 还对一个容易被忽视的上下文开销进行了优化——**工具定义 (Tool Schema)**。

9 个工具的完整 Schema 定义（包括参数说明、类型描述等）大约占用 2000-3000 字符。在文本协议路径下，如果每轮都发送完整的 Schema，这部分开销就是纯粹的浪费——因为工具定义在相邻轮次之间通常不会变化。

GA 的优化策略是：

- **相邻轮次 Schema 不变时省略**：用一句简短的自然语言提醒替代完整定义（如“工具定义与上轮相同”）
- **定期重发完整 Schema**：每隔若干轮，或当累积提示词长度超过安全阈值时，重新发送完整定义，防止模型因长时间未见完整 Schema 而产生工具调用格式漂移

这个优化在原生 API 路径下不适用——因为 API 协议要求每次调用都携带完整的工具定义。它主要服务于文本协议场景，每轮可节省约 2000-3000 字符的固定开销。

11.5 实验印证：压缩与记忆协同下的效率收敛

本章介绍的压缩机制并非孤立运作——它与记忆层和自我进化层协同工作，共同产生了一个显著的效率收敛效果。论文在 § 4.3.1 (Memory System Effectiveness 下的 Continuous Efficiency Improvement) 中报告了这一实验。

实验设置：以 GPT-5.4 为骨干模型，比较 CodeX、Claude Code、OpenClaw 和 GA 在 HuggingFace 数据集下载任务上的表现。实验包含五个不同的数据集，每轮实验在全新的独立会话中执行，以严格避免残留上下文的干扰。

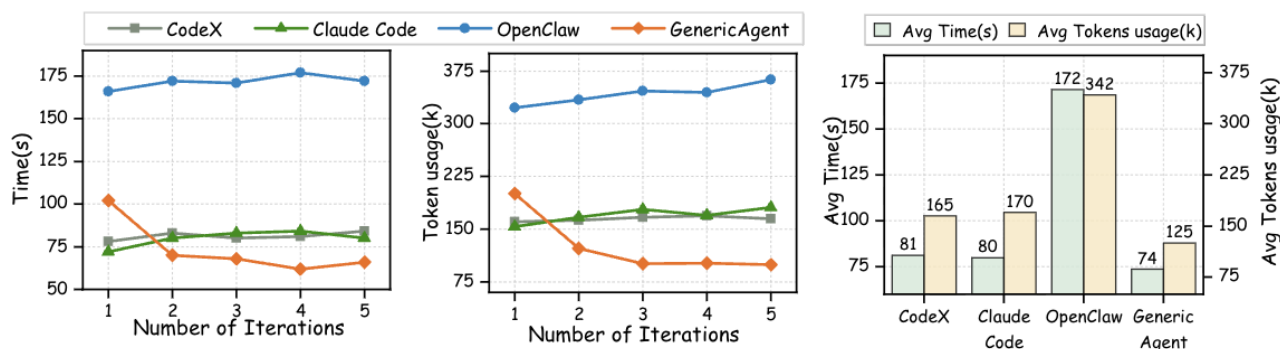


Figure 6: 效率收敛趋势

图 11-1：不同 Agent 系统在五轮重复实验中的操作时间和 token 消耗趋势。CodeX、Claude Code 和 OpenClaw 在各轮之间基本保持稳定，而 GA 展现出明显的效率收敛模式——运行时间和 token 消耗均随轮次稳步下降，表明任务经验被成功转化为可复用的记忆。（来源：论文 Figure 4）

实验结果：

- CodeX、Claude Code、OpenClaw 的操作时间和 token 消耗在各轮之间基本保持稳定，没有明显的改善趋势
- GA 的操作时间从第一轮的 **102 秒** 下降到后续轮次的约 **66 秒**，token 消耗从 **200,439** 降至约 **100,000**

论文将这种稳步下降归因于 GA 的记忆系统：GA 不是简单地累积历史，而是**将任务经验转化为可复用的 L3 SOP**，从而在后续运行中减少了重复理解、不必要的推理和决策开销。效率的提升不是因为“存了更多内容”，而是因为**只保留了模型尚未掌握的、直接影响行为的高价值信息**。

需要注意的是，这个收敛效果是**压缩层、记忆层和进化层三者协同**的结果：压缩机制确保每轮上下文不膨胀，为高密度信息留出空间；记忆机制确保经验被结构化保存在 L2/L3 中；进化机制（将在第 12 章详细讨论）负责将经验蒸馏为更精炼的 SOP 和代码。三者缺一不可。

11.6 本章小结

本章详细展示了 GA 如何主动管理每一轮的上下文预算。核心要点：

1. **压缩优于扩展**：GA 不依赖扩大上下文窗口，而是将预算控制在 30k token 以内——因为当前模型的无幻觉上下文长度远小于名义窗口
2. **字符域预算公式**以 $\alpha \approx 3$ 的启发式系数将 token 窗口转换为字符预算，实现模型无关的预算管理
3. **四阶段压缩流水线**从细到粗逐层处理：工具级截断（控制增量）→ Tag 级压缩（控制存量）→ 消息驱逐（控制总量）→ 锚点注入（补偿损失）
4. **浏览器层的 DOM 语义提取**在信息源头就将 token 消耗降低了一个数量级

5. 工具 Schema 省略减少了每轮的固定开销

这套机制的设计哲学可以概括为：**与其让模型在一堆冗余信息中艰难寻找关键内容，不如主动确保模型看到的每一个字符都物有所值。**

现在，我们已经理解了 GA 的工具层（第 9 章）、记忆层（第 10 章）和压缩层（本章）如何协同工作来最大化上下文信息密度。但这三个机制都在解决“当前任务如何高效执行”的问题。还有一个更深层的挑战尚未触及——**跨任务的能力增长**。GA 能不能像一个经验丰富的员工一样，把过去的成功经验转化为未来的效率？

这就是下一章的主题——**自我进化**。

第 12 章 自我进化——从“用完即忘”到“越用越强”

在前面几章中，我们看到了 GA 的工具层如何控制信息密度（第 9 章）、记忆层如何组织跨任务知识（第 10 章）、压缩层如何管理单次任务内的上下文预算（第 11 章）。这三个机制共同解决了“**当前任务如何高效执行**”的问题。

但还有一个更深层的挑战尚未触及——**跨任务的能力增长**。

一个没有进化能力的 Agent，即使拥有完美的工具和记忆系统，每次面对相似的任务仍然要从头探索。它就像一个永远的新员工——每天都在重新学习同一套操作流程，永远无法积累经验。GA 如何打破这种“停滞循环”？

论文对此给出了一个关键的设计原则：**进化的是策略，不是工具 (What evolves: strategy, not tools)**。GA 将固定的工具层与可进化的知识层严格分离——工具接口和用户交互方式在运行期间始终不变，而所有任务特定的能力都以 SOP 文件和可复用脚本的形式存储在记忆系统中。Agent 可以自由地读取、创建和修改这些知识资产，而不会干扰底层工具的稳定性。这种分离确保了知识的安全增长：常见的子任务会自然地由纯文本指令进化为稳定的、可复用的脚本，而 Agent 的运行基础设施始终保持不变。

12.1 经验蒸馏的动机

12.1.1 直觉理解：新员工 vs 老员工

想象两个员工在做同一件事——从 GitHub 上调研一个开源项目的 PR 记录并生成报告。

新员工的做法：打开浏览器 → 搜索项目 → 找到 PR 页面 → 一个个点开看 → 手动记录 → 尝试不同的筛选方式 → 走了几条弯路 → 最终完成。整个过程需要 30 分钟，中间有大量的试错和重复操作。

老员工的做法：直接打开书签 → 用 GitHub API 批量拉取 PR 数据 → 按预定模板整理 → 5 分钟完成。他不需要“探索”，因为他已经把整个流程内化为一套标准操作。

这两个员工的差异不在于智商，而在于**经验是否被有效沉淀**。老员工过去的试错经历已经被压缩成了一套高效的操作流程——这就是**经验蒸馏 (Experience Distillation)**。

12.1.2 与信息密度的关系

从信息密度的视角来看，进化的价值在于：**让后续任务从更高密度的起点出发**。

没有进化时，每次任务的上下文都必须包含大量的探索性内容——尝试不同方案、处理意外错误、搜索替代路径。这些内容对“完成当前任务”是必要的，但对“执行一个已知流程”来说就是纯粹的冗余。

有了进化后，过去的探索成果被压缩为紧凑的 SOP 或可执行代码，直接注入后续任务的上下文。模型不需要重新推理“该怎么做”，而是直接执行“已经验证过的做法”。信息密度从一开始就处于高位。

12.2 GA 的三阶段进化路径

GA 的自我进化并非一步到位，而是经历三个渐进的阶段。我们用一组真实的实验数据来展示这个过程。

12.2.1 实验设置

实验任务是 **LangChain GitHub 研究任务**——调研 LangChain 项目的 PR 记录并生成结构化报告。实验以 **Claude Opus 4.6** 为骨干模型，对比基线为同样使用 Claude Opus 4.6 的 **OpenClaw**，以确保差异来自系统架构而非模型能力。同一个任务被重复执行 9 轮，GA 可以在每轮结束后将经验写入长期记忆。

12.2.2 Stage 1: 自然语言执行 (Round #1)

第一轮是纯粹的**从零探索**。GA 没有任何先验知识，需要通过 in-context reasoning 和 trial-and-error 来完成的任务：

- 搜索 LangChain 的 GitHub 地址
- 尝试不同的 PR 筛选策略
- 处理各种意外情况（页面加载失败、数据格式变化等）
- 逐步摸索出一套可行的流程

执行数据：耗时 **7 分 30 秒**，LLM 调用 **32 次**，总 token 消耗 **222,203**。

这是一个典型的”高熵探索”过程——大量的推理循环、大量的上下文重建、大量的试错开销。

12.2.3 Stage 2: SOP 蒸馏 (Rounds #2-#5)

从第二轮开始，GA 将第一轮的成功经验压缩为一份**文本化 SOP (Standard Operating Procedure)**。这份 SOP 记录了：哪些步骤是必要的、按什么顺序执行、在什么地方容易出错。

后续轮次不再从零探索，而是**读取 SOP → 按步骤执行 → 根据执行结果优化 SOP**。

轮次	阶段	耗时	LLM 调用次数	Cache Read	总 Token
#1	初始执行	7m30s	32	183,375	222,203
#2	SOP 优化	4m19s	12	51,883	66,341
#3	SOP 优化	2m53s	8	39,534	49,825
#4	SOP 优化	2m29s	9	41,376	51,758
#5	SOP 优化	2m50s	7	27,268	35,536

从 Round #1 到 Round #5，token 消耗下降了 **84%**，LLM 调用次数下降了 **78%**。但更重要的是理解**为什么效率在提升**：

- **消除了重复的推理循环**。Round #1 中模型需要反复推理”下一步该做什么”，而有了 SOP 后，这些推理被压缩为直接的步骤执行。效率提升的主要来源不是单次回复变短，而是 LLM 调用次数的大幅减少——整个”理解 → 推理 → 生成”的循环被从执行轨迹中移除了
- **减少了运行时上下文开销**。从表中的 Cache Read 列可以看到，Round #1 的 cache-read token 高达 183,375，而 Round #5 降到了 27,268——Agent 不再需要在每轮调用中重建大量的上下文
- **SOP 仍在迭代优化**。注意 Round #3 到 #4 的 token 略有回升——这是 SOP 在适应新发现的边界情况时产生的正常波动

注意这里的关键事实：**阶段转换不需要人工干预**。Agent 在任务执行过程中，会在适当的时机（如子目标完成或故障恢复后）主动调用 start_long_term_update 将经验蒸馏为 SOP 并写入 L3 层。下一次执行同类任务时，L1 索引会路由到这份 SOP，Agent 自动切换到”按 SOP 执行”模式。这里的”自动”指的是 Agent 自主完成了从探索到 SOP 蒸馏再到 SOP 执行的整个转换过程，而非系统在后台自动改写记忆。

12.2.4 Stage 3: 代码化执行 (Rounds #6-#9)

当文本 SOP 经过多轮验证趋于稳定后，GA 会将其进一步**编译为可执行代码**。这意味着模型不再需要”阅读 SOP 然后一步步执行”，而是直接调用一段预编写的脚本来完成整个流程。

轮次	阶段	耗时	LLM 调用次数	Cache Read	总 Token
#6	代码化 SOP	2m24s	6	20,913	25,762
#7	代码化 SOP	1m41s	5	18,249	23,014
#8	代码化 SOP	1m35s	5	18,229	22,689

轮次	阶段	耗时	LLM 调用次数	Cache Read	总 Token
#9	代码化 SOP	1m38s	5	19,034	23,010

进入代码化阶段后，系统收敛到一个**极窄的稳定带**：约 **23k±1k token**，**5 次 LLM 调用**，**1 分 40 秒**左右完成。

相对于 Round #1 的初始执行：- 时间减少 **78.2%**（7m30s → 1m38s）- LLM 调用减少 **84.4%**（32 → 5）- Token 消耗减少 **89.6%**（222,203 → 23,010）

三阶段进化的本质可以用信息密度来理解：

Stage 1: 自然语言执行 Stage 2: SOP 蒸馏 Stage 3: 代码化执行

大量探索性推理		SOP 指导执行		脚本直接执行	
大量试错分支		少量适应性推理		极少推理开销	
大量上下文重建		中等上下文需求		最小上下文需求	
信息密度：低		信息密度：中		信息密度：高	
222k tokens		36k-66k tokens		23k tokens	

12.2.5 对比实验：其他系统为什么不能进化？

上面的九轮实验展示了 GA 自身的进化轨迹。但一个关键问题是：**其他 Agent 系统在重复执行任务时也能实现类似的效率提升吗？**

在 HuggingFace 数据集下载任务的五轮重复实验中，答案是否定的：

- CodeX、Claude Code、OpenClaw 的操作时间和 token 消耗在各轮之间**基本保持稳定**，没有明显的改善趋势
- GA 的操作时间从第一轮的 **102 秒**下降到后续轮次的约 **66 秒**，token 消耗从 **200,439** 降至约 **100,000**

这些系统之间的差异不在于模型能力，而在于**架构是否支持经验积累**。CodeX、Claude Code 和 OpenClaw 没有将执行经验蒸馏为可复用知识的机制——每次执行都是从零开始的独立过程，无论重复多少次，效率都不会提升。

12.2.6 跨任务泛化

一个自然的后续质疑是：GA 的进化效果是否只对反复执行的**同一任务**有效？

论文通过一个包含 **8 类不同网页任务**的跨任务基准测试来回答这个问题，GA 和 OpenClaw 都使用 Claude Opus 4.6 作为骨干模型。结果表明，GA 后续执行相比首次执行的 token 节省率范围为 **61.0% 到 92.4%**，总体降低 **79.3%**。所有任务都展现出相同的内部趋势：第一次执行的 token 成本显著较高，第二、三次执行快速收敛到稳定的低成本区间。

图 12-1：GA 与 OpenClaw 在 8 个不同任务上三轮重复执行的 token 收敛对比。每个子图对应一个基准任务，红线为 OpenClaw，蓝线为 GA。GA 在所有任务上都展现出从首次执行到后续执行的快速收敛，而 OpenClaw 的 token 消耗在各轮之间波动甚至上升。（来源：论文 Figure 5）

这种收敛模式揭示了一个重要机制：即使有了 SOP 记忆，首次执行仍然存在 **SOP 适配成本**——Agent 需要将通用化的 SOP 知识映射到当前任务实例的具体页面结构和交互流程上。但这种适配只需发生一次。一旦成功路径与环境对齐，后续执行就可以直接复用，token 成本迅速趋于稳定。

进化收益与任务复杂度呈正相关：涉及状态转移和错误恢复的长链任务（如跨页面状态传递）节省率高达 **92.0%**，因为 SOP 充当了”路径压缩器”——一旦验证过的路线被存储，Agent 可以完全跳过大部分探索性分支。

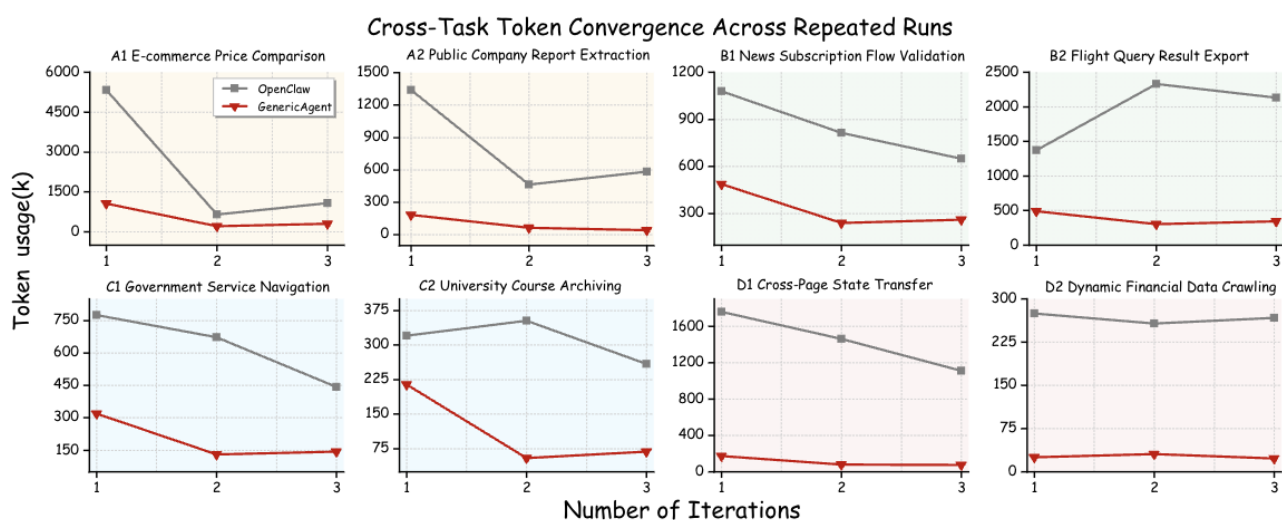


Figure 7: 跨任务 Token 收敛趋势

作为对比，OpenClaw 在三轮重复执行中没有表现出可比的收敛模式——token 消耗在各轮之间波动甚至上升，说明系统持续在重新探索而非复用经验。

12.3 进化的触发与守门

12.3.1 什么时候触发记忆蒸馏

GA 提供了一个专门的工具 `start_long_term_update` 来触发长期记忆的更新。模型在以下情境中被引导调用这个工具：

- **子目标成功完成时**：一个阶段性的成功值得记录
- **故障恢复转换时**：从错误中恢复的过程包含了宝贵的调试经验
- **发现可复用模式时**：识别到某个操作序列在未来可能被重用

12.3.2 准入标准：防止“垃圾进化”

并非所有经验都值得保存。如果不加筛选地将所有执行记录写入长期记忆，记忆库很快就会被一次性的调试信息、失败的尝试、以及过时的配置所污染。这就是**垃圾进化 (Junk Evolution)** 问题。

GA 通过两道“守门”机制来防止垃圾进化：

第一道门：执行验证。只有经过实际执行并验证成功的经验才能写入。“我觉得应该这样做”不够——必须是“我这样做了，而且成功了”。

第二道门：跨任务可复用性。一次性的、特定于当前任务上下文的信息不应写入长期记忆。写入的信息必须具有在未来任务中被重用的潜力。

需要注意的是，这两道准入标准应被理解为**引导性规则 (Consolidation Guidance)**，而非完全自动化的强制过滤。GA 提供了记忆管理规范来引导 Agent 的写入行为，但当前实现并未包含对信息有效性、稳定性或未来复用价值的完整自动验证机制。

12.3.3 自我改进日志

除了 SOP 和代码形式的经验沉淀，GA 还维护了一个**独立的轻量级自我改进日志 (Self-Improvement Log)**，记录三类特定的信息：

1. **观察到的错误及其修正**：某次操作失败的原因和对应的纠正方案

2. **用户显式偏好**：用户在交互中表达的工作习惯和偏好
3. **经验证的成功模式**：被实践验证有效的操作模式

这些条目会被**自动注入系统提示词**，持续引导 Agent 的未来决策。与 L3 SOP 不同，这个日志关注的不是“怎么做一个任务”，而是“在做任何任务时应该注意什么”。

需要注意的是，当前实现中这个日志的维护**依赖人工参与**——用户需要确认哪些错误和偏好值得记录。这限制了 Agent 自主影响核心行为的程度，是当前系统的一个已知局限。

12.3.4 故障升级作为进化安全阀

进化过程中的一个关键保护机制是**故障升级 (Failure Escalation)**。当 GA 在执行过程中遇到失败，它不会盲目重试，而是遵循论文 § 2.3.3 描述的**三步渐进恢复流程**：

1. **局部修正**：分析当前错误信息，做小幅定向调整后重试
2. **策略切换**：如果失败持续，放弃当前方案，强制切换到全新策略或搜索环境中缺失的信息
3. **人类介入**：如果所有自动化尝试均失败，暂停执行并请求人类干预

这个机制确保了 GA 不会将错误的“经验”固化为 SOP。如果一条路径反复失败，它会被标记为死胡同，而不是被记忆为“标准做法”。

12.4 自主探索：让 Agent 自己“上课”

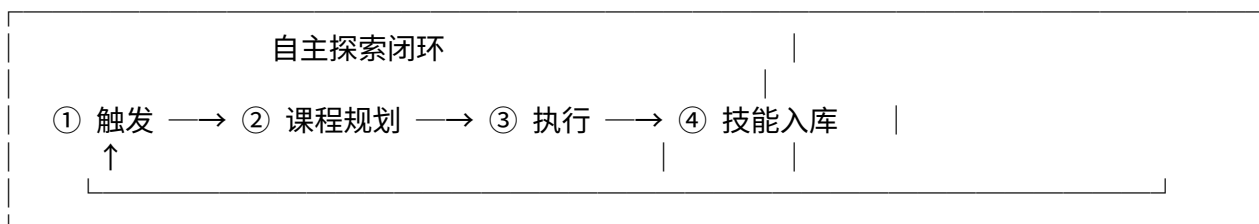
除了在执行用户任务时被动地积累经验，GA 还支持一种更激进的进化方式——**自主探索 (Autonomous Exploration)**。

12.4.1 直觉理解

自主探索就像一个员工在没有紧急任务时，主动学习新技能、探索新工具、扩展自己的能力范围。它不是等着老板布置任务才开始学习，而是自己规划课程表。

12.4.2 探索的闭环流程

自主探索由四个阶段组成一个闭环：



① **触发**：自主探索可以通过两种方式启动。在**任务模式 (task mode)** 下，外部编排器（如 cron job）向文件系统信箱提交一次性任务；在**反射模式 (reflect mode)** 下，系统轮询一个用户提供的回调函数（默认每 6 分钟一次），当回调返回非空字符串时，该字符串被注入为新的提示词，启动自主学习。两种模式都是 Reflect Mode 原语的具体实例化，不需要对核心架构做任何扩展。

② **课程规划**：GA 检查当前技能库的分布，生成一份优先级排序的任务清单。任务候选项通过一个**四维评分公式**来排序：

$$S(t) = w_b \cdot B(t) + w_d \cdot D(t) + w_u \cdot U(t) + w_i \cdot I(t)$$

四个维度的直觉含义：
- **广度 $B(t)$** ：这个技能是否能填补技能树中的空白区域？如果某个类别的技能数量远低于平均水平，学习该类别的新技能得分更高
- **深度 $D(t)$** ：这个技能是否能强化已经常用的

能力？使用频率越高的技能，进一步深化的价值越大 - **实用性** $U(t)$ ：这个技能在实际工作中被用到的可能性有多大？ - **创新性** $I(t)$ ：这个技能是否涉及新的技术领域或方法论？

广度和深度有具体的计算公式，基于技能树的统计数据：

$$B(t) = 10 \times \max \left(0, 1 - \frac{|S_c|}{\bar{S} + 1} \right)$$

$$D(t) = 10 \times \frac{u(t)}{u_{\max} + 1}$$

其中 $|S_c|$ 是目标类别中的技能数量， $\bar{S}$ 是所有类别的平均技能数， $u(t)$ 是目标技能的使用次数， $u_{\max}$ 是所有技能中的最高使用次数。

默认权重为 $(w_b, w_d, w_u, w_i) = (0.3, 0.2, 0.3, 0.2)$ ，优先考虑广度和实用性。

③ **执行**：GA 使用标准的工具集（搜索、代码执行、文件读写）来执行任务，受 **30 轮交互上限** 约束。所有操作限制在临时目录内，禁止访问密钥和核心源码。

④ **技能入库**：执行完成后，结果被写入结构化报告，新技能注册到技能树中。即使是失败的实验也会被记录，以便未来的规划能避开同样的死胡同。

12.4.3 反思驱动的权重自适应

四维评分的权重不是固定不变的。GA 实现了一个**反思驱动的权重自适应机制（Reflection-Based Weight Adaptation）**：

- 如果一个任务的预测得分很高 ($S(t) > 8.0$) 但实际使用率很低 (30 天内使用次数 < 3)，说明得分中**贡献最大的那个维度**的权重被高估了 → 该主导维度的权重降低 10%
- 如果一个任务的预测得分很低 ($S(t) < 5.0$) 但实际使用率很高 (使用次数 > 5)，说明某个维度的权重被低估了 → 对应维度的权重增加 10%
- 调整后的权重被重新归一化，确保总和为 1

这个反馈循环让系统能够逐渐发现**哪些维度在当前用户的工作流中最能预测实际价值**，无需手动调参。

12.4.4 技能树结构

技能库被组织为一个**二级映射**：

技能树

```
|—— web_automation (类别)
|   |—— web_scraping (技能) → 使用次数: 12
|   |—— form_filling (技能) → 使用次数: 5
|   |—— ...
|—— data_processing (类别)
|   |—— csv_parsing (技能) → 使用次数: 8
|   |—— ...
|—— ...
```

每个技能关联着对应的工具脚本和一个单调递增的使用计数器。技能树通过一组 API 支持查询、修改、使用追踪和原子化的完成操作。

12.4.5 局限性

自主探索机制目前存在几个已知局限：

1. **30 轮上限**：复杂任务可能需要跨多个会话完成，连续性仅通过报告和任务清单注释来维护
 2. **权重自适应的初步性**：反思机制尚未积累足够的数据来验证其在多样化用户工作流中的有效性
 3. **自我改进日志的手动性**：记录错误和偏好的自我改进日志目前依赖人工维护，限制了 Agent 自主影响核心行为的程度
 4. **技能树维护的手动性**：合并冗余类别、弃用过时技能、扩展类别体系——这些工作目前仍需人工参与
-

12.5 本章小结

本章展示了 GA 如何从”用完即忘”进化到”越用越强”。核心要点：

1. **进化的是策略，不是工具**：GA 将固定的工具层与可进化的知识层严格分离，所有任务特定的能力以 SOP 和脚本形式存储，确保知识安全增长
2. **三阶段进化路径**（自然语言执行 → SOP 蒸馏 → 代码化执行）将高熵探索逐步压缩为低成本的可确定性执行，实现了 89.6% 的 token 节省
3. **进化的触发与守门**通过执行验证和可复用性筛选，防止垃圾进化污染记忆库
4. **自我改进日志**记录错误修正、用户偏好和成功模式，自动注入系统提示词以持续引导行为
5. **自主探索**让 Agent 在空闲时主动扩展能力边界，通过四维评分和反思自适应来规划学习课程
6. **故障升级**确保失败经验不会被固化为错误的标准操作

自我进化的核心意义在于：**它将每一次任务执行都变成了一次学习机会，让后续任务从更高的信息密度起点出发**。这不是参数层面的学习（模型权重没有改变），而是**环境层面的进化**——模型运行的信息环境在持续优化。

至此，我们已经完整地剖析了 GA 的四大核心机制：工具层（第 9 章）、记忆层（第 10 章）、压缩层（第 11 章）、进化层（本章）。一个自然的问题浮现：这些机制加在一起，能产生什么超出各部分之和的效果？为什么一个只有 9 个工具、没有专用子 Agent 管理器、没有事件总线的系统，却能实现并行任务分发、定时监控、自主运维这些”高级功能”？

这就是最后一章的主题——**涌现能力**。

第 13 章 涌现能力——当约束催生创造力

在前面五章中，我们逐一剖析了 GA 的四大核心机制：最小原子工具集（第 9 章）、分层记忆架构（第 10 章）、上下文截断与压缩（第 11 章）、自我进化（第 12 章）。每个机制都在各自的维度上最大化信息密度。

现在，一个自然的问题浮现：这些机制加在一起，能产生什么**超出各部分之和**的效果？

答案出人意料——一个只有 9 个工具、没有专用子 Agent 管理器、没有事件总线、没有调度守护进程的系统，却能实现**子 Agent 并行分发、看门狗监控、定时任务调度、自主空闲行为**这些”高级功能”。而且每一种功能的实现代码都极其简短，有的甚至不到 10 行。

这不是魔法，也不是”意外涌现”——而是**当组合成本足够低时，复杂行为从简单原语中自然产生**。

13.1 从”功能内置”到”能力涌现”

13.1.1 传统框架的做法

大多数 Agent 框架采用**功能内置（Feature-Built-In）**的设计策略：每当需要一个新的高级功能，就添加一个专用模块。

- 需要子 Agent？→ 添加一个子 Agent 管理器（SubAgent Manager）
- 需要定时任务？→ 添加一个调度守护进程（Scheduler Daemon）
- 需要事件驱动？→ 添加一个事件总线（Event Bus）
- 需要多 Agent 协作？→ 添加一个编排层（Orchestration Layer）

这种方法的好处是直观——每个功能都有明确的模块对应。但代价是**系统复杂度线性增长**：每个新模块都带来新的接口、新的配置、新的故障模式、新的维护负担。

13.1.2 GA 的做法：组合而非扩展

GA 走了一条完全不同的路。论文指出，GA 在两个维度上展现架构极简性：**代码极简性（code minimality）**和**接口极简性（interface minimality）**。

代码极简性：GA 的核心代码仅约 **3,300 行**，其中 Agent Loop 仅 **92 行**。作为对比，OpenClaw 的代码量约 **530,000 行**——是 GA 的 **160 倍以上**。GA 没有依赖大量模块依赖或复杂的工具注册基础设施，而是将核心逻辑完整地表达在一个极小的代码库中。

接口极简性：GA 将自身暴露为一个**自托管的 CLI 程序（Self-Hosted CLI）**，命令行不是内部平台的外壳包装，而是系统的原生执行界面。GA 不需要插件框架、事件总线或专用编排层来支持高级行为——所有交互（任务提交、进度监控、运行时干预）都通过这个统一的 CLI 接口完成。

这种极端的极简性不是功能上的妥协，而是一个释放更大能力的设计选择。因为架构被精简到只剩 CLI，**子 Agent 分发（Subagent Dispatch）**和**反射模式（Reflect Mode）**等高级行为就从这个单一原语中自然涌现。

这种设计背后的赌注是：**一个原语集足够小、组合成本足够低的系统，能够覆盖比”逐一内置”更广泛的行为空间**。

这里的”涌现”不是物理学意义上的涌现——不是”微观规则产生不可预测的宏观行为”。而是工程意义上的涌现：**当基础组件足够通用且组合足够廉价时，设计者未预先规划的功能可以在需要时被轻松实现**。关键不在于”意外”，而在于”低成本”。

13.2 两种执行模式与基础原语

论文 § 2.2 定义了 GA 的两种执行模式：

- **交互模式 (Interact Mode)**：处理用户发起的任务——代码执行、信息检索、文件操作
- **反射模式 (Reflect Mode)**：不需要用户指令；持续监控环境变化，当检测到特定条件时自动触发对应任务

两种模式共享同一个 Agent Loop 和记忆系统，区别仅在于**任务如何触发**和**输出如何处理**。这种统一意味着 GA 不需要为”后台守护”“定时调度”等行为引入新的架构层——它们都是反射模式的具体实例化。

这两种模式加上文件系统约定，构成了 GA 所有高级行为的基础：

原语	本质	一句话描述
可调用 CLI 入口点	Agent 就是一个命令行程序	通过参数或文件接收任务，运行到完成。可前台、后台或轮询模式启动
文件协议	目录约定的跨进程通信	input 文件送任务、output 文件流结果、reply 文件续对话、control 文件做干预
反射模式（轮询 + 热重载）	外部脚本定义触发条件	运行时周期性求值触发脚本；脚本文件被修改时自动重新加载，无需重启

让我们逐一理解这些原语为什么重要。

13.2.1 可调用 CLI 入口点

GA 采用**自托管 CLI (Self-Hosted CLI)** 模型：命令行不是内部平台的外壳包装——它就是系统的原生执行界面。这意味着：

- 任何能调用命令行的程序都能调用 GA
- GA 自己也能调用自己（这直接催生了子 Agent）
- 不需要插件框架、SDK 或 API 网关

这个设计选择看似简单，实际上打开了一扇巨大的组合之门。因为命令行是操作系统中最普遍的进程间通信方式，GA 天然地与整个 Unix 工具生态兼容。

13.2.2 文件协议

跨进程通信使用一个**目录约定 (Directory Convention)**：

```
task_directory/
├── input.txt          # 任务输入
├── output.txt         # 执行输出（带轮次完成标记）
├── reply.txt          # 多轮对话的续接
├── stop.txt           # 请求优雅终止
├── inject_context.txt # 向工作记忆注入内容
└── inject_message.txt # 追加指令到执行中
```

这不是”发射后不管 (fire-and-forget)”的模式。父进程可以：- 通过读取 output.txt 来**监控进度** - 通过写入 reply.txt 来**继续对话** - 通过 control 文件来**运行时干预** - 通过 verbose 标志让子进程输出原始工具结果，实现**中间结果审计**

文件协议的关键优势：它不依赖任何框架特定的通信层——不需要 gRPC、消息队列或共享内存。任何能读写文件的程序都能参与协作。

13.2.3 反射模式：轮询循环 + 热重载

反射模式加载一个外部脚本，该脚本定义了一个触发条件和一个检查间隔。运行时周期性地求值这个条件；当条件满足时，返回的字符串被作为任务分发到标准流水线。

论文指出，看门狗和定时任务是反射模式的**两个具体应用（two concrete applications）**。两者共享完全相同的底层机制，区别仅在于触发脚本的编写方式——看门狗监控环境变化，定时任务检查时间条件。

热重载是这个原语的关键特性：因为触发规则作为独立脚本维护在 Agent 之外，开发者可以在运行时修改它们，**无需重启核心 Agent 进程**。

13.3 子 Agent：递归自调用

有了 CLI 入口点和文件协议，**子 Agent（SubAgent）** 的实现就是水到渠成的事情。

13.3.1 实现原理

父 Agent 通过 `code_run` 工具启动一个新的 GA 进程——这个新进程就是子 Agent。它不是一个特殊对象，不需要专门的管理器。父和子是**同构进程（Homogeneous Processes）**——不存在特权的子 Agent 运行时对象，双方遵循完全相同的交互协议。

```
# 父 Agent 的视角（伪代码）
# 1. 准备子任务
write_file("subtask_dir/input.txt", " 请调研 LangChain 的最新 PR")

# 2. 启动子 Agent（就是启动自己的另一个实例）
code_run("python3 ga_cli.py --task-dir subtask_dir --background")

# 3. 监控进度
while not is_complete("subtask_dir/output.txt"):
    progress = read_file("subtask_dir/output.txt")
    # 必要时干预
    if needs_correction(progress):
        write_file("subtask_dir/inject_message.txt", " 请调整方向...")

# 4. 收集结果
result = read_file("subtask_dir/output.txt")
```

13.3.2 天然的上下文隔离

这种设计带来了一个重要的架构优势：**上下文天然隔离**。每个子 Agent 维护自己独立的对话历史，子任务之间不会互相干扰。

在传统框架中，实现子 Agent 的上下文隔离通常需要复杂的状态管理——哪些历史消息属于哪个子任务、如何在切换子任务时保存和恢复上下文。在 GA 中，这些问题根本不存在——因为每个子 Agent 是一个独立进程，有自己独立的内存空间。

需要注意的是，协调开销是真实存在的。父 Agent 需要主动监控子 Agent 的进度并及时干预——这些协调行为完全通过共享的文件约定来表达，而非通过额外的管理层。低架构复杂度不等于零运行时成本。

13.3.3 Map-Reduce 模式

对于**尴尬并行（Embarrassingly Parallel）**的工作负载，子 Agent 自然形成一个 Map-Reduce 模式：

1. **Map**：父 Agent 准备 N 个独立的 input 文件，为每个文件启动一个子 Agent
2. **Reduce**：等所有子 Agent 完成后，父 Agent 合并 N 个 output 文件

实际案例：当 GA 需要做一个并行调研时（如“分析 5 个竞品的技术架构”），父 Agent 为每个竞品准备一个独立的 input 文件，为每个文件启动一个子 Agent，待所有子 Agent 完成后合并 output 文件，做最终的综合分析。整个过程不需要扩展核心运行时的任何一行代码。

13.4 看门狗：反射模式的第一个应用

13.4.1 实现原理

看门狗（Watchdog）是反射模式的第一个具体应用：监控环境变化（如新文件出现、错误日志更新），一旦检测到变化立即触发 GA。实现方式就是一个触发脚本编码了一个任意条件；轮询循环按配置的间隔求值这个条件。当条件满足时，返回的字符串进入标准任务流水线。

一个看门狗触发脚本的示例

```
def check():
    # 检查某个目录是否有新文件
    import os
    new_files = [f for f in os.listdir("/watch/dir") if f.endswith(".csv")]
    if new_files:
        return f"发现 {len(new_files)} 个新 CSV 文件，请处理并生成报告"
    return None # 条件未满足，不触发
```

13.4.2 触发与执行的解耦

这个设计的精妙之处在于**触发与执行的彻底解耦**：

- **触发脚本**只负责回答一个问题：“是否需要创建一个新任务？”
- **执行**完全走标准的任务流水线——与用户手动提交的任务没有任何区别

这种解耦带来了极大的灵活性：- 触发规则可以是任意的 Python 代码——文件监控、API 状态检查、系统资源告警、邮件到达通知……- 修改触发规则不需要修改执行逻辑 - 热重载让规则修改即时生效

13.5 定时任务：反射模式的第二个应用

13.5.1 实现方式

定时任务（Scheduled Task）是反射模式的第二个应用：使用基于时间的规则，在特定间隔或精确时间点生成 GA 任务。GA 的定时任务系统是一个**约 100 行的触发脚本**，建立在反射模式和文件协议之上。它的工作方式是：

1. 轮询一个目录中的 JSON 任务定义
2. 每个 JSON 文件指定了：调度时间、重复策略（每日/工作日/每周/每月/一次）、最大延迟窗口
3. 脚本检查每个任务是否到了执行时间
4. 到时间的任务被分发到标准流水线

```
{
  "name": " 每日报告",
  "schedule": "09:00",
```

```
"repeat": "daily",
"max_delay": "30m",
"task": " 生成昨日工作总结并发送到飞书"
}
```

13.5.2 辅助机制

- **冷却检测 (Cooldown Detection)**：防止同一任务被重复触发。通过检查上次执行时间来判断是否已经在冷却期内
- **最大延迟窗口**：如果 Agent 在调度时间不在线（比如电脑关机了），在超过延迟窗口后该次调度会被跳过，而不是在开机后立即执行所有积压任务
- **执行报告作为 last-run 追踪**：每次执行后生成的报告同时作为”上次运行记录”的持久化存储

13.5.3 关键事实

论文强调，看门狗和定时任务的共同特征是：**两者都不需要扩展核心循环**。两者都从 CLI 这一个原语中涌现为反射模式的具体实例，而核心运行时始终保持稳定。这体现了架构极简性的深层价值：一个足够简单的系统可以在最低成本下支持最广泛的行为组合。

13.6 自主空闲行为：<10 行的持续自治

13.6.1 实现

自主空闲行为的触发脚本**不到 10 行**：

```
# 自主空闲行为触发脚本（简化版）
CHECK_INTERVAL = 1800 # 30 分钟
```

```
def check():
    return " 从待办清单中选择一个自我提升任务并执行"
```

每隔 30 分钟，当用户处于非活跃状态时，Agent 自动从维护的任务待办清单中选取一项并执行。这就是第 12 章中介绍的自主探索机制的一个触发入口。

注意：第 12 章介绍自主探索时提到的”默认每 6 分钟检查一次”来自论文 § 3.3 对通用自主探索触发频率的描述，而本节的 30 分钟间隔来自 GA 实际实现中 reflect/autonomous.py 脚本的默认配置。两者是同一机制的不同配置——触发间隔由脚本定义，可根据场景调整。

13.6.2 意义

这个功能的意义不在于实现的复杂性——它几乎是平凡的。意义在于它**证明了一个观点**：持续的自主行为不需要一个专门的”自主模式”或”后台守护进程”。它只是已有原语的一个简单配置。

13.7 为什么”少即是多”

让我们回顾一下这四种高级能力的实现方式：

高级能力	使用的原语	额外代码量	是否修改核心运行时
子 Agent	CLI + 文件协议	0（直接组合）	否
看门狗	反射模式 + 文件协议	~20 行触发脚本	否

高级能力	使用的原语	额外代码量	是否修改核心运行时
定时任务	反射模式 + 文件协议	~100 行触发脚本	否
自主空闲	反射模式	<10 行触发脚本	否

没有一种能力需要扩展核心循环。子 Agent 复用 CLI 和文件协议。看门狗、定时任务和自主行为都是反射模式的不同配置。

这与传统框架形成了鲜明对比。在传统框架中，每增加一种高级能力通常意味着：新的模块、新的配置项、新的 API 接口、新的文档页面、新的测试用例、新的潜在故障点。而在 GA 中，新的高级能力只是**已有原语的新排列组合**。

这就是第 7 章提到的“信息密度最大化”原则在架构层面的终极体现：**不是通过增加复杂度来获得更多功能，而是通过保持简单来释放更多可能性**。约束不是限制，而是创造力的来源。当你只有三个原语可以使用时，你会发现它们的组合空间远比你想象的要大。

13.8 本章小结

本章展示了 GA 如何从极简架构（3,300 行代码、92 行 Agent Loop）和两种执行模式（交互模式 + 反射模式）中组合出四种高级能力（子 Agent、看门狗、定时任务、自主空闲行为），而无需扩展核心运行时。

这不是偶然的涌现，而是**有意的设计选择**：定义一组足够通用的底层原语，让它们的组合成本足够低，高级行为就会在需要时自然出现。这种设计哲学与 Unix 的管道哲学一脉相承——小工具、明确接口、自由组合。

全书回顾

至此，我们完整地走过了 GA 的原理世界。让我们站在高处回望整个旅程：

- **第 7 章**确立了第一性原理：**上下文信息密度最大化**是 Agent 效能的根本杠杆
- **第 8 章**展示了系统全貌：四大机制如何围绕这一原理协同工作
- **第 9 章**深入工具层：9 个原子工具如何通过组合覆盖无限的任务空间
- **第 10 章**深入记忆层：四层分级存储如何在“记住更多”和“保持精炼”间取得平衡
- **第 11 章**深入压缩层：四阶段流水线如何主动管理每一轮的信息预算
- **第 12 章**深入进化层：三阶段蒸馏如何将探索性经验转化为高效的确定性执行
- **第 13 章**展示涌现能力：三个原语如何组合出四种高级行为

贯穿这七章的核心洞见只有一个：**好的 Agent 设计不是做加法，而是做减法**。减少工具数量，让每个工具承载更高的信息密度。减少默认注入，让按需加载保持上下文的精炼。减少架构模块，让简单原语的组合释放更大的可能性。

这就是 GA 给我们的启示：在 AI Agent 的世界里，**少，就是多**。

Case 1：自动在社交软件上发消息 / 图片 / 文件

学完本章，你将掌握如何让 GA 自动操控桌面社交软件，聪明的读者一定会触类旁通，把流程用到你想用的国民级社交软件上

[:bullseye:] 学习目标

- 1. 理解 GA 操控桌面 GUI 应用的通用思路
- 2. 学会「激活窗口 → 搜索联系人 → 验证会话 → 发送内容 → 验证结果」的标准五步流程
- 3. 掌握 SOP + 工具函数的封装模式，让 GA 可以自主完成复杂的多步任务

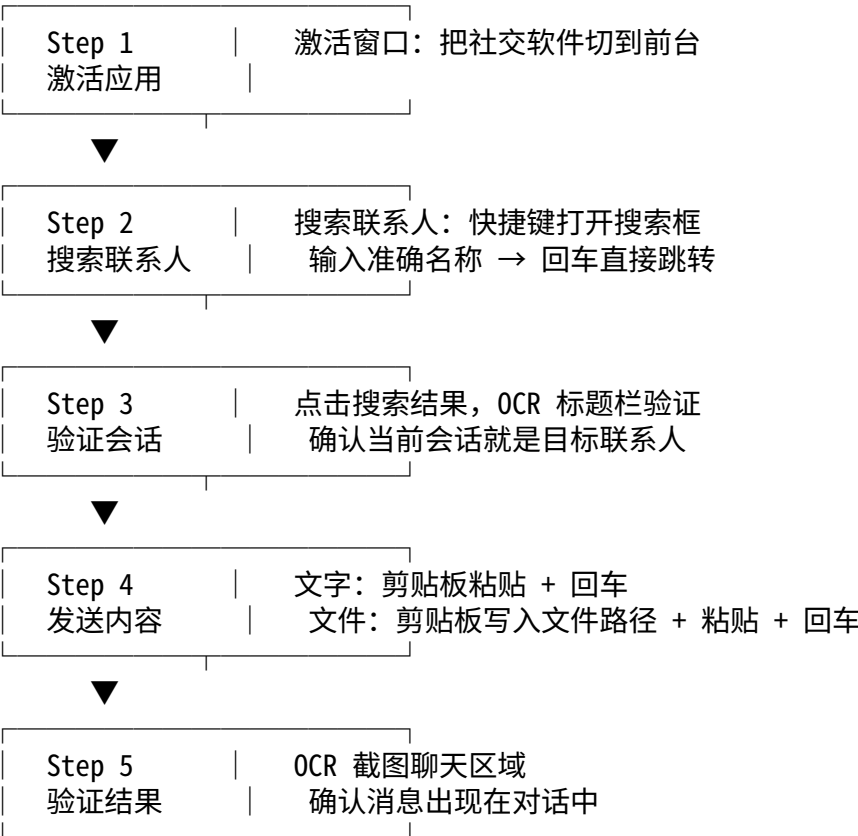
14.1 适用场景

[:light_bulb:] 本章介绍的方法适用于 **不提供命令行接口（CLI）或开放 API 的社交软件**。GA 通过「看屏幕 + 点鼠标 + 敲键盘」的方式来模拟人的操作，因此无论软件是否开放接口，都能实现自动化。

如果你使用的社交软件提供了 CLI 或 API 接口（如飞书、Slack 等），推荐优先通过接口实现自动化，效率更高也更稳定。可以参考 [第 3 章：CLI 工具的使用](#) 中的方法。

14.2 整体流程概览

在开始之前，我们先理解整个自动化的流程。不管是什么社交软件，自动发消息的核心步骤都是一样的：



[:warning:] **安全原则：**Step 3 验证通过之前，绝对不能执行 Step 4 的发送操作——否则可能发错人！

14.3 Step 1: 激活应用窗口

第一步是把社交软件的窗口切到前台。GA 需要做三件事：

1. **查找窗口**：通过窗口标题关键词找到社交软件
2. **恢复窗口**：如果窗口被最小化，需要先恢复
3. **置顶窗口**：确保窗口在最前面，这样后续的点击和截图才能正常工作

激活后，记录窗口的四个边界坐标（左、上、右、下），后续所有步骤都要基于这些坐标来定位界面元素。

[light_bulb] **小知识**：Windows 有一个安全限制——只有当前前台进程才能调用“设为前台窗口”的 API。通常需要先模拟一次 Alt 键事件来绕过这个限制。这类细节 GA 会自动帮你处理。

14.4 Step 2: 搜索联系人

窗口激活后，接下来要找到目标联系人。

:::tip [bullseye] 搜索比直接点击更可靠 不要试图直接点击会话列表中的联系人——列表的位置会随新消息到来而变化，很容易点错人。**一律走搜索流程。** :::

操作流程

1. **快捷键打开搜索框**：按下 Ctrl+F（Windows）或 Command+F（macOS）唤出搜索框。用快捷键比点击搜索框更可靠——不需要计算搜索框的坐标位置，也不会因为界面布局变化而失效
2. **清除残留文字**：先全选（Ctrl+A），防止上次搜索的文字还在
3. **输入联系人名称**：通过剪贴板粘贴，不要直接键盘输入——中文输入法可能导致乱码
4. **回车跳转**：按下 Enter 键，直接跳转到该联系人的聊天界面

[warning] **前提**：用户需要提供 **准确的联系人名称**，确保搜索结果能精确匹配并一步到位跳转到正确的会话。

14.5 Step 3: 验证当前会话

点击搜索结果后，**不要急着发消息！**必须先验证当前打开的会话确实是你要找的人。

为什么要验证？

- 点击位置可能偏移，打开了错误的会话
- 某些软件会弹出广告窗口拦截你的点击
- OCR 识别可能有误差，导致点了错误的搜索结果

验证流程

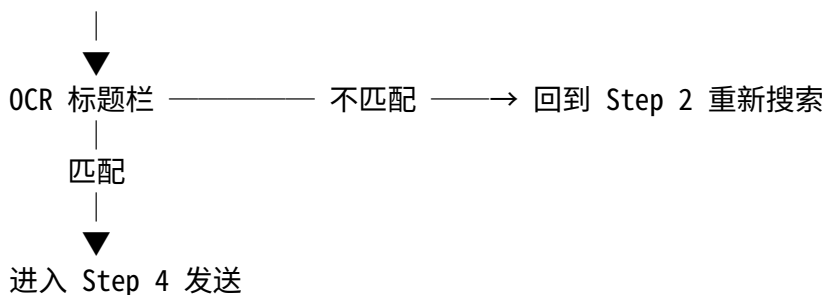
点击搜索结果



检测广告弹窗 ————— 有弹窗 ———→ 关闭弹窗，回到 Step 2 重新搜索



无弹窗



- **广告弹窗检测**：有些社交软件搜索结果中混入了广告，点击后会弹出新窗口而不是切换会话。GA 可以通过枚举系统窗口、检查窗口类名来识别并自动关闭这些弹窗。
- **OCR 标题栏验证**：截取聊天窗口顶部的标题栏区域做 OCR，确认显示的名字与目标联系人一致。大模型会做模糊匹配，容忍 OCR 的形近字误差。

14.6 Step 4：发送内容

验证会话正确后，终于可以发送内容了。根据内容类型，有两种方式：

发送文字消息

1. **点击输入框**：根据窗口坐标计算输入框位置，点击使其获得焦点
2. **粘贴消息**：把要发送的文字复制到剪贴板，然后 Ctrl+V 粘贴
3. **回车发送**：按下 Enter 键

[light_bulb:] **为什么用剪贴板而不是直接输入？** 直接键盘输入会受输入法干扰——如果当前是中文输入法，英文可能被转成拼音。通过剪贴板粘贴可以完美避免这个问题，而且支持任何语言和 emoji。

发送文件 / 图片

发送文件的思路非常巧妙：模拟「从资源管理器复制文件，然后在聊天窗口粘贴」的操作。

1. **写入剪贴板**：把文件的绝对路径以操作系统的“文件拖放”格式（CF_HDROP）写入剪贴板——这和你在资源管理器里右键复制一个文件的效果完全一样
2. **点击输入框**：点击聊天输入框
3. **粘贴**：Ctrl+V，文件会以附件或图片形式出现在输入框中
4. **回车发送**：按 Enter 键

社交软件会自动判断文件类型：图片以图片形式展示，其他文件以附件形式展示。

[warning:] **注意**：文件路径必须是 **绝对路径**，相对路径会导致软件找不到文件。

14.7 Step 5：验证发送结果

消息发出后，我们需要确认它真的发送成功了。

OCR 验证法

发送完成后，对聊天区域进行截图并 OCR 识别，检查是否包含刚才发送的消息内容：

1. **截取聊天区域**：截取消息列表的下半部分（最新消息通常在底部）
2. **OCR 识别**：提取截图中的所有文字
3. **匹配验证**：检查 OCR 结果中是否包含刚才发送的关键词

对于文件和图片，可以检查是否出现了文件名，或者通过视觉识别确认缩略图已出现在聊天区域中。

[light_bulb:] GA 还可以用视觉能力 (ask_vision) 直接”看”截图来判断消息是否发送成功，这在 OCR 不够准确时是一个很好的补充手段。

14.8 实战：让 GA 读本章内容并实践

理解了整个流程之后，最好的学习方式就是让 GA 亲自动手试一试。具体做法非常简单：

1. **把本章内容喂给 GA**：将本章的 Markdown 给 GA 看，或者直接给网址告诉 GA 去阅读这份文档
2. **给 GA 一个实际任务**：比如「帮我在社交软件上给张三发一条消息：下午开会别迟到」
3. **观察 GA 的执行过程**：GA 会参考本章的流程，逐步完成激活窗口、搜索联系人、验证会话、发送消息、验证结果
4. **遇到问题一起调试**：如果某个步骤失败了（比如搜索框快捷键不对、输入框坐标偏了），和 GA 一起排查原因并修正

将经验沉淀为 SOP

实践成功后，最重要的一步是 **把这次成功的经验固化为一份 SOP**，存入 GA 的记忆中。

你可以告诉 GA：

你：刚才发消息的流程跑通了，请把这次的经验整理成一份 SOP，保存到记忆目录里。

GA 会自动把本次实践中用到的具体参数（搜索框快捷键、输入框坐标区域、等待时间等）和踩过的坑一起写入 SOP。下次再执行同样的任务时，GA 会直接读取这份 SOP，不需要重新摸索。

[graduation_cap:] **这就是 GA 的核心工作模式**：先学习通用流程 → 实践验证 → 沉淀为专属 SOP。每一次实践都在让 GA 变得更强大。

14.9 避坑指南

在实际使用中，有一些常见的坑需要注意：

[prohibited:] 绝对禁止的操作

操作	后果	替代方案
按 Esc 键 (Windows)	某些社交软件在 Windows 上按 Esc 会完全隐藏窗口，macOS 不受影响	点击侧栏其他会话来清除搜索状态
直接键盘输入中文	输入法干扰，可能输出乱码	剪贴板复制 + Ctrl+V 粘贴
发送前不验证会话	消息发错人	必须 OCR 标题栏确认

[light_bulb:] 实用技巧

1. **DPI 缩放处理**：高分辨率屏幕需要处理缩放比例。窗口坐标是逻辑坐标，OCR 和点击需要物理坐标，需要做坐标转换
2. **搜索框残留文字**：每次搜索前先全选 (Ctrl+A) 再粘贴，覆盖掉上次搜索的内容
3. **随机偏移**：连续多次点击同一位置可能触发风控，可以给坐标加一个小范围的随机偏移
4. **群聊和个人会话**：发送流程完全一样，不需要区分处理

14.10 本章小结

本章我们通过「在社交软件上发消息」这个 Case，学习了 GA 操控桌面应用的完整流程：

核心概念	说明
五步流程	激活 → 搜索 → 验证 → 发送 → 验证结果
SOP 模式	把操作流程固化为 SOP，GA 按步骤执行
工具封装	每个步骤封装为函数，放入 <code>utils.py</code>
决策点介入	确定性步骤自动执行，模糊判断交给大模型
安全原则	验证前不发送、剪贴板粘贴、OCR 验证结果

[[:graduation_cap:]] **举一反三**：这套「激活 → 定位 → 验证 → 操作 → 验证」的模式不仅适用于社交软件，任何桌面应用的自动化都可以套用。在后续章节中，我们会看到更多类似的案例。